

Eclipse, OpenOCD, OpenJTAG 嵌入式开发教程



1个命令2秒钟即可在裸板上启动u-boot
再加3个命令即可烧写单板

官方网站: <http://www.100ask.net>

技术支持: QQ(17653039), MSN/Email(thisway.diy@163.com)

联系购买: <http://www.100ask.net/shop/contactus.html>

关键字: USB 转 JTAG, USB 转串口, 在线调试, Eclipse, OpenOCD, OpenJTAG, 教程, 改造 u-boot(支持 Nor/Nand Flash 启动, 烧写 yaffs 映象, USB/网络传输)的完全源码

Eclipse,OpenOCD,OpenJTAG嵌入式开发教程.....	1
1. 调试工具简介	3
1.1 OpenJTAG套件简介	3
1.2 嵌入式软件的交叉开发系统.....	5
2. Windows下OpenJTAG套件的使用	9
2.1 硬件、软件安装.....	9
2.1.1 安装OpenJTAG驱动程序.....	9
2.1.2 安装OpenOCD、交叉编译工具链、Eclipse.....	11
2.2 使用OpenOCD、OpenJTAG烧写程序、调试程序.....	13
2.2.1 启动OpenOCD, OpenOCD常用命令.....	13
2.2.2 使用OpenOCD烧写小程序到内部RAM, 并运行、调试.....	17
2.2.3 使用OpenOCD下载u-boot, 通过u-boot烧写Nor/Nand Flash	20
2.3 使用Eclipse进行开发	23
2.3.1 启动Eclipse, 进行简单设置:	23
2.3.2 新建一个Eclipse工程:	24
2.3.3 配置Eclipse工程:	27
2.3.4 编译、清除程序:	30
2.3.5 使用Eclipse调试程序: 以leds、u-boot为例	31
3. Ubuntu下OpenJTAG套件的使用.....	45
3.1 硬件、软件安装.....	45
3.1.1 修改udev规则, 使得识别OpenJTAG后自动加载驱动	45
3.1.2 安装OpenOCD、GDB、交叉编译工具链、Eclipse.....	46
3.2 使用OpenOCD、OpenJTAG 烧写程序、调试程序	49
3.2.1 启动OpenOCD, OpenOCD常用命令.....	49
3.2.2 使用OpenOCD烧写小程序到内部RAM, 并运行、调试.....	52
3.2.3 使用OpenOCD下载u-boot, 通过u-boot烧写Nor/Nand Flash	56
3.3 使用Eclipse进行开发.....	59
3.3.1 启动Eclipse, 进行简单设置:	59
3.3.2 新建一个Eclipse工程:	60
3.3.3 配置Eclipse工程:	64
3.3.4 编译、清除程序:	66
3.3.5 使用Eclipse调试程序: 以leds、u-boot、Linux内核为例.....	67
附录: u-boot使用简要说明	81

1. 调试工具简介

1.1 OpenJTAG 套件简介

在嵌入式开发中，有很多优秀的调试、仿真工具，比如 Keil、IAR、Rowley Associates 等。它们的安装、使用都很便利，功能强大，但是价格昂贵(几百美元甚至更多)；还要购买相应的硬件，比如 J-Link、U-Link 等 USB 到 JTAG 的转换盒，这也是一笔不小的开支。

对于学生，或者是开发预算有限的工程师来说，完全可以使用免费的开发工具 Eclipse、OpenOCD，然后通过一些便宜的 JTAG 转接器(比如并口 JTAG 等)就可以达到接近、甚至超越上述商业软件的效果。

并口 JTAG 速率太低，一般很少使用。本教程使用 OpenJTAG 转换器，配合 Eclipse、OpenOCD 等开源软件，就可以完成下载、烧写、调试等任务。

OpenJTAG 的外观及性能见图 1.1、表 1.1，它含有 3 大功能：USB 转串口，USB 转 JTAG，在线调试。无论是台式机，还是缺乏串口、并口的笔记本，都可以使用。



图 1.1 OpenJTAG 的外观

表 1.1

OpenJTAG 套件的性能

硬件特点	
USB	USB2.0 全速接口
	使用 USB 电源
	即插即用
JTAG	IEEE 1149.1 标准
	最大速率 6Mbits/sec
	是并口 JTAG 的 150 倍
	Multi-ICE 10-pin 插座
	Multi-ICE 20-pin 插座
	目标系统 3.3V
串口	RS232 标准
	7 或 8 位数据位
	1 或 2 位停止位
	奇校验/偶校验/标志位/空位/无校验
	最大波特率 235kbps
在线调试功能	支持的多种 CPU 系列: ARM7, ARM9, Cortex-M3, XSCALE
软件特点	
• Eclipse 集成开发环境的安装程序及教程(Linux/Windows 版)	
• 在线调试器 OpenOCD 的安装程序及教程(Linux/Windows 版)	
• 实时调试功能: 单步、全速运行、复位、软/硬件断点、跳转等	
• CPU 寄存器、存储器、变量观察窗口: 动态变化, 实时察看	
• 支持 ARM 内置的 Cache 和 MMU 功能	
• 在线烧写多种 Nor Flash 和 Nand Flash	
• 驱动程序: 支持 Linux 2.4 或更高版本, Windows ME, 2000, Server 2003, XP	

1.2 嵌入式软件的交叉开发系统

一套完整的交叉开发系统从上到下，可以分为 4 个部件：集成开发环境(IDE, Integrated Development Environment, 比如 Eclipse)、交叉编译工具链、一个接收各种操作命令(复位、暂停、读/写等)的守护进程(Daemon program, 如 OpenOCD)、用来连接 PC 与单板的 JTAG 接口板。

它们的联系可以用图 1.2 来表示，下面详细介绍。

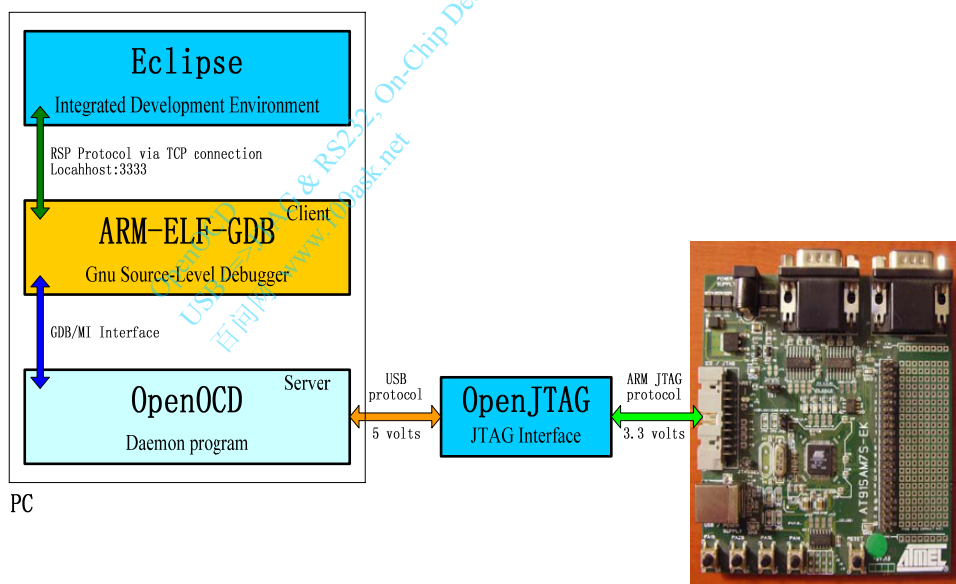


图 1.2 交叉开发系统的 4 层结构

交叉编译工具链是嵌入式开发中最核心的工具，它被用来编译、链接、调试程序。在集成开发环境出现之前，我们通常是先使用其他编辑工具(比如记事本、Source Insight 等)写好代码，然后通过命令行进行编译、链接。

集成开发环境将各种命令“封装”起来，通过各种按钮就可以编译、调试程序；它还提供一个现代化的软件编辑器。Eclipse 的操作界面如图 1.3 所示。

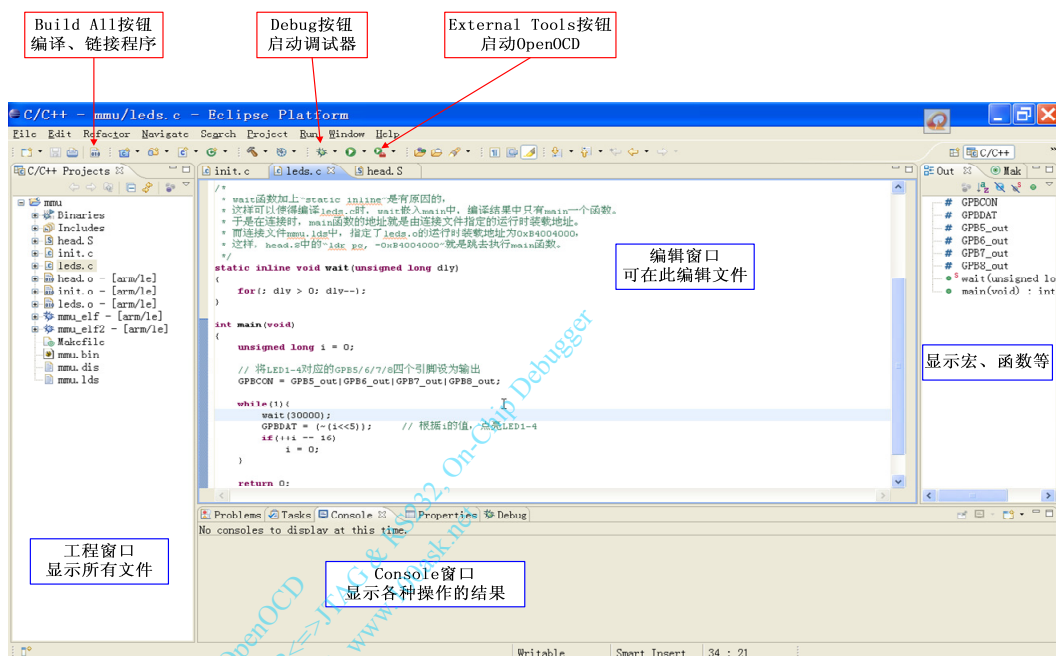


图 1.3 Eclipse 的操作界面

下面以一个例子来看看 Eclipse 是如何与交叉编译工具链协同，来编译、链接程序的。

在图 1.3 的“工程窗口”中有一个“Makefile”文件，Eclipse 根据它来调用交叉编译工具链。Makefile 内容如下：

```
objs := head.o init.o leds.o

all : $(objs)
    arm-elf-ld -Tmmu.lds -o mmu_elf $^
    arm-elf-objcopy -O binary -S mmu_elf mmu.bin
    arm-elf-objdump -D -m arm mmu_elf > mmu.dis

%.o:%.c
    arm-elf-gcc -Wall -g -c -o $@ $<

%.o:%.S
    arm-elf-gcc -Wall -g -c -o $@ $<

clean:
    rm -f mmu.bin mmu_elf mmu.dis *.o
```

当在 Eclipse 里按下“Build All”、“Build Project”按钮时，就相当于在命令行里执行“make all”命令，它根据 Makefile 的指示编译出 head.o、init.o、leds.o，然后链接成 elf 格式的可执行文件 mmu_elf，最后制作二进制可执行文件 mmu.bin、制作反汇编文件 mmu.dis。

当点击 Eclipse 的菜单 “Project -> Clear...” 时，就相当于在命令行里执行 “make clean” 命令，它根据 Makefile 的指示删除 mmu.bin、mmu_elf、mmu.dis、head.o、init.o、leds.o。

当在 Eclipse 里按下 “External Tools” 按钮时，它将根据当前工程的配置信息启动 OpenOCD，这将在后面说明。

当在 Eclipse 里按下 “Debug” 按钮时，它将根据当前工程的配置信息启动 GNU 源码级别的调试器 arm-elf-gdb(Windows 下)或 arm-linux-gdb(Linux)下，这将在后面说明。

可见，Eclipse 就是一个提供友好操作界面的图形程序，它最终还是根据 Makefile 来编译、链接、清除程序，根据配置信息启动守护进程 OpenOCD、调试器 (arm-elf-gdb.exe 或 arm-linux-gdb) 来调试程序。

Eclipse 的作用并不仅仅是提供按钮来启动其他程序，它的更大用处在于使调试程序更加便利：直接在源码中双击即可设置断点、将鼠标放到某个变量上即可看到它的值、查看内存、查看 CPU 寄存器、点击按钮即可暂停/重新运行程序等等。其实这些功能也是调试器 (arm-elf-gdb.exe 或 arm-linux-gdb) 实现的，但是 Eclipse 再次把它们 “封装” 起来，使得 “傻瓜都会用”。

下面介绍调试程序时，这 4 个部件的相互作用。

使用 Eclipse 进行调试时，它会自动启动调试器 (arm-elf-gdb.exe 或 arm-linux-gdb)。请参考图 1.2，Eclipse 使用 GDB/MI 协议与调试器进行通信。

举个例子，当你把鼠标放在 i 上时，Eclipse 就会发出 “print i” 的命令给 GDB，如图 1.4 所示。

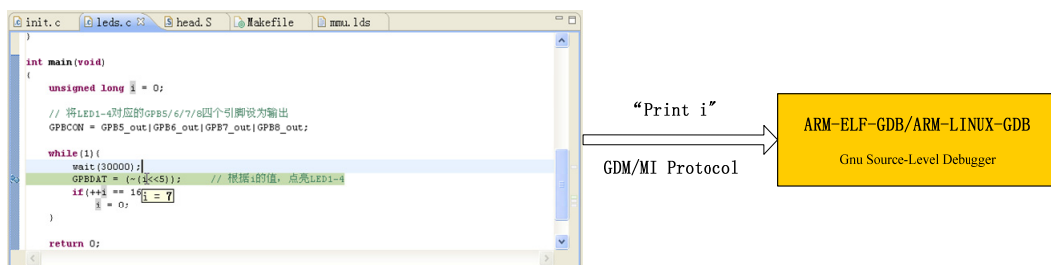


图 1.4 Eclipse 与调试器的交互

调试器根据前面编译出来的 mmu_elf 文件 (里面包含指令和符号信息) 得知变量 i 的类型是 “unsigned long”，地址是 0xb2ffffec。于是它向守护进程 OpenOCD 发出一个 “读内存” 的命令 “\$m0xb2ffffec,4#cs”，这个命令遵循 RSP 协议 (Remote Serial Protocol)，表示从内存地址 0xb2ffffec 处读出 4 字节，如图 1.5 所示。

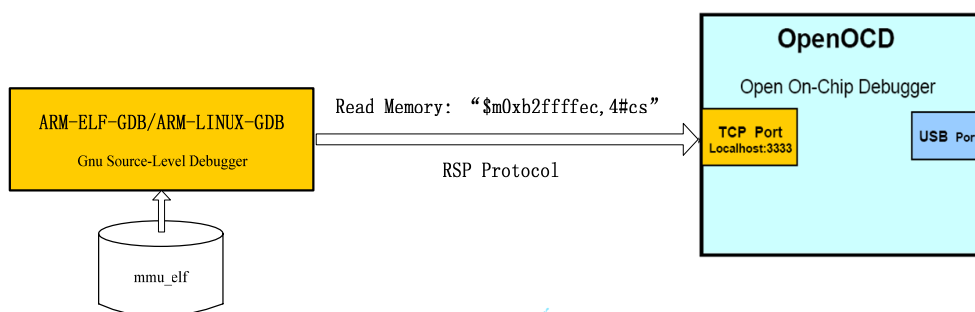


图 1.5 调试器与 OpenOCD 的交互

调试器 OpenOCD 接收到 GDB 的命令后，将它通过 USB 协议发送给 JTAG 接口板，如图 1.6 所示。

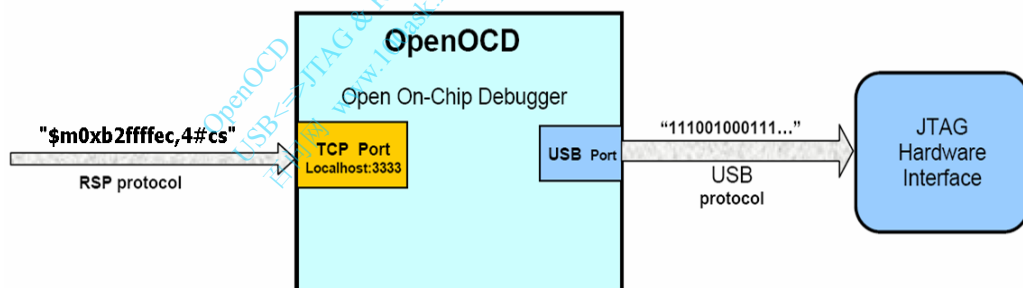


图 1.6 OpenOCD 与 JTAG 接口板的交互

最后一步就是 JTAG 接口板与 CPU 的交互了。ARM7、ARM9 等 CPU 核里有一个被称为“嵌入式 ICE 宏单元” (Embedded ICE macrocell) 的部件，借助它可以对系统进行实时仿真和跟踪调试，要通过 JTAG 接口来访问它。JTAG 接口板完成 USB 协议到 JTAG 协议的转换，如图 1.7 所示。

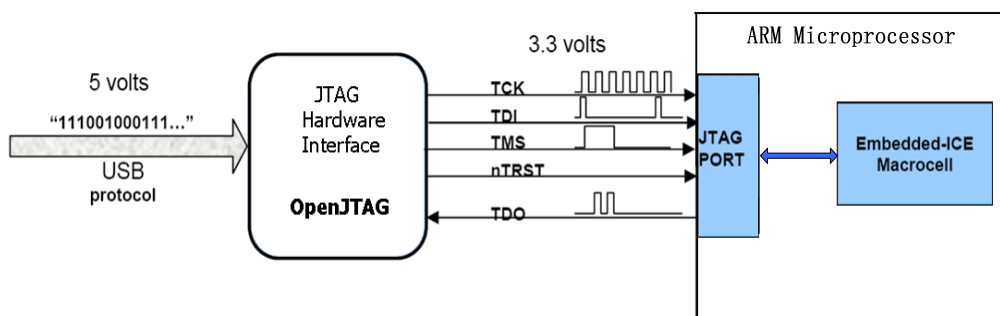


图 1.7 JTAG 接口板与 CPU 的交互

2. Windows 下 OpenJTAG 套件的使用

2.1 硬件、软件安装

如果只是想把 OpenJTAG 当作 USB 转串口卡来使用，可以只看 2.1.1 小节。

2.1.1 安装 OpenJTAG 驱动程序

将 OpenJTAG 板子接到 PC 的 USB 口后，Windows 会检测到新硬件，如图 2.1 所示。选择“从列表或指定位置安装(高级)(S)”，然后点击下一步。



图 2.1 安装 OpenJTAG 驱动程序(第 1 步)

假设 OpenJTAG 光盘是 G 盘，如图 2.2 所示，在新出现的窗口中指定搜索驱动程序的位置。

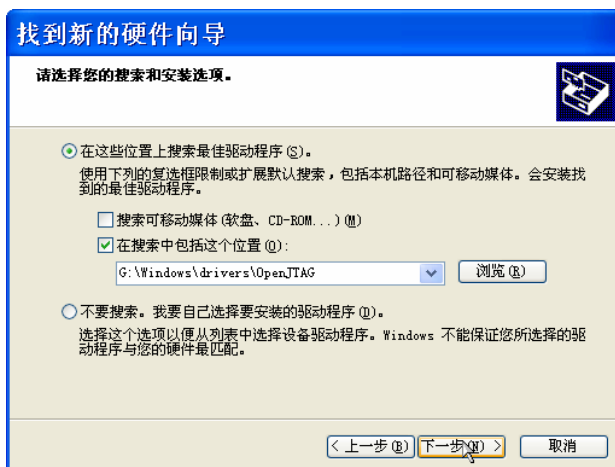


图 2.2 安装 OpenJTAG 驱动程序(第 2 步)

在图 2.2 中点击下一步后，将出现如图 2.3 所示的界面，表示找到了驱动程序，点击“仍然继续”。



图 2.3 安装 OpenJTAG 驱动程序 (第 3 步)

等当前驱动程序安装完后，会重复出现两次类似图 1.8 的界面，按照图 2.1~图 2.3 再次安装就可以了——第 1 次是“USB<=>JTAG&RS232”，第 2 次是“100ASK Serial Port”，第 3 次是“USB Serial Port”，安装方法完全一样。

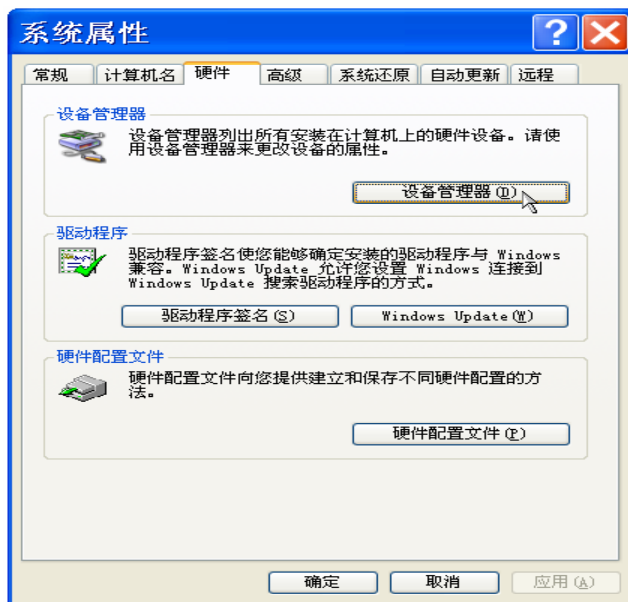


图 2.4 打开设备管理器

现在在桌面“我的电脑”上单击右键，选择“属性”，在如图 2.4 所示的“硬件”TAB 页中，点击“设备管理器(D)”。

在设备管理器中，点开“端口(COM 和 LPT)”，可以发现新增了一个串口 COM4(不同的 PC, 序号可能不同)；点开“通用串行总线控制器”，可以发现新增了 3 个设备：100ASK JTAG、100ASK Serial Port、USB Composite Device。如图 2.5 所示。

如果只是把 OpenJTAG 当作 USB 转串口卡使用，现在就可以像普通串口一样使用这个新增的串口，后面的章节不需要看了。

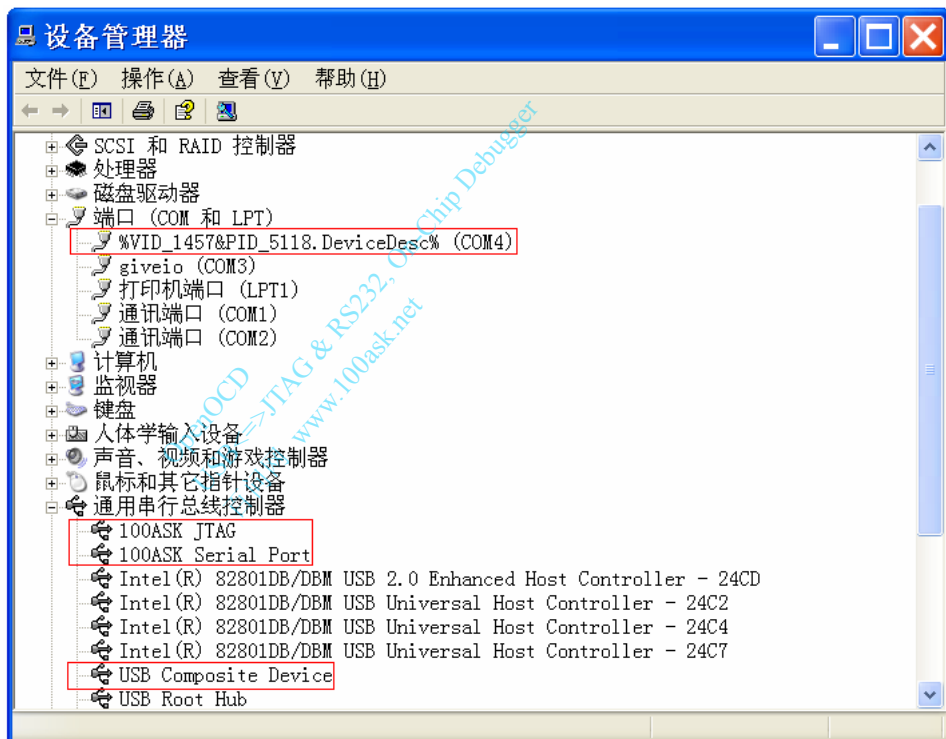


图 2.5 新增加的设备

2.1.2 安装 OpenOCD、交叉编译工具链、Eclipse

在光盘 Windows\install 目录下有如图 2.6 所示的 5 个文件。

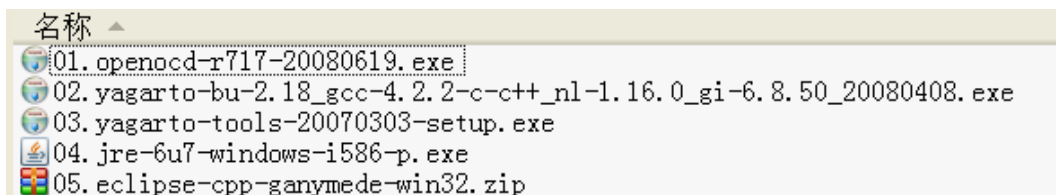


图 2.6 Windows 下的 5 个安装程序

按照文件名的序号安装即可。

只是有两点要**注意**：

- (1) 如果系统中已经安装了高于 1.4.2 版本的 java 运行环境，就不用安装第 4 个文件 04.jre-6u7-windows-i586-p.exe 了。

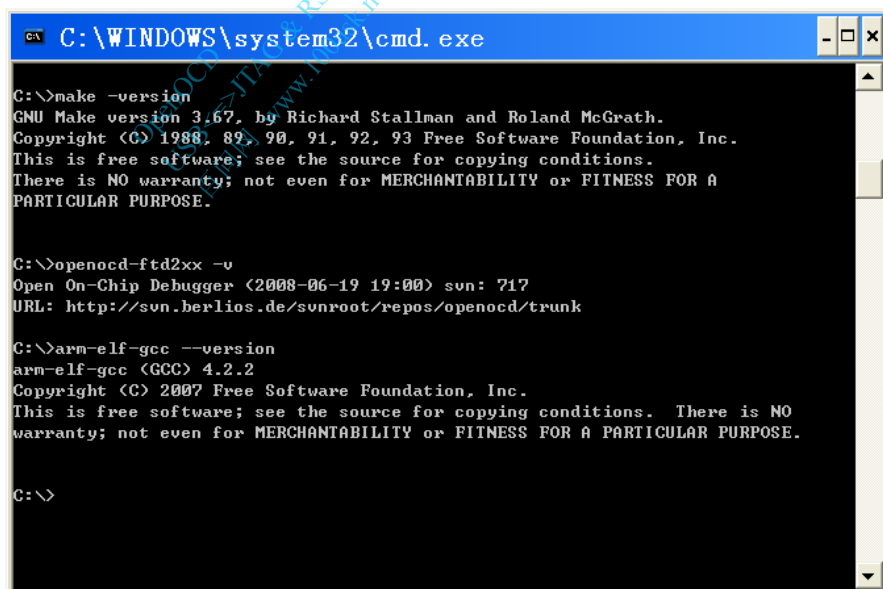
可以在命令行中运行“java -version”命令确认是否已经安装了 java 运行环境及它的版本，如图 2.7 所示：如果这个命令出错或是版本不大于 1.4.2，则需要安装 04.jre-6u7-windows-i586-p.exe。

```
C:\>java -version
java version "1.6.0_07"
Java(TM) SE Runtime Environment (build 1.6.0_07-b06)
Java HotSpot(TM) Client VM (build 10.0-b23, mixed mode, sharing)
```

图 2.7 确认 java 运行环境的版本

(2) 05.eclipse-cpp-ganymede-win32.zip 是 Eclipse 压缩包，解压即可，无需安装即可使用。建议将它解压到 C:\Program Files 目录下。

现在来确定一下安装是否成功，在命令行中执行“make -version”、“openocd-ftd2xx -v”、“arm-elf-gcc --version”，如图 2.8 所示。如果这些命令没有出错，则安装成功。



```
C:\WINDOWS\system32\cmd.exe

C:\>make -version
GNU Make version 3.67, by Richard Stallman and Roland McGrath.
Copyright (C) 1988, 89, 90, 91, 92, 93 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.
There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
PARTICULAR PURPOSE.

C:\>openocd-ftd2xx -v
Open On-Chip Debugger (2008-06-19 19:00) svn: 717
URL: http://svn.berlios.de/svnroot/repos/openocd/trunk

C:\>arm-elf-gcc --version
arm-elf-gcc (GCC) 4.2.2
Copyright (C) 2007 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

C:\>
```

图 2.8 确认程序是否安装成功

2.2 使用 OpenOCD、OpenJTAG 烧写程序、调试程序

对于 S3C2410、S3C2440 开发板，大多使用 sjf2410.ext、sjf2440.exe 等工具通过并口 JTAG 接口板烧写程序，并口 JTAG 的速率非常低，烧写 128K 的文件至少耗时 10 多分钟。

后来又出现了 H-JTAG 工具，它也是使用并口 JTAG 接口板并且速度很快，但是目前只支持烧写 Nor Flash，不支持 Nand Flash。

使用 OpenOCD，既可以烧写 Nor Flash，也可以烧写 Nand Flash，还可以进行简单的调试。

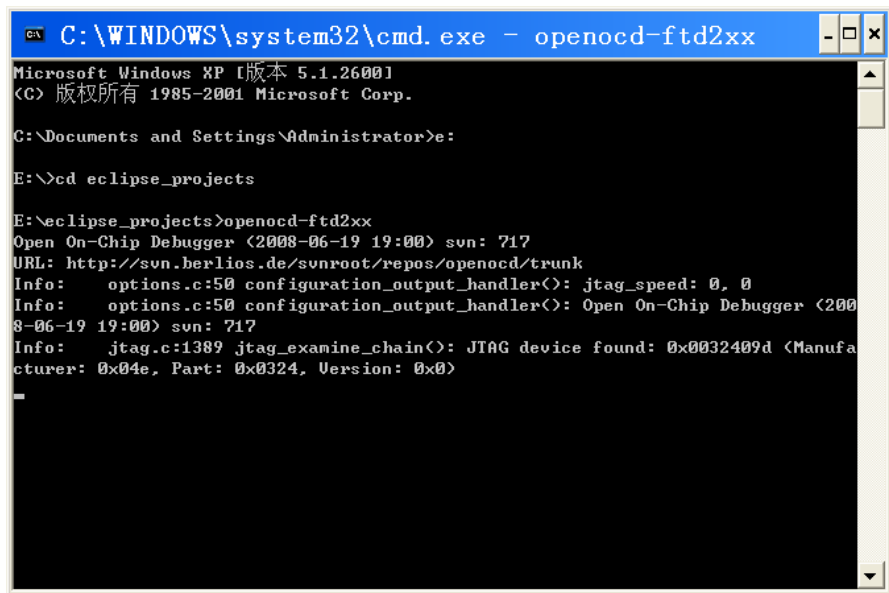
注意：OpenOCD 目前只支持 CFI 接口的 Nor Flash，但是可以通过 OpenOCD 将 u-boot 下载到单板内存后，启动 u-boot 来烧写 Flash，它支持 CFI 接口、JEDEC 接口的 Nor Flash，当然也支持 Nand Flash。

本节都假设工作目录在 E:\eclipse_projects 目录下。假设已经把光盘上的 Windows\examples 目录下的所有文件夹、文件都复制到了 E:\eclipse_projects 目录下。

2.2.1 启动 OpenOCD，OpenOCD 常用命令

把开发板的 JTAG 口、串口都接到 OpenJTAG 板上去；然后给开发板供电(注意：使用 JTAG 线连接 OpenJTAG 与开发板时，开发板一定要上电)；最后使用 USB 线连接 OpenJTAG 和计算机。

openocd-ftd2xx 的运行需要一些配置信息，这些配置信息写在 E:\eclipse_projects\openocd.cfg 文件中，在命令行里进入 E:\eclipse_projects 目录，执行 openocd-ftd2xx 命令即可启动 OpenOCD，如图 2.9 所示。



```
C:\WINDOWS\system32\cmd.exe - openocd-ftd2xx
Microsoft Windows XP [版本 5.1.2600]
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator>e:

E:\>cd eclipse_projects

E:\eclipse_projects>openocd-ftd2xx
Open On-Chip Debugger (2008-06-19 19:00) svn: 717
URL: http://svn.berlios.de/svnroot/repos/openocd/trunk
Info: options.c:50 configuration_output_handler(): jtag_speed: 0, 0
Info: options.c:50 configuration_output_handler(): Open On-Chip Debugger (2008-06-19 19:00) svn: 717
Info: jtag.c:1389 jtag_examine_chain(): JTAG device found: 0x0032409d (Manufacturer: 0x04e, Part: 0x0324, Version: 0x0)
-
```

图 2.9 在 Windows 下启动 OpenOCD

注意：openocd.cfg 中有一行“#arm7_9 fast_memory_access enable”，如果不是在 VMware 中使用 OpenOCD，可以把它前面的“#”号去掉，这将把 OpenOCD 的下载速度增加 6、7 倍。

如果 `openocd-ftd2xx` 运行出错，请确认开发板是否已经供电。如果供电正常，则先拔下 OpenJTAG 板的 USB 线、与开发板相连的 JTAG 线，然后重新接上 JTAG 线、USB 线，最后重新启动 `openocd-ftd2xx`。

OpenOCD 启动后，在另一个命令行中执行“`telnet localhost 4444`”即可登录 OpenOCD，以后就可以在 telnet 界面操作 OpenOCD 了：包括读/写开发板内存，烧写 Flash，使用命令行的方式调试程序(使用图形界面调试程序需要使用 Eclipse)。

telnet 界面如图 2.10 所示。

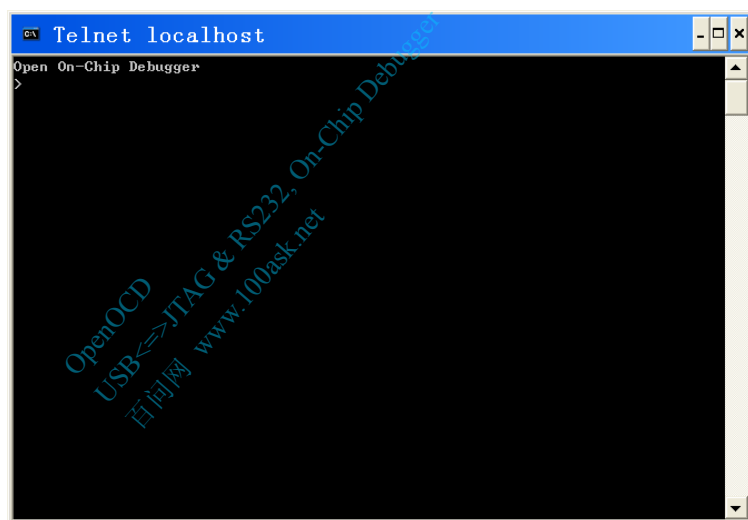


图 2.10 Windows 下 telnet 登录 OpenOCD 的界面

OpenOCD 的使用文档是“开始 -> 程序 -> OpenOCD -> OpenOCD Documentation”，表 2.1 中列出常用命令。

表 2.1

OpenOCD 常用命令

目标板状态处理命令(Target state handling)	
poll	查询目标板当前状态
halt	中断目标板的运行
resume [address]	恢复目标板的运行，如果指定了 address，则从 address 处开始运行
step [address]	单步执行，如果指定了 address，则从 address 处开始执行一条指令
reset	复位目标板
断点命令	
bp <addr> <length> [hw]	在地址 addr 处设置断点，指令长度为 length，hw 表示硬件断点
rbp <addr>	删除地址 addr 处的断点
内存访问指令(Memory access commands)	
mdw <addr> [count]	显示从地址 addr 开始的 count(缺省是 1)个字(4 字节)
mdh <addr> [count]	显示从地址 addr 开始的 count(缺省是 1)个半字(2 字节)
mdb <addr> [count]	显示从地址 addr 开始的 count(缺省是 1)个字节
mww <addr> <value>	向地址 addr 写入一个字，值为 value
mwh <addr> <value>	向地址 addr 写入一个半字，值为 value
mwb <addr> <value>	向地址 addr 写入一个字节，值为 value
load_image <file> <address> ['bin' 'ihex' 'elf'] 将文件<file>载入地址为 address 的内存，格式有'bin'、'ihex'、'elf'	
dump_image <file> <address> <size> 将内存从地址 address 开始的 size 字节数据读出，保存到文件<file>中	
verify_image <file> <address> ['bin' 'ihex' 'elf'] 将文件<file>与内存 address 开始的数据进行比较，格式有'bin'、'ihex'、'elf'	
CPU 架构相关命令(Architecture Specific Commands)	
armv4 reg	打印寄存器的值
arm7_9 sw_bkpts <enable disable> 使能或禁止软件断点，在 ARMv4 系统中有两个硬件断点，软件断点将占用一个硬件断点	
arm920t cp15 <num> [value]	显示/修改 cp15 协处理器的寄存器
arm920t md<bhw>_phys <addr> [count]	显示物理地址 addr 处的内存
arm920t mw<bhw>_phys <addr> <value>	修改物理地址 addr 处的内存
arm920t virt2phys <va>	获得虚拟地址 va 对应的物理地址
其他命令	
script <file>	执行 file 文件中的命令

下面试验一下，S3C2410 或 S3C2440 从 Nand 启动时，内部内存地址是 0；从 Nor Flash 启动时，内部内存地址是 0x40000000。以下假设从 Nand 启动，您可以按照图 2.11 操作一下。

```

C:\ Telnet localhost
Open On-Chip Debugger
> poll
target state: halted
target halted in ARM state due to debug request, current mode: Undefined
cpsr: 0xf00000db pc: 0x00000004
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> mdw 0x0
0x00000000: 56190527
> mww 0x0 0x12345678
> mdw 0x0
0x00000000: 12345678
> armv4_5 reg
r0: 7d7fbfb0 r0: 7d7fbfb0 r0: 7d7fbfb0 r0: 7d7fbfb0
r1: fff69f7e r1: fff69f7e r1: fff69f7e r1: fff69f7e
r2: ffffffff r2: ffffffff r2: ffffffff r2: ffffffff
r3: 963ffede r3: 963ffede r3: 963ffede r3: 963ffede
r4: bff697ff r4: bff697ff r4: bff697ff r4: bff697ff
r5: b7d7fbfb r5: b7d7fbfb r5: b7d7fbfb r5: b7d7fbfb
r6: d95bdf7f r6: d95bdf7f r6: d95bdf7f r6: d95bdf7f
r7: ffe7dd7f r7: ffe7dd7f r7: ffe7dd7f r7: ffe7dd7f
r8: 57efdebf r8_fiq: ffdabddf r8: 57efdebf r8: 57efdebf
r9: 747bffde r9_fiq: 6ab9ffdb r9: 747bffde r9: 747bffde
r10: eff96f3f r10_fiq: d4ehea9e r10: eff96f3f r10: eff96f3f
r11: bfeff76f r11_fiq: ebcdfec r11: bfeff76f r11: bfeff76f
r12: fe23efea r12_fiq: f7dfbfb r12: fe23efea r12: fe23efea
r13_usr: 7efffd7b r13_fiq: bff7fdff r13_irq: ecf7ffd7 r13_svc: e5e5e5e5 r13
abt: 77db6bff r13_und: e5e5e5e5
lr_usr: d7fcf7ee lr_fiq: b6f7dbf7 lr_irq: be7bffff lr_svc: dfd7f6ef lr
abt: 9f757fff lr_und: 00000028
pc: 00000004 pc: 00000004 pc: 00000004 pc: 00000004
cpsr: f00000db spsr_fiq: 40000010 spsr_irq: 10000032 spsr_svc: 700000ff spsr
abt: 0000003c spsr_und: f00000db
>
  
```

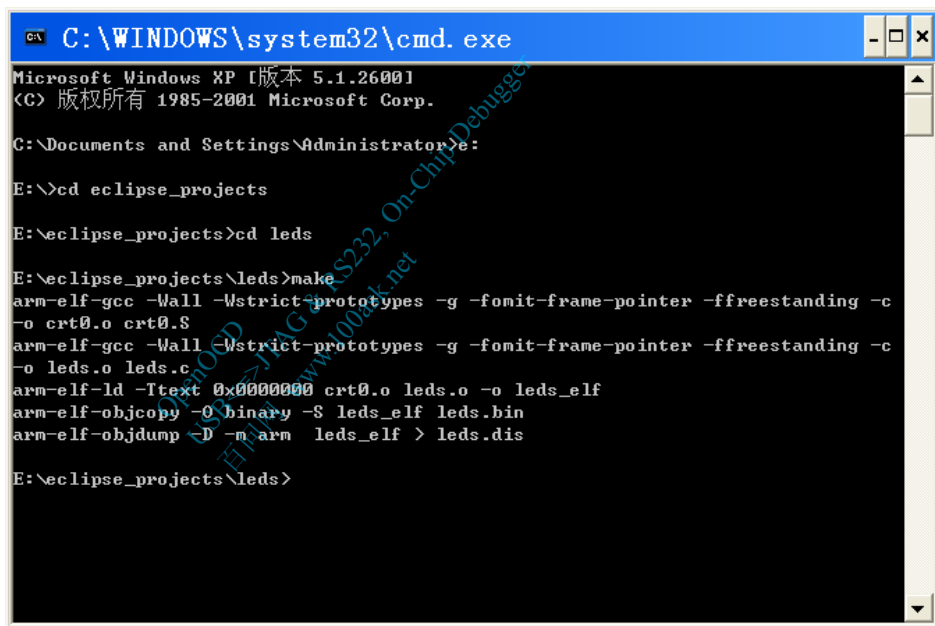
图 2.11 OpenOCD 命令操作示例

2.2.2 使用 OpenOCD 烧写小程序到内部 RAM，并运行、调试

S3C2410、S3C2440 有 4K 的内部 RAM，当程序小于 4K 时，可以通过 OpenOCD 直接把程序烧到内部 RAM 中、运行、调试。

(1) 编译程序：

以光盘上的 E:\eclipse_projects\examples\leds 程序为例，先编译程序：在命令行进入文件源码所在目录，执行“make”命令，如图 2.12 所示。

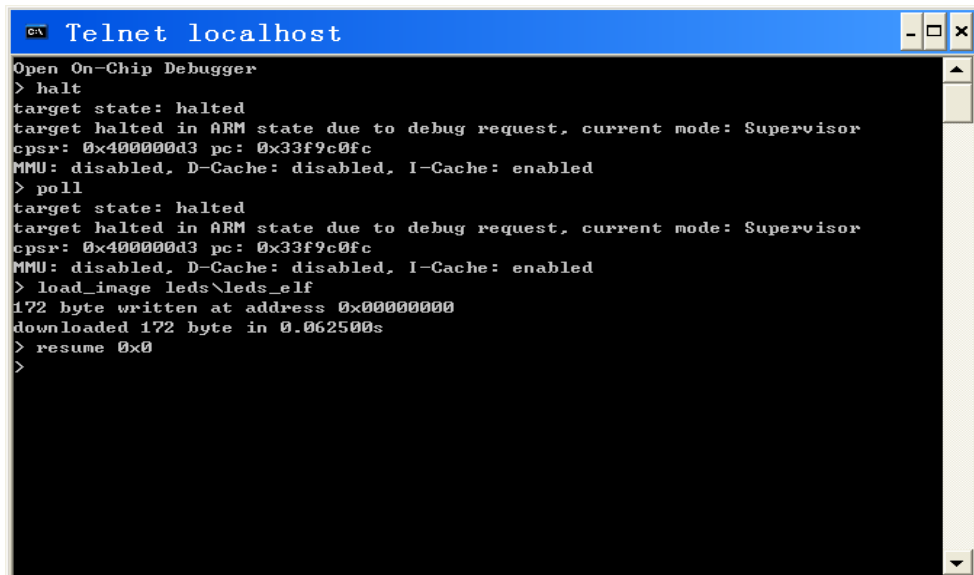


```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [版本 5.1.2600.1
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator\e:
E:\>cd eclipse_projects
E:\eclipse_projects>cd leds
E:\eclipse_projects\leds>make
arm-elf-gcc -Wall -Wstrict-prototypes -g -fomit-frame-pointer -ffreestanding -c
-o crt0.o crt0.S
arm-elf-gcc -Wall -Wstrict-prototypes -g -fomit-frame-pointer -ffreestanding -c
-o leds.o leds.c
arm-elf-ld -Ttext 0x00000000 crt0.o leds.o -o leds_elf
arm-elf-objcopy -O binary -S leds_elf leds.bin
arm-elf-objdump -D -m arm leds_elf > leds.dis
E:\eclipse_projects\leds>
```

图 2.12 在命令行编译程序

(2) 下载、运行程序：



```
Telnet localhost
Open On-Chip Debugger
> halt
target state: halted
target halted in ARM state due to debug request, current mode: Supervisor
cpsr: 0x400000d3 pc: 0x33f9c0fc
MMU: disabled, D-Cache: disabled, I-Cache: enabled
> poll
target state: halted
target halted in ARM state due to debug request, current mode: Supervisor
cpsr: 0x400000d3 pc: 0x33f9c0fc
MMU: disabled, D-Cache: disabled, I-Cache: enabled
> load_image leds\leds_elf
172 byte written at address 0x00000000
downloaded 172 byte in 0.062500s
> resume 0x0
>
```

图 2.13 使用 OpenOCD 加载、运行程序

注意：下载程序之前先使用“halt”命令暂停单板，才能下载代码；如果使用“poll”命令发现单板的 MMU 或 D-cache 已经使能，则需要使用“arm920t cp15 2 0”命令禁止 MMU 和 D-cache。

在 telnet 界面，先执行“load_image leds\leds_elf”加程序，它会根据 leds_elf 里面的信息把可执行代码加载到地址为 0 的内存处，“load_image leds\leds.bin 0x0”可以达到同样的效果；然后执行“resume 0x0”命令执行程序。如图 2.13 所示。

(3) 简单的调试命令：

运行 leds 程序后，可以使用“halt”命令暂停程序，使用“resume”命令恢复运行程序，使用“arm7_9 sw_bkpts enable”命令使能软件断点，使用 bp 命令设置断点，使用 rbp 命令删除断点。如图 2.14 所示。

```

Open On-Chip Debugger
> load_image leds\leds_elf 加载代码到内部RAM中
172 byte written at address 0x00000000
downloaded 172 byte in 0.031250s
> resume 0x0 启动程序
> halt 暂停程序
target state: halted
target halted in ARM state due to debug request, current mode: Undefined
cpsr: 0x200000db pc: 0x00000024
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> arm7_9 sw_bkpts enable 使能软中断
software breakpoints enabled
> bp 0x84 4 在地址0x84处设置断点
breakpoint added at address 0x00000084
> bp 查看断点
0x00000084, 0x4, 0, 0x00
> resume 恢复运行程序
target state: halted
target halted in ARM state due to breakpoint, current mode: Undefined
cpsr: 0x600000db pc: 0x00000084
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> resume 在断点处停下了，再次恢复运行程序
target state: halted
target halted in ARM state due to breakpoint, current mode: Undefined
cpsr: 0x600000db pc: 0x00000084
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> resume 在断点处停下了，再次恢复运行程序
target state: halted
target halted in ARM state due to breakpoint, current mode: Undefined
cpsr: 0x600000db pc: 0x00000084
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> rbp 0x84 删除断点
> bp 查看断点，无输出表示无断点
> resume 恢复运行程序，不会再停下了
>
  
```

图 2.14 在 telnet 界面操作 OpenOCD 调试程序

程序在断点停下时，可以看到 LED 也停止闪烁，恢复运行时 LED 就再次变化、再次停止。

至于为什么在 0x84 处设置断点，主要是想在 leds.c 中操作 GPBDA 寄存器以控制 LED 的代码处设置断点，如图 2.15 所示。

```
int main(void)
{
    unsigned long i = 0;

    GPBCON = GPB5_out|GPB6_out|GPB7_out|GPB8_out;    // 将LED1-4对应的GPB5/6/7/8四个引脚设为输出

    while(1){
        wait(30000);
        GPBDAT = ~(i<<5);    // 根据i的值，点亮LED1-4    想在此设置断点
        if(++i == 16)
            i = 0;
    }

    return 0;
}
```

图 2.15 leds.c 部分代码

可以从反汇编 leds.dis 中知道这段代码的地址在 0x84 处，如图 2.16 所示。

```
00000044 <main>:
44: e52de004 push {lr} ; (str lr, [sp, #-4]!)
48: e24dd004 sub sp, sp, #4 ; 0x4
4c: e3a03000 mov r3, #0 ; 0x0
50: e58d3000 str r3, [sp]
54: e3a03456 mov r3, #1442840576 ; 0x56000000
58: e2833010 add r3, r3, #16 ; 0x10
5c: e3a02b55 mov r2, #87040 ; 0x15400
60: e5832000 str r2, [r3]
64: e3a00c75 mov r0, #29952 ; 0x7500
68: e2800030 add r0, r0, #48 ; 0x30
6c: ebffffe9 bl 18 <wait>
70: e3a02456 mov r2, #1442840576 ; 0x56000000
74: e2822014 add r2, r2, #20 ; 0x14
78: e59d3000 ldr r3, [sp]
7c: e1a03283 lsl r3, r3, #5
80: e1e03003 mvn r3, r3
84: e5823000 str r3, [r2]    给GPBDA寄存器赋值
88: e59d3000 ldr r3, [sp]
8c: e2833001 add r3, r3, #1 ; 0x1
90: e58d3000 str r3, [sp]
94: e59d3000 ldr r3, [sp]
98: e3530010 cmp r3, #16 ; 0x10
9c: 1afffff0 bne 64 <main+0x20>
a0: e3a03000 mov r3, #0 ; 0x0
a4: e58d3000 str r3, [sp]
a8: eaafffed b 64 <main+0x20>
```

图 2.16 反汇编文件 leds.dis 部分内容

从反汇编代码确定程序功能，是件难度很高的事情，所以一般不直接操作 OpenOCD 来调试程序，而是通过其他工具来调试：

- ① 使用 arm-elf-gdb 调试器：可以指定在 leds.c 的某行设置断点；
- ② 使用 Eclipse 来调试：可以直接在源文件中通过双击设置断点。

2.2.3 使用 OpenOCD 下载 u-boot, 通过 u-boot 烧写 Nor/Nand Flash

首页所说的“1 个命令 2 秒钟即可在裸板上启动 u-boot”, 是通过在 telnet 界面上执行“script u-boot.cmd”命令实现的。u-boot.cmd 文件就是把下面(1)、(2)小节所描述的命令写在一起。

通过 OpenOCD 可以快速地读写开发板的 SDRAM, 那么可以把 u-boot 先放到 SDRAM 中, 然后启动它, 就可以使用 u-boot 来烧写 Nor/Nand Flash 了——u-boot 的功能及速度都比一般的烧写工具要强。

在 SDRAM 能被访问之前, 必须初始化存储控制器, 这可以通过下载一个特定的小程序到内部 RAM 中, 并运行它, 由它来进行初始化, 这个程序在 E:\eclipse_projects\init 目录下。

(1) 下载、运行初始化程序 init.bin:

注意: 下载程序之前先使用“halt”命令暂停单板, 才能下载代码; 如果使用“poll”命令发现单板的 MMU 或 D-cache 已经使能, 则需要使用“arm920t cp15 2 0”命令禁止 MMU 和 D-cache。

init.bin 是位置无关的代码, 所以无论是从 Nor Flash 启动, 还是从 Nand Flash 启动, 都可以将它下载到内部 RAM 去运行——不同启动方式下内部 RAM 的地址不同, Nor Flash 启动时, 地址是 0x40000000; Nand Flash 启动时, 地址是 0。

启动 OpenOCD(参考图 2.9)、telnet 登录(参考图 2.10)后, 执行以下命令下载、运行 init.bin:

① 从 Nand Flash 启动:

```
load_image init\init.bin 0x0
resume 0x0
```

② 从 Nor Flash 启动:

```
load_image init\init.bin 0x40000000
resume 0x40000000
```

这时, 存储控制器已经初始化好, 下面就可以把 u-boot 下载到 SDRAM 去了。

(2) 下载、运行 u-boot:

u-boot 的可执行文件为 E:\eclipse_projects\u-boot\u-boot.bin, 接着在 telnet 界面执行以下命令, 即可启动 u-boot:

```
halt
load_image u-boot\u-boot.bin 0x33f80000
resume 0x33f80000
```

然后, 就可以在串口工具上看到 u-boot 的启动信息, 如下:

```
U-Boot 1.1.6 (Sep 23 2008 - 02:50:03)

DRAM: 64 MB
Flash: 1 MB
NAND: 64 MiB
```

```

*** Warning - bad CRC, using default environment
In:    serial
Out:   serial
Err:   serial
Use these steps to program the image to flash:
1. In OpenOCD
    Run the 'halt' command to halt u-boot
    Run the 'load_image <file> <address>' command to load file to SDRAM
    Run the 'resume' command to resume u-boot
2. In u-boot, use the flash commands to program the image to flash
Or, use the tftp or nfs command to download file, and then program the flash.
OpenJTAG>

```

现在，可以通过 u-boot 来烧写文件了，有两种方法将文件下载到单板内存中：

① 通过 OpenOCD 的命令：

这需要在 telnet 界面，执行以下 3 个命令：

```

halt: 暂停u-boot
load_image <file> <address>: 将文件烧写到内存中(这个地址一般取0x30000000)
resume: 恢复运行u-boot

```

② 通过 u-boot 的网络命令：

可以使用 tftp 命令，或者 nfs 命令，这需要在 PC 上启动 TFTP 服务或者 NFS 服务。

当文件被下载到单板内存中后，就可以使用 u-boot 的各种 Flash 命令进行烧写了。图 2.17、图 2.18 是两个例子，分别使用上述两种下载方式，将 u-boot 本身烧写到 Nor Flash、Nand Flash 上去。

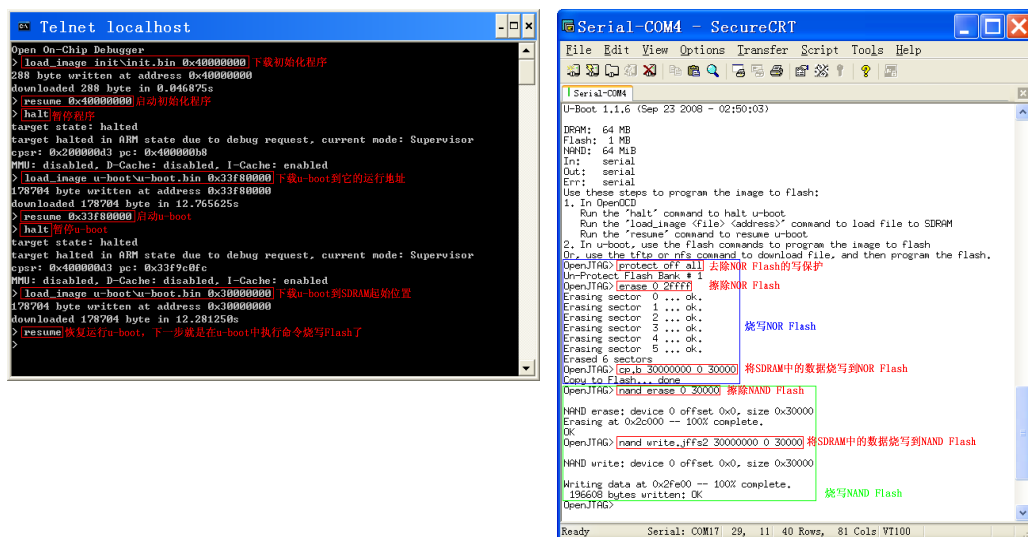


图 2.17 通过 OpenOCD 启动 u-boot、传输数据，通过 u-boot 烧写 Flash

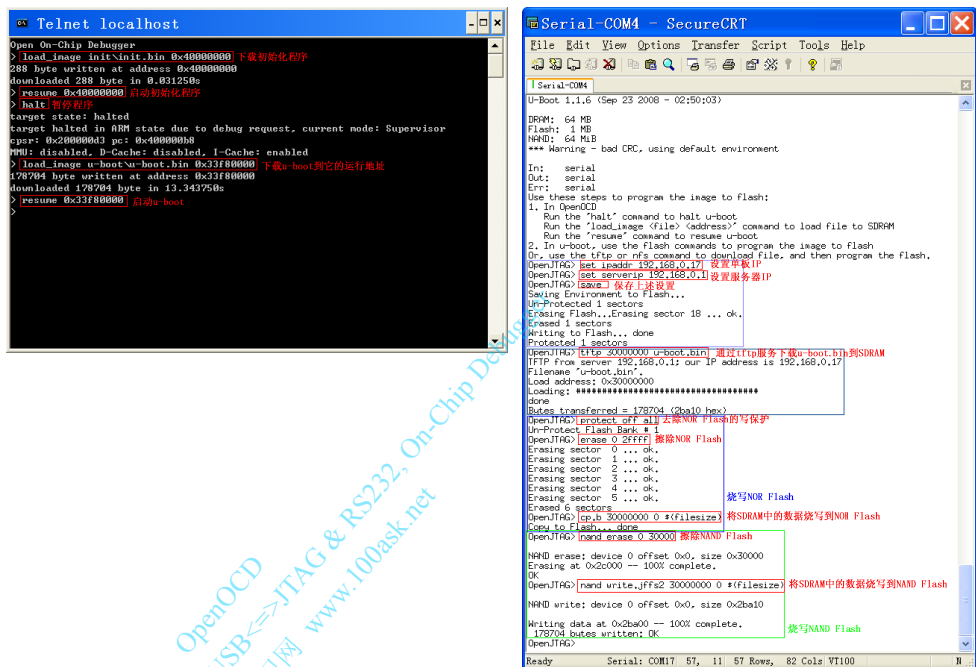


图 2.18 通过 OpenOCD 启动 u-boot，通过 u-boot 下载、烧写 Flash

注意：

- u-boot 的命令把所有的数字当作 16 进制，所以不管是否在数字前加前缀“0x”，这个数字都是 16 进制的。
- 图 1.29、图 1.30 中，擦除 Flash 的长度、烧写的数据长度，这些数值都是根据要烧写的文件的长度确定的。u-boot.bin 的长度是 178704 字节，即 0x2BA10 字节，上面使用的长度都是 0x30000，一是为了与 Flash 的可擦除长度相配(16K 的整数倍)，二是方便。

完成烧写之后，重启单板即可看到运行情况。

首页所说的“再加 3 个命令即可烧写单板”，就是指下载、擦除、烧写这 3 个命令。

E:\eclipse_projects\u-boot\u-boot.bin 是经过改造的，它有如下特点：

- 支持 S3C2410、S3C2440；
- 可以从 Nor Flash 或 Nand Flash 启动；
- 可以烧写 Nor Flash 和 Nand Flash；
- 支持 CS8900 或 DM9000 网卡、USB 传输(通过 OpenOCD 下载、启动时，USB 功能不可用)；
- 支持烧写 jffs2、yaffs 文件系统映像文件

注意：单板从 Nand Flash 启动时，无法操作 Nor Flash。

2.3 使用 Eclipse 进行开发

2.3.1 启动 Eclipse，进行简单设置：

在 Eclipse 的安装目录 C:\Program Files\eclipse 下，双击  eclipse.exe 启动 Eclipse，等待一会，可以看到如图 2.19 所示界面。在里面设置工作目录为 E:\eclipse_projects，并选择“Use this as the default and do not ask again”，表示使用 E:\eclipse_projects 作为缺省目录，并不再提示。

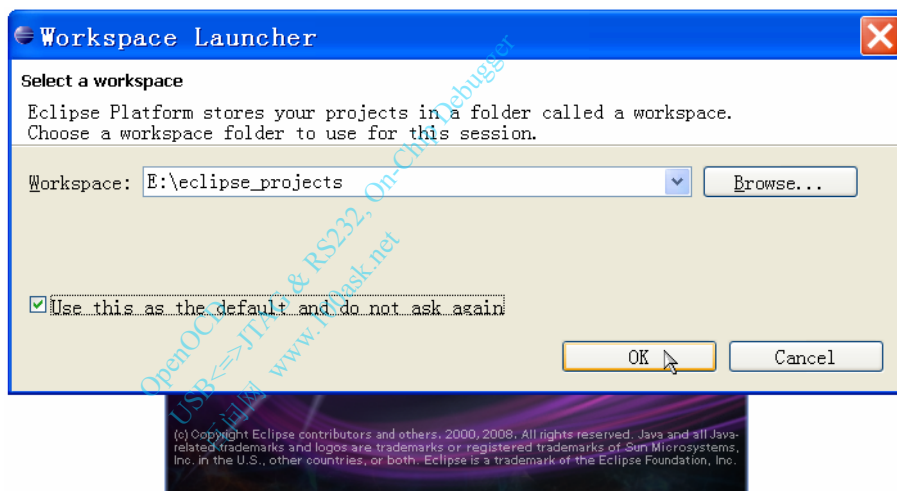


图 2.19 设置 Eclipse 工作目录

在图 2.19 中，点击 OK 后，得到如图 2.20 所示界面。

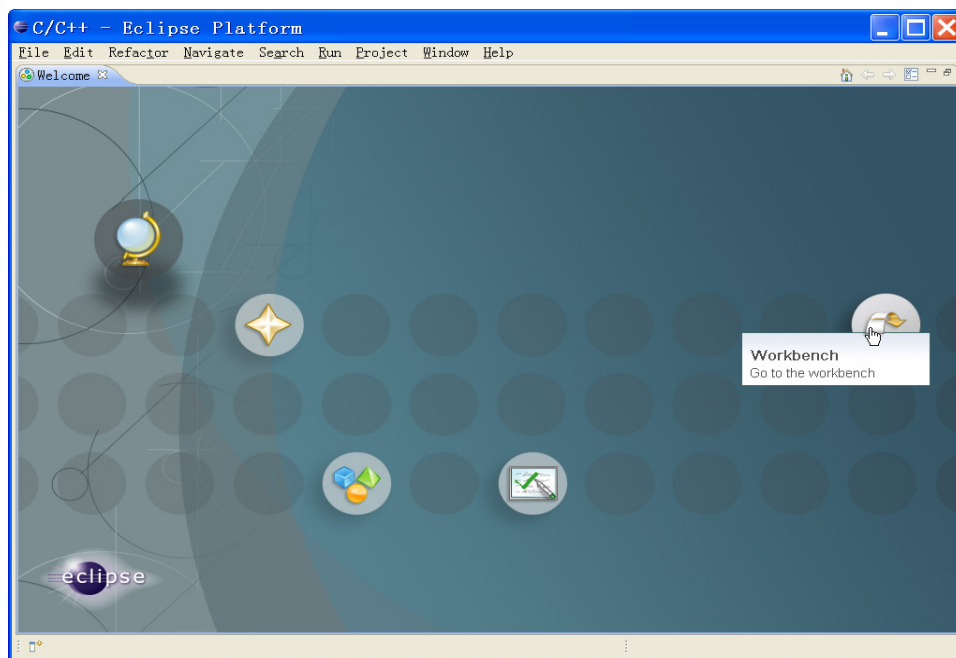


图 2.20 Eclipse 启动后的界面

图 2.20 中右边的图标名为 “Workbench”，点击它，得到图 2.21 所示界面。

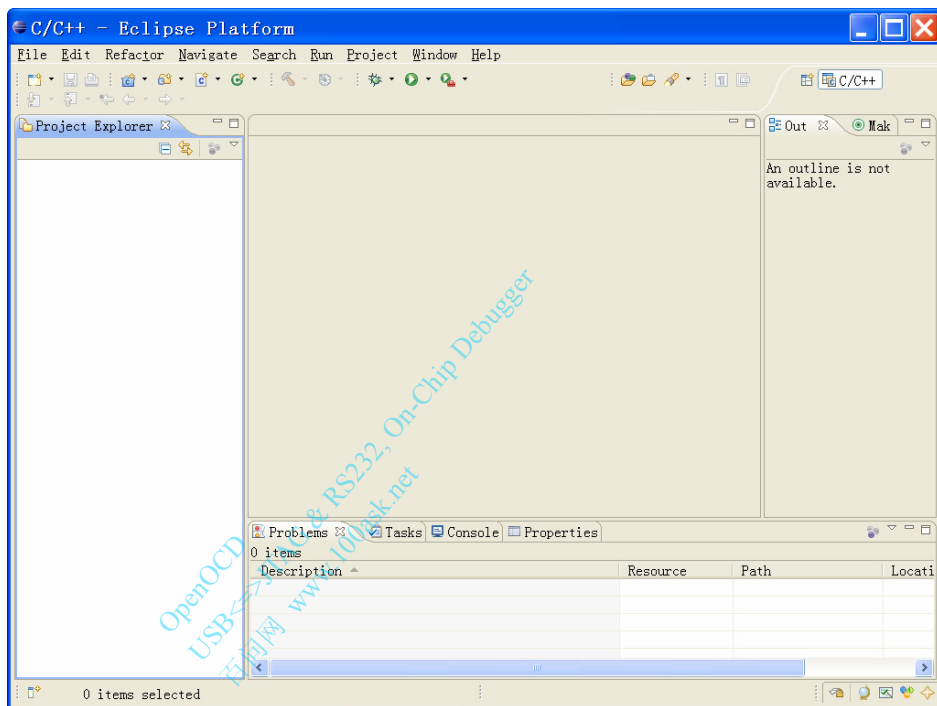


图 2.21 Eclipse 工作界面

2.3.2 新建一个 Eclipse 工程:

(1) 创建工程:

在图 2.21 的界面中，选择菜单项 “File -> New -> C Project”，如图 2.22 所示。

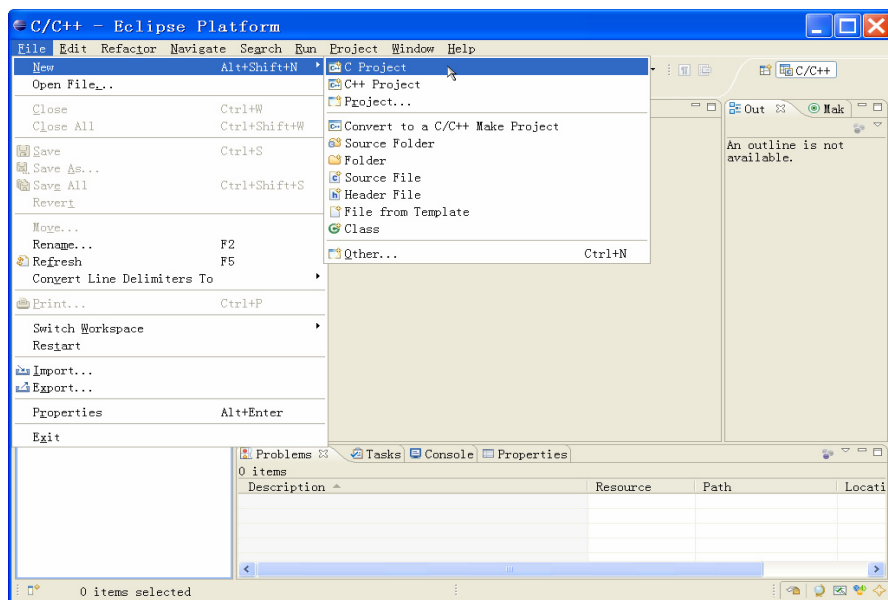


图 2.22 新建一个工程(步骤 1)

在随后出现的窗口中(如图 2.23 所示), 指定工程名字、选择 “Makefile project”、“Other Toolchain”, 然后点击 “Finish” 按钮。

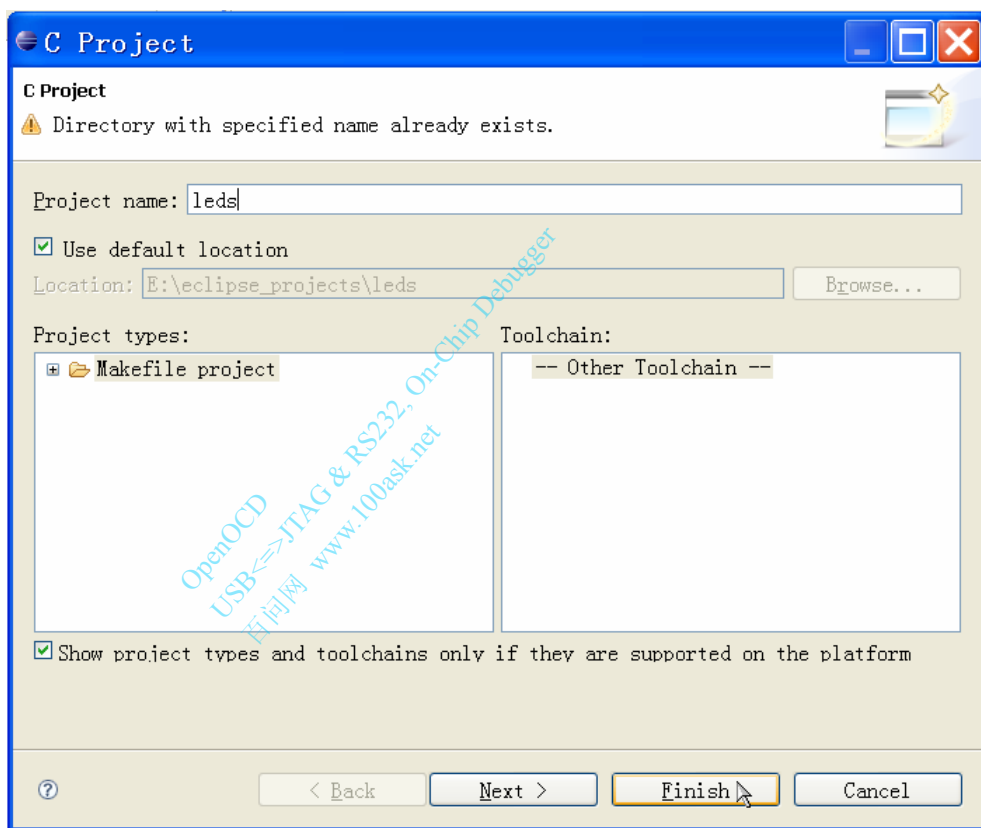


图 2.23 新建一个工程(步骤 2)

(2) 添加文件:

在创建 leds 工程时, 如果 E:\eclipse_projects\leds 目录下已经有源文件, 则这些文件会被自动加入工程中。如果 E:\eclipse_projects\leds 目录是空的话, 或者想添加新文件时, 可以使用以下方法导入文件。

下面先在 E:\eclipse_projects 下建立一个测试文件 test_file_to_add.txt, 然后以它为例介绍怎么添加文件到工程里。

选择菜单项 “File -> Import...”, 然后选择 “File System” 作为文件来源, 如图 2.24 所示。

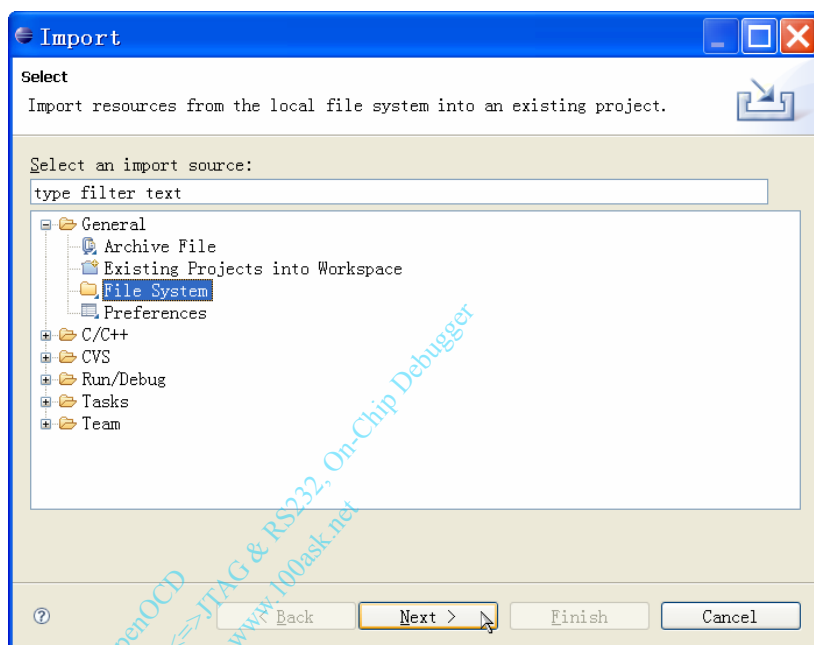


图 2.24 导入文件(步骤 1)

点击“Next”，打开“Import”对话框，在里面指定源目录“From directory”、选择要导入的文件、指定目的目录“Into folder”。如图 2.25，点击“Finish”按钮即可添加文件。

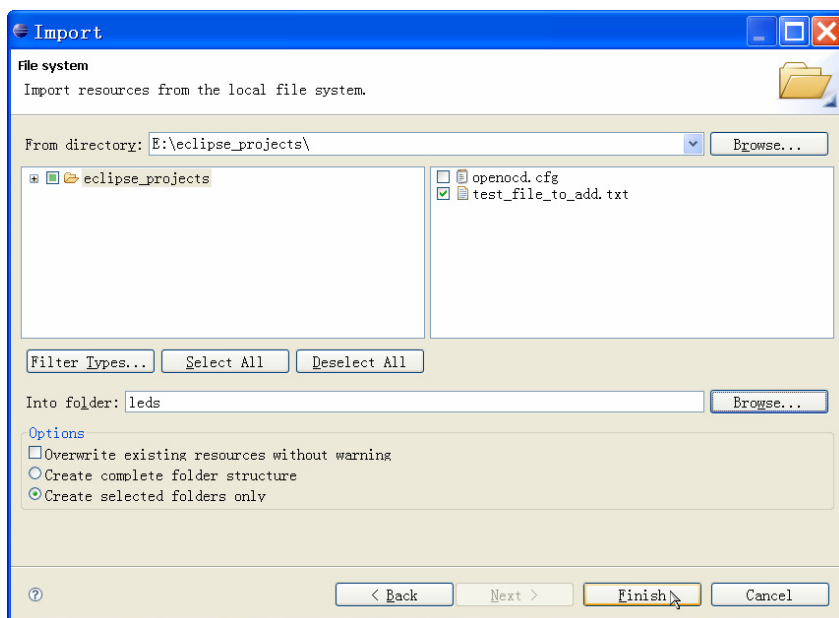


图 2.25 导入文件(步骤 2)

至此，新的 Eclipse 工程建立完毕，如图 2.26 所示。双击左边的文件名，即可打开进行编辑。

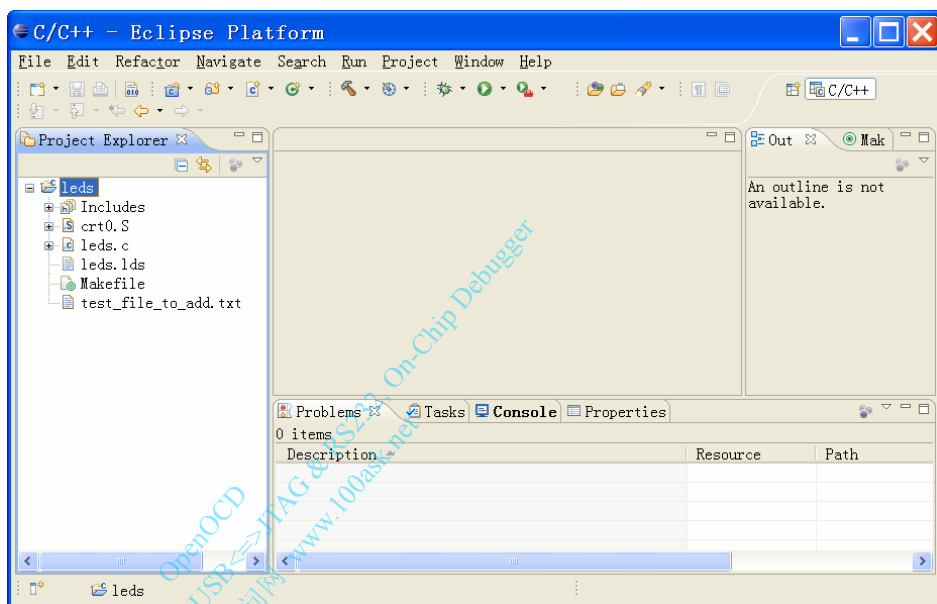


图 2.26 Eclipse 工程建立完成

2.3.3 配置 Eclipse 工程:

建立一个 Eclipse 工程后，需要进行一些配置。在“Project”菜单里，点击去掉“Build Automatically”前面的标记，表示不自动编译工程，如图 2.27 所示。

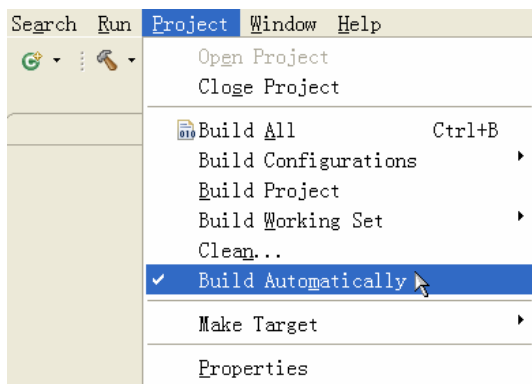


图 2.27 配置工程(步骤 1)

然后再次选择“Project”菜单，点击“Properties”项，如图 2.28 所示。

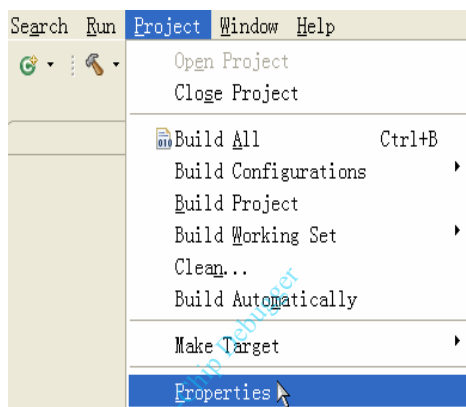


图 2.28 配置工程 (步骤 2)

新出现的窗口如图 2.29 所示。

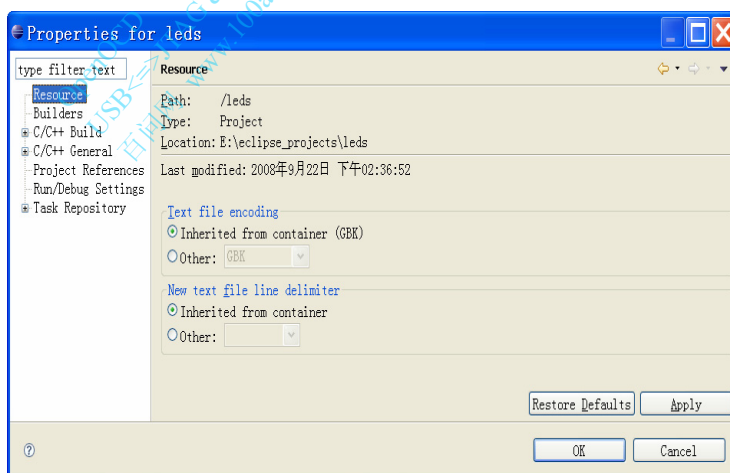


图 2.29 配置工程 (步骤 3)

选择其中的“C/C++ Build -> Discovery options”，将“Compiler invocation command”改为 arm-elf-gcc.exe(前面将 YAGARTO 交叉编译工具链安装在 C:\Program Files\yagarto 目录下)，如图 2.30 所示。

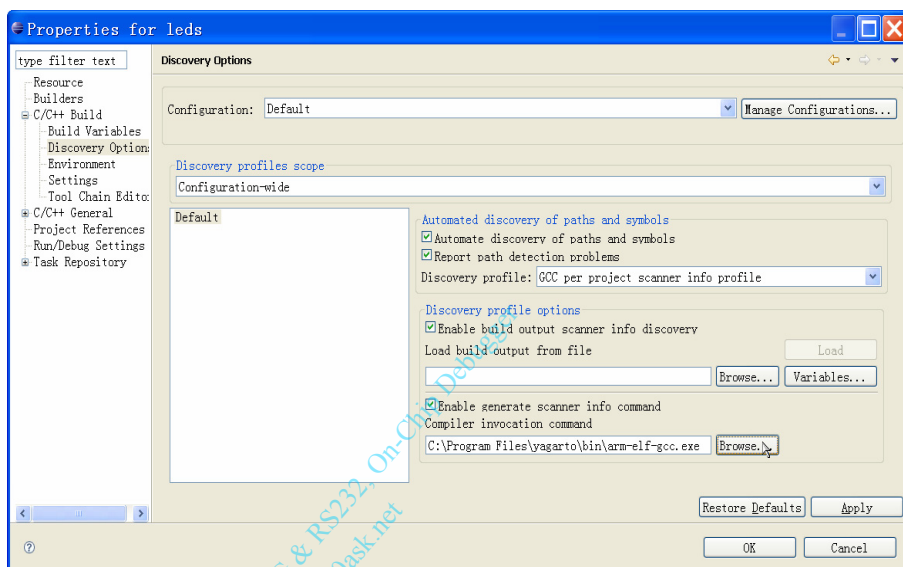


图 2.30 配置工程 (步骤 4)

继续选择 “C/C++ Build -> Settings”，在 “Binary Parsers” 中选择 “GNU Elf Parser” (点击 “GNU Elf Parser” 文字，然后勾选)，然后将 “addr2line Command”、“c++filt Command” 换成 arm-elf-addr2line.exe 和 arm-elf-c++filt.exe，如图 2.31 所示。

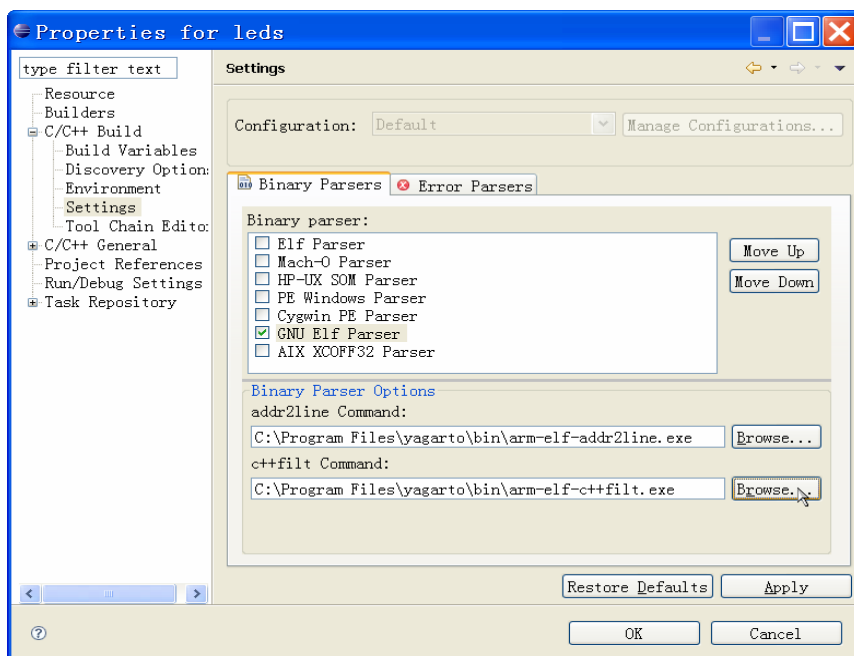


图 2.31 配置工程 (步骤 5)

2.3.4 编译、清除程序：

当配置完成后，就可以使用按钮来编译、清除程序了。点击菜单“Project -> Build Project”开始编译程序，如图 2.32 所示；编译过程见图 2.33。

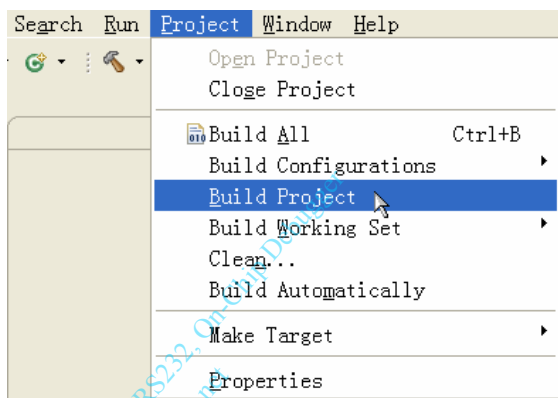


图 2.32 点击此菜单项开始编译程序

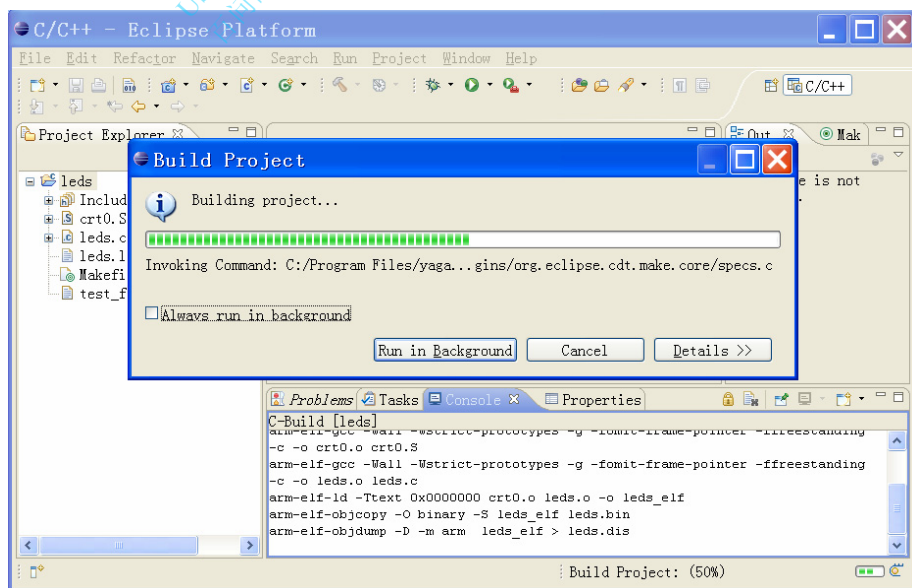


图 2.33 编译程序过程

编译的效果与在命令行执行“make all”命令完全一样。

清除程序的方法是，选择菜单“Project -> Clean...”(如图 2.34)，即可弹出一个对话框(如图 2.35)。

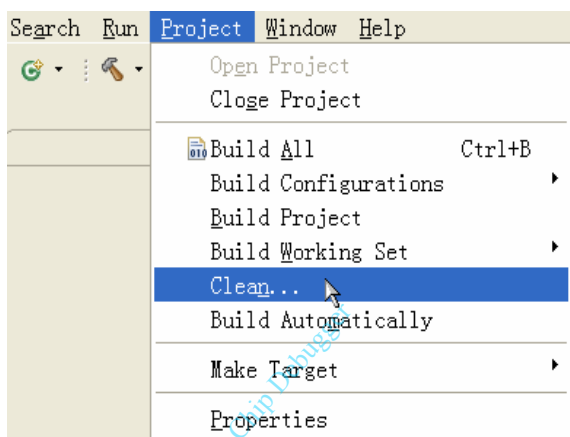


图 2.34 点击此菜单项弹出清除对话框

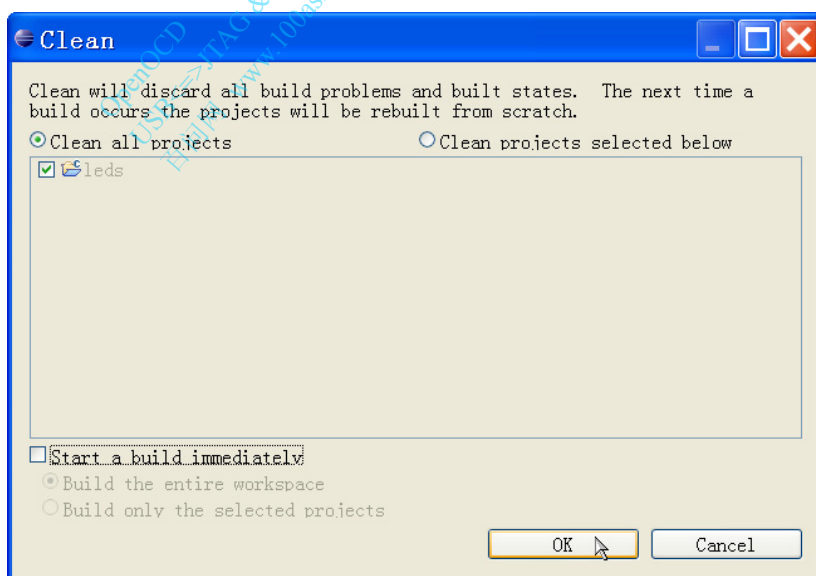


图 2.35 点击此菜单项弹出清除对话框

在图 2.35 中，不要选择“Start a build immediately”，然后点击“OK”即可清除程序。

2.3.5 使用 Eclipse 调试程序：以 leds、u-boot 为例

在调试之前，需要配置调试器、启动 OpenOCD——这也可以在 Eclipse 里启动；如果程序是在 SDRAM 中运行，还要使用 init.bin 程序来初始化存储控制器。

下面以两个例子来进行说明，leds 程序在内部 RAM 运行，无需运行 init.bin 来进行初始化；u-boot 程序在 SDRAM 中运行，需要运行 init.bin 来进行初始化。

假设已经建立好 leds 工程、u-boot 工程(u-boot 无法在 Windows 下编译，可以先使用 Linux 编译出可执行程序)。

(1) 配置调试器:

点击调试图标  右边的小箭头，如图 2.36 所示，选择 “Debug Configurations...”。

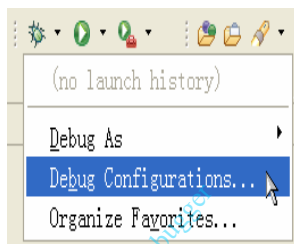


图 2.36 配置调试器(步骤 1)

弹出的窗口如图 2.37 所示，在里面可以看到 Zylín 字样的调试器。

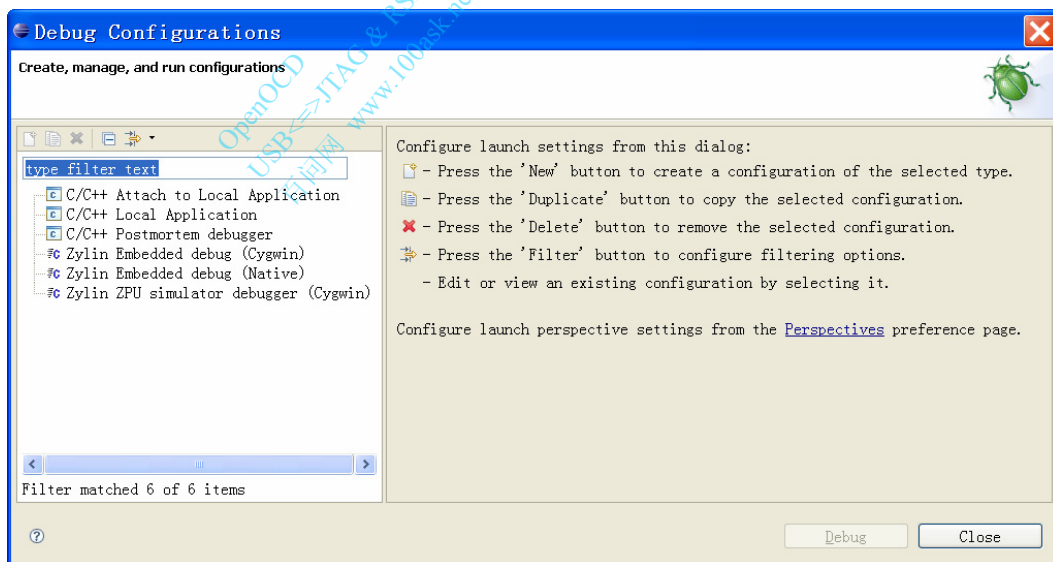


图 2.37 配置调试器(步骤 2)

选择里面的“Zylin Embedded debug (Native)”，如图 2.38 所示。

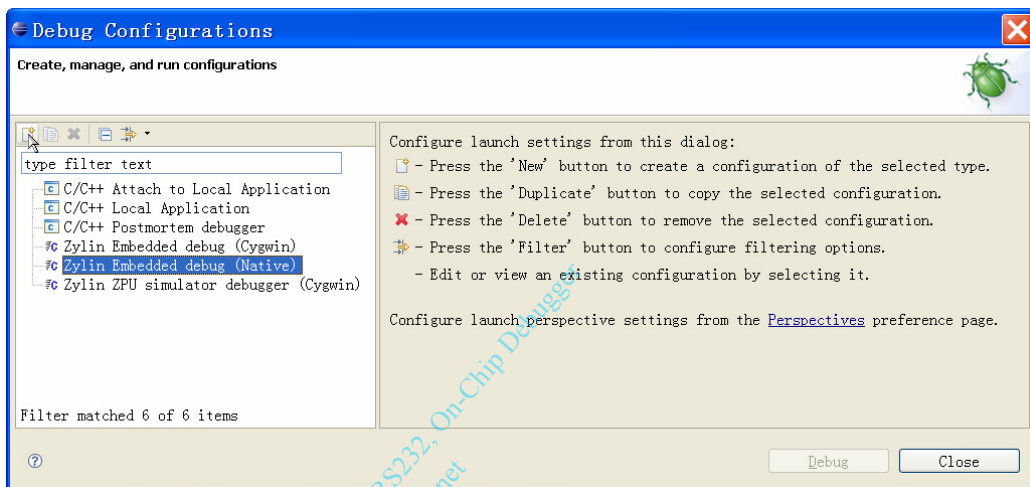


图 2.38 配置调试器(步骤 3)

在图 2.38 中，然后点击左上角的  按钮后，配置界面将发生变化，如图 2.39 所示，对于 leds 工程，图 2.39 中的名字是“leds Default”；对于 u-boot 工程则是“u-boot Default”。

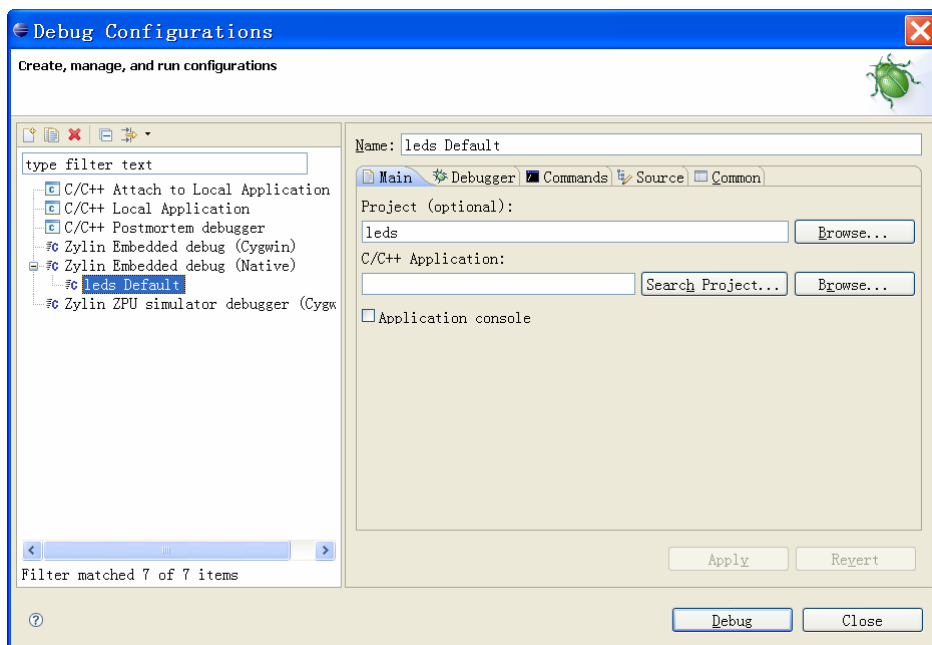


图 2.39 配置调试器(步骤 4)

现在来设置调试器。

首先选择 “C/C++ Application”，点击底下的 “Browse” 按钮，选中 ELF 的可执行文件，对于 leds 工程，这个文件是 leds_elf；对于 u-boot 工程，这个文件是 u-boot；如图 2.40 所示。

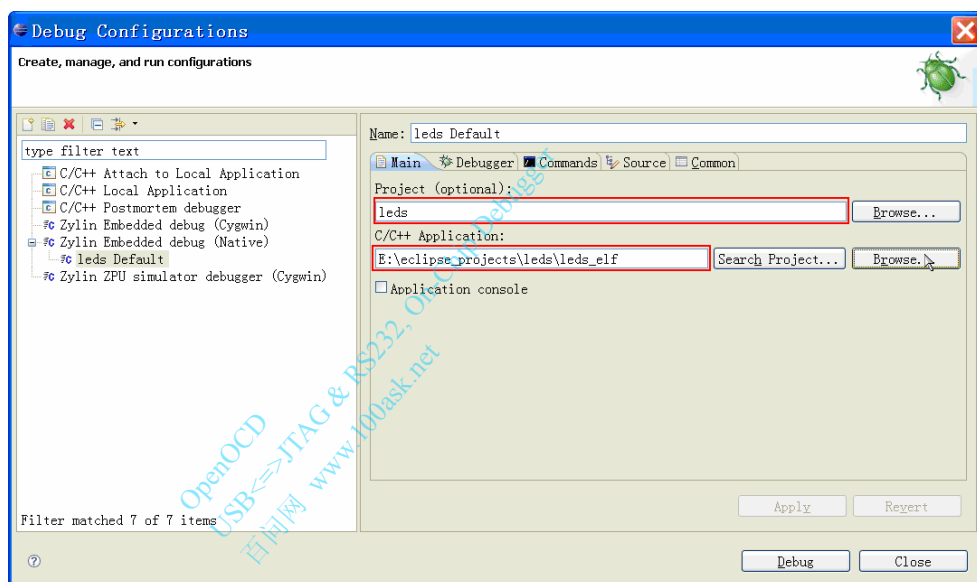


图 2.40 配置调试器(步骤 5)

然后选择调试器为 arm-elf-gdb.exe，如图 2.41 所示：设置 “GDB debugger” 为 “C:\Program Files\yagarto\bin\arm-elf-gdb.exe”，设置 “GDB command file” 为空。

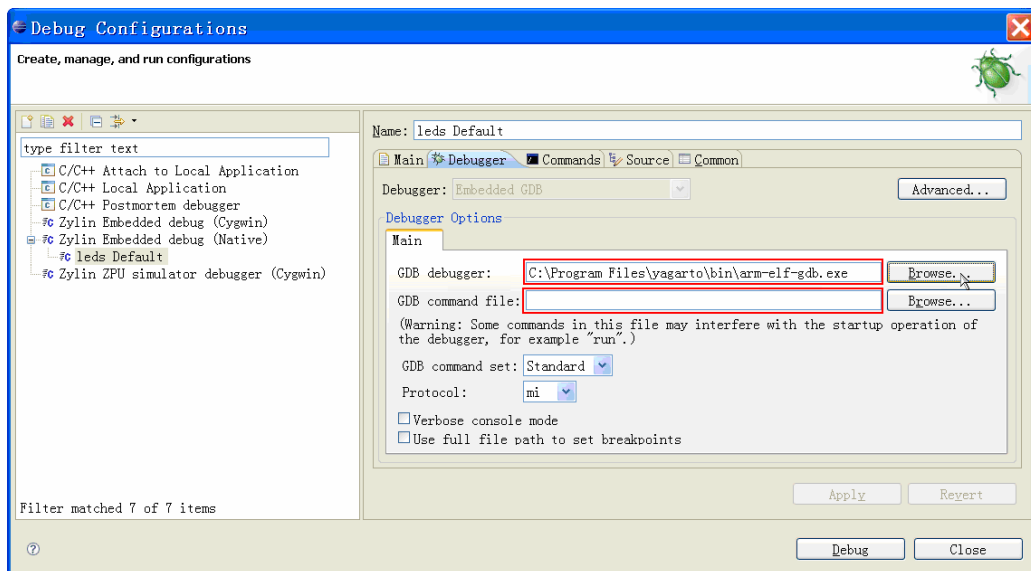


图 2.41 配置调试器(步骤 6)

最后设置启动调试器 arm-elf-gdb.exe 后，自动运行的命令，如图 2.42 所示。

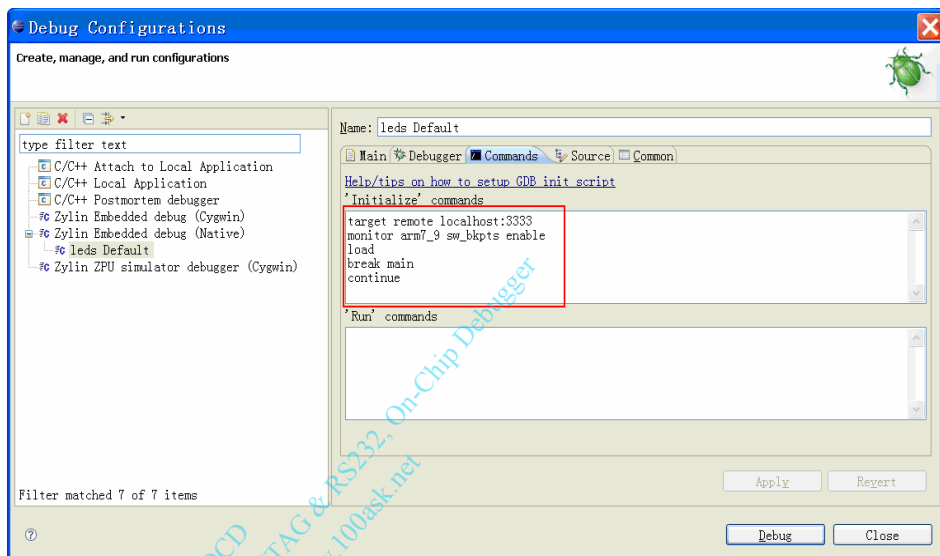


图 2.42 配置调试器(步骤 7)

当上述设置都完成后，然后点击“Apply”、“Close”退出。

对于 leds 工程，这些命令如下(“//”开始的是注释，不要把它们写进 Eclipse 中)：

```
target remote localhost:3333           // 登录本地3333接口，即连接OpenOCD
monitor arm7_9 sw_bkpts enable         // 使能软中断
load                                  // 加载leds_elf，elf格式的文件包含地址，所以不用指定
break main                            // 在main函数设置断点
continue                              // 运行程序，它会在执行到main函数时停下来
```

对于 u-boot 工程，命令与上面类似，只不过由于 u-boot 是在 SDRAM 中运行，所以要先使用 init.bin 初始化存储控制器(“//”开始的是注释，不要把它们写进 Eclipse 中)：

```
target remote localhost:3333           // 登录本地3333接口，即连接OpenOCD
monitor arm7_9 sw_bkpts enable         // 使能软中断
monitor load_image E:\eclipse_projects\init\init.bin 0x0 // 下载init.bin到内部RAM中
monitor resume 0x0                    // 运行init.bin
monitor halt                           // 停止运行init.bin
load                                   // 加载u-boot，elf格式的文件包含地址，所以不用指定
break start_armboot                   // 在start_armboot函数设置断点
continue                              // 运行程序，它会在执行到start_armboot函数时停下来
```

至此，其实已经可以调试程序了：您可以在命令行启动 OpenOCD(启动方法请参考 2.2.1 小节)，然后参考“(3) 调试程序”的内容来调试。

(2) 配置 Eclipse 的外部工具——目的是通过点击按钮启动 OpenOCD:

点击调试图标 右边的小箭头,如图 2.43 所示,选择“External Tools Configurations...”。

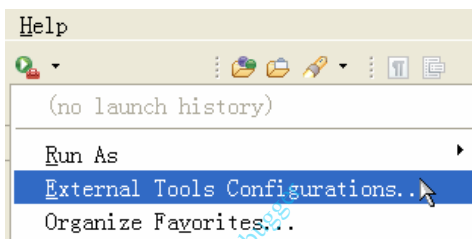


图 2.43 配置外部工具(步骤 1)

弹出的窗口如图 2.44 所示,在里面选中“Program”,然后点击左上角的按钮。

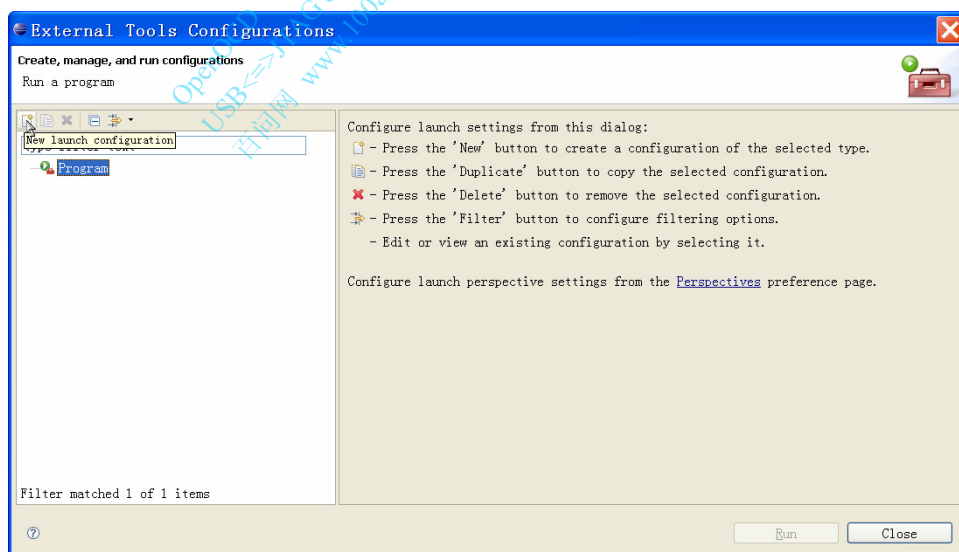


图 2.44 配置外部工具(步骤 2)

这时候配置界面发生了变化,如图 2.45、图 2.46 所示,按照图中的红框进行设置,然后点击“Apply”、“Close”退出。

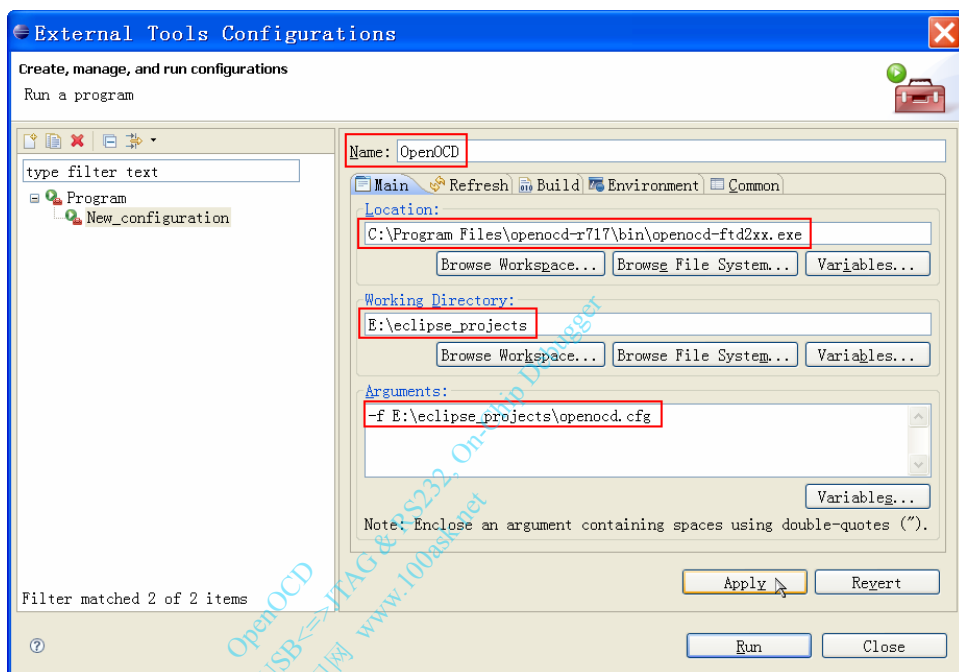


图 2.45 配置外部工具(步骤 3)

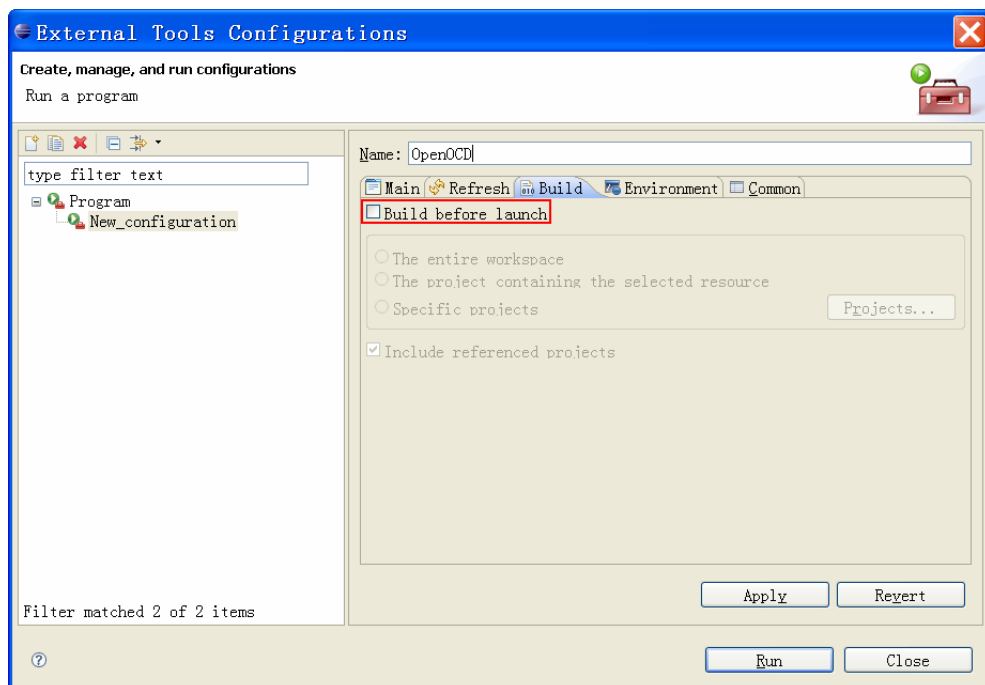


图 2.46 配置外部工具(步骤 4)

(3) 调试程序：

① 在调试程序之前，需要先启动 OpenOCD，有两种方法：

a. 可以通过命令行启动 OpenOCD(启动方法请参考 2.2.1 小节)；

b. 如果参照“(2) 配置 Eclipse 的外部工具”配置了外部工具，则可以点击图标右边的小箭头，选择“External Tools Configurations...”，进入如图 2.47 所示的界面，然后点击“Run”按钮启动 OpenOCD。

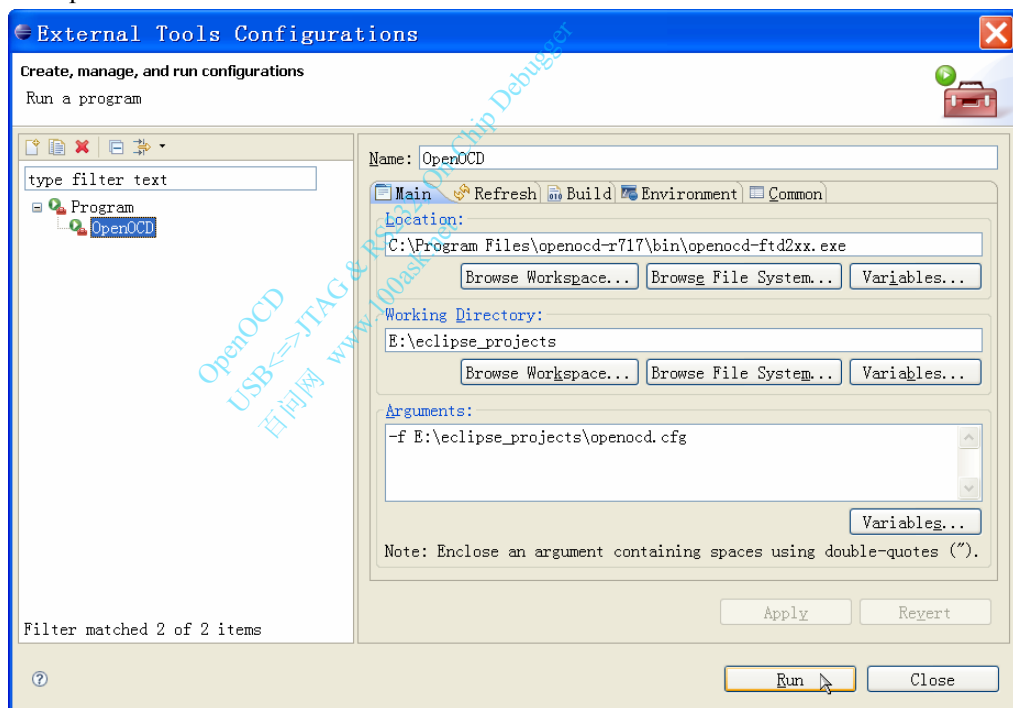


图 2.47 在对话框里通过按钮启动 OpenOCD

注意：第一次通过按钮启动 OpenOCD 之后，以后要启动它时，就可以直接点击图标右边的小箭头，从中选择“OpenOCD”，如图 2.48 所示。OpenOCD 在 Eclipse 的整个运行过程中，只需要启动 1 次。

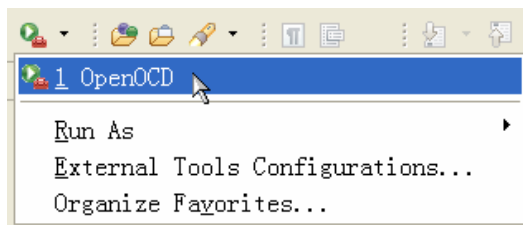


图 2.48 在下拉框中通过按钮启动 OpenOCD

② 启动调试器

点击调试图标  右边的小箭头，选择“Debug Configurations...”，进入如图 2.49 所示的界面，然后点击“Debug”按钮启动调试器。

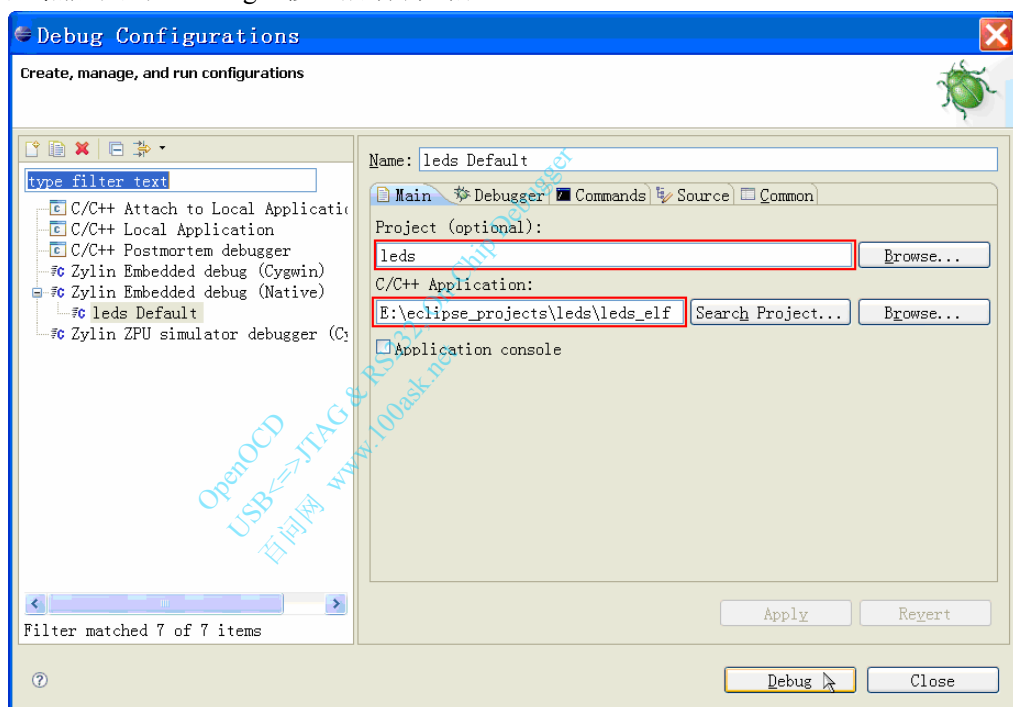


图 2.49 在对话框里通过按钮启动调试器

注意：第一次通过按钮启动调试器之后，以后要启动它时，就可以直接点击图标  右边的小箭头，从中选择“leds Default”（对于 leds 工程），如图 2.50 所示。

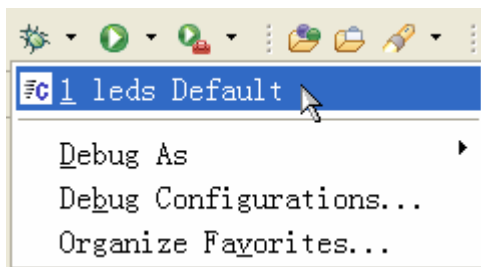


图 2.50 在下拉框中通过按钮启动调试器

③ 调试程序简介：以 leds 工程为例

调试器启动后，它会根据图 2.42 所设置的命令，将代码下载到单板上，设置断点，运行。

首先出现如图 2.51 所示的对话框询问是否进入调试界面，选择“Remember my decision”，然后选择“Yes”，表示以后调试器启动自动进入调试界面。

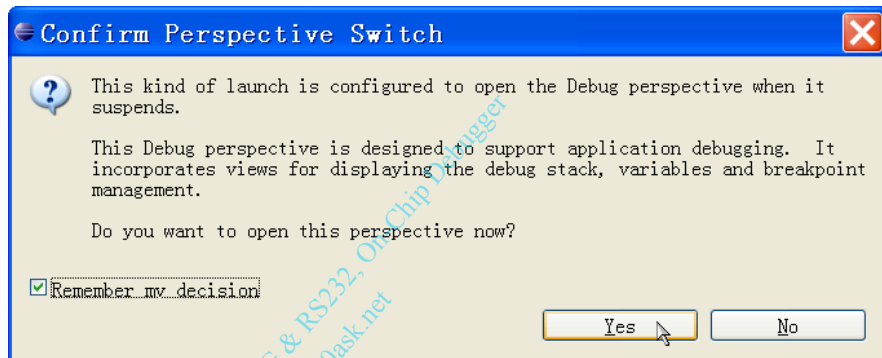


图 2.51 设置启动调试器时自动进入调试界面

紧接着出现的就是调试界面，如图 2.52 所示。

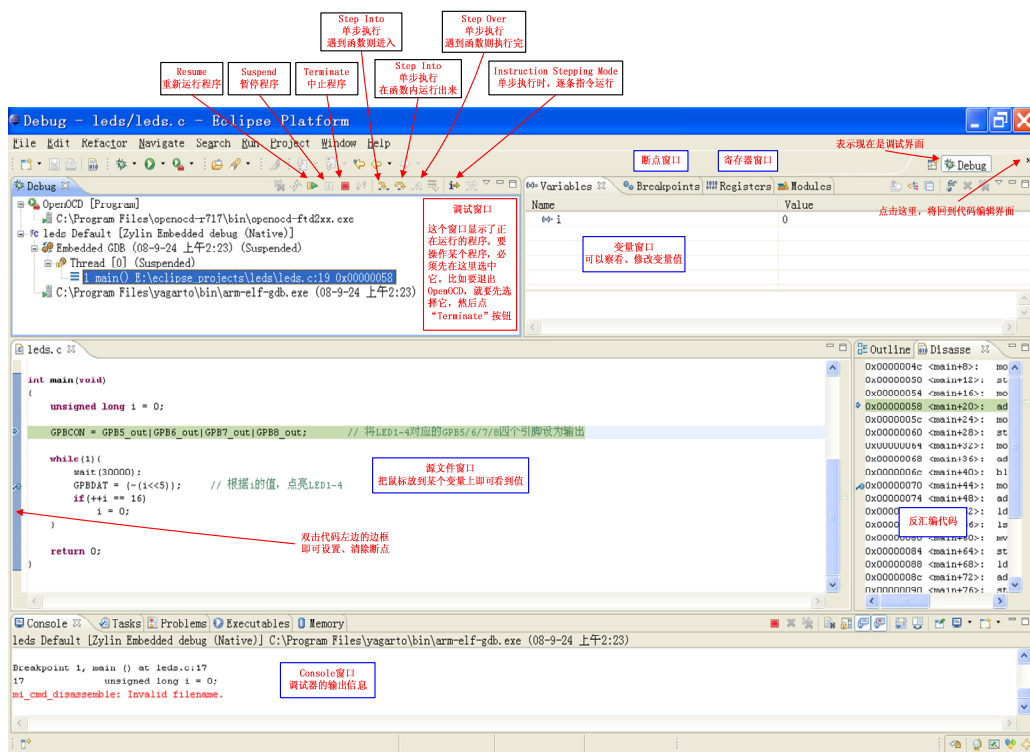


图 2.52 设置启动调试器时自动进入调试界面

在调试界面中，可以很方便地进行各类操作。比如双击代码 “GPBDAT = (~(i<<5));
// 根据 i 的值，点亮 LED1-4” 左边的边框就可以设置断点，然后不断点击 “Resume”

按钮 ，就可以看到 4 个 LED 逐渐地变化。

④ 调试程序简介：以 u-boot 工程为例

注意：调试网上下载的 u-boot 时，需要定义 **CONFIG_SKIP_LOWLEVEL_INIT**，它表示“跳过底层的初始始化”，就是不要初始化存储控制器，不要再次复制 u-boot 本身到 SDRAM 中。对于光盘中的 u-boot，已经增加的自动识别代码，无需定义这个宏。

虽然不能在 Windows 下编译 u-boot，但是只要有 elf 格式的可执行文件和代码，还是可以在 Eclipse 下编译的。

E:\eclipse_projects\u-boot 目录下的 u-boot.bin、u-boot 是 u-boot-1.1.6_OpenJTAG.tar.bz2 在 Linux 下编译好的可执行文件，u-boot.bin 是二进制格式，u-boot 是 elf 格式。

首先，在 E:\eclipse_projects\u-boot 目录下将 u-boot-1.1.6_OpenJTAG.tar.bz2 解压缩。

然后，仿照前面创建 leds 工程的方法创建 u-boot 工程。

最后，为 u-boot 重新配置调试器，配置内容如图 2.53~2.66 所示。

注意：

a. 图 2.55 中的 “Commands” 显示不全，它们的内容如下：

```
target remote localhost:3333
monitor arm7_9 sw_bkpts enable
monitor load_image E:\eclipse_projects\init\init.bin 0x0
monitor resume 0x0
monitor halt
load
break start_armboot
continue
```

b. 图 2.56 指定源码位置：由于 u-boot 代码不在 E:\eclipse_projects\u-boot 目录下，而是在次级目录 u-boot-1.1.6_OpenJTAG 下，需要自己指定。

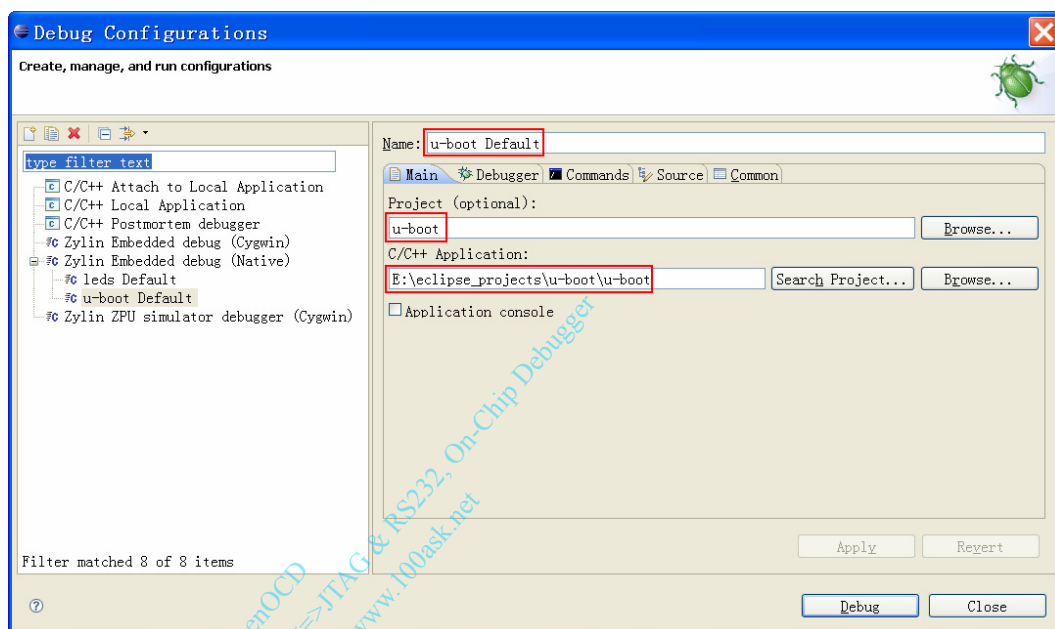


图 2.53 u-boot 工程调试器配置信息(1)

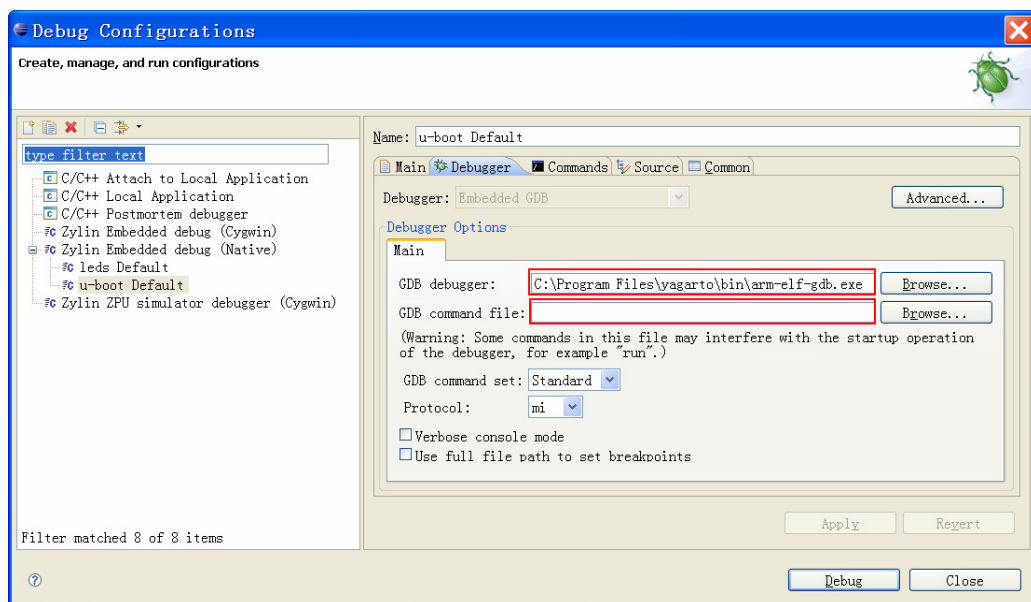


图 2.54 u-boot 工程调试器配置信息(2)

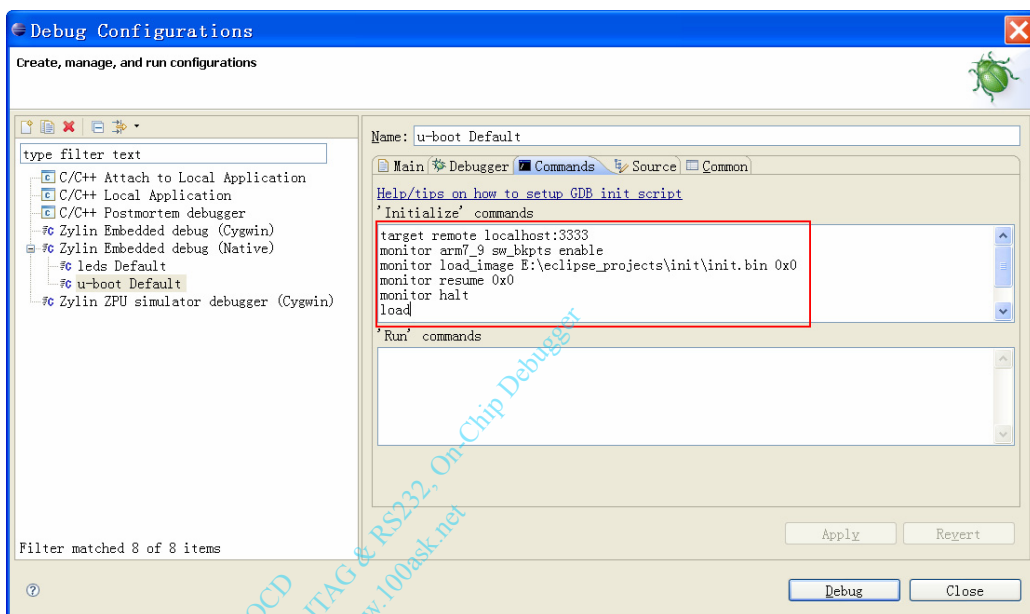


图 2.55 u-boot 工程调试器配置信息 (3)

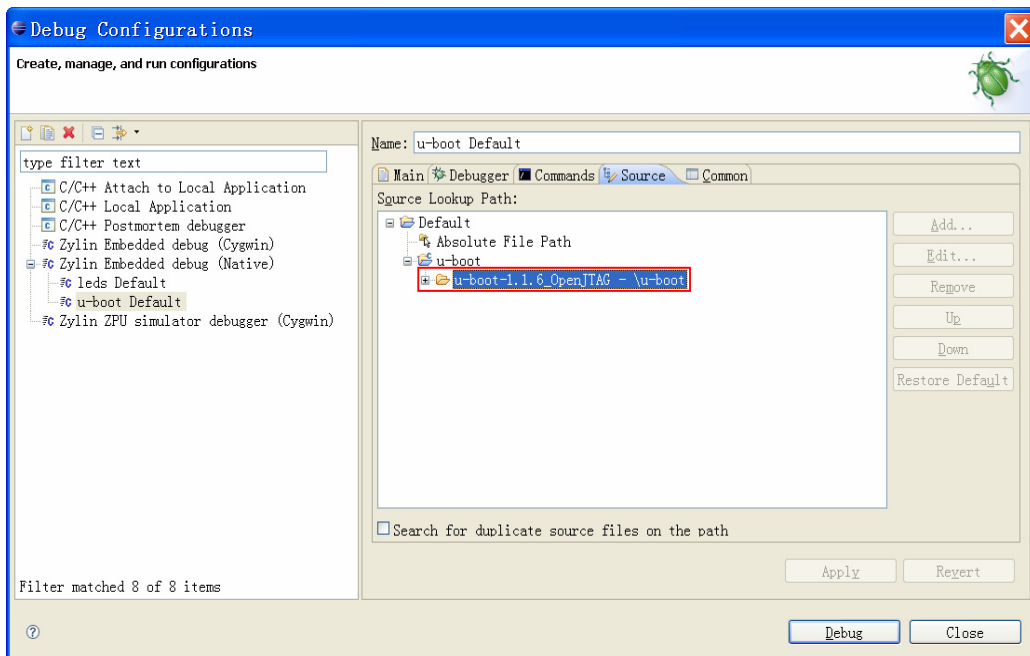


图 2.56 u-boot 工程调试器配置信息 (4)

配置完成后，先启动 OpenOCD，然后启动“u-boot Default”调试器，稍等一会就可以看到如图 1.69 所示的界面，u-boot 停在 start_armboot 函数的第一条指令上。

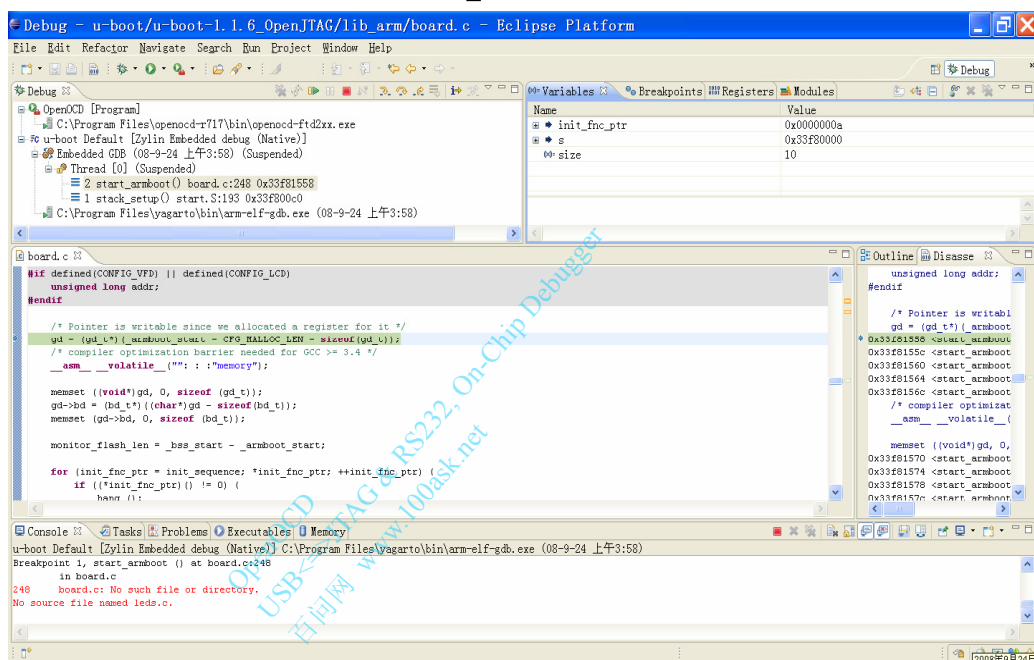


图 2.57 u-boot 工程调试界面

除调试器的设置不同外，u-boot 工程的调试方法与 leds 工程的基本一样，不再赘述。

3. Ubuntu 下 OpenJTAG 套件的使用

3.1 硬件、软件安装

如果只是想把 OpenJTAG 当作 USB 转串口卡来使用，可以只看 3.1.1 小节。

3.1.1 修改 udev 规则，使得识别 OpenJTAG 后自动加载驱动

把光盘文件 Ubuntu/install/50-ftdi.rules 复制到/etc/udev/rules.d/目录下：

```
$ sudo cp /media/cdrom0/Ubuntu/install/50-ftdi.rules /etc/udev/rules.d/
```

如果想立刻让这个 udev 规则生效，可以运行以下命令(下次启动后，无需运行此命令)：

```
$ sudo udevcontrol reload_rules
```

将 OpenJTAG 板子接到 PC 的 USB 口后，Ubuntu 即会自动加载驱动，在/dev 目录下自动创建 ttyUSB0、ttyUSB1 两个设备：

```
$ ls /dev/ttyUSB* -l
crw-rw---- 1 root dialout 188, 0 2008-10-03 13:25 /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 1 2008-10-03 13:25 /dev/ttyUSB1
```

此时，就可以将/dev/ttyUSB1(注意，是 ttyUSB1，不是 ttyUSB0)作为一般串口使用了。

在 Ubuntu 下，可以使用 kermit 或 minicom 等工具操作中口。使用以下命令安装：

```
$ sudo apt-get install ckermit
$ sudo apt-get install minicom
```

使用 kermit 之前，先在/home/book(假设用户名为 book)目录下创建一个名为.kermrc 的配置文件，内容如下：

```
set line /dev/ttyUSB1
set speed 115200
set carrier-watch off
set handshake none
set flow-control none
robust
set file type bin
set file name lit
set rec pack 1000
set send pack 1000
set window 5
```

然后，运行“\$ sudo kermit -c”命令即可启动串口；要关闭串口，先同时按住“Ctrl”和“\”键，然后松开再按“C”键，最后输入“exit”并回车。

对于 minicom, 使用之前先运行“minicom -s”命令进入设置界面, 选择“Serial port setup”, 按照图 3.1 所示进行设置, 然后选择“Save setup as dfl”。

```

+-----+
| A -   Serial Device       : /dev/ttyUSB1
| B - Lockfile Location    : /var/lock
| C -   Callin Program     :
| D -   Callout Program    :
| E -   Bps/Par/Bits       : 115200 8N1
| F - Hardware Flow Control : No
| G - Software Flow Control: No
|
| Change which setting? █
+-----+
|
| Screen and keyboard
| Save setup as dfl
| Save setup as..
| Exit
| Exit from Minicom
|
+-----+

```

图 3.1 minicom 的设置

以后, 就可以直接运行“minicom”命令启动串口了; 要想退出 minicom, 先同时按住“Ctrl + A”, 松开后再按住 X。

3.1.2 安装 OpenOCD、GDB、交叉编译工具链、Eclipse

(1) 安装 OpenOCD、GDB:

将光盘 Ubuntu/install/目录下的 openocd、arm-linux-gdb 复制到/usr/bin 目录下, 并设置“可执行”属性:

```

$ sudo cp /media/cdrom0/Ubuntu/install/openocd /usr/bin/
$ sudo cp /media/cdrom0/Ubuntu/install/arm-linux-gdb /usr/bin/
$ sudo chmod +xs /usr/bin/openocd /usr/bin/arm-linux-gdb

```

注意: 一定要给/usr/bin/arm-linux-gdb 加上可执行权限“x”, 超级用户权限“s”。

如果 Ubuntu 还没有安装交叉编译工具链(可以执行“arm-linux-gcc -v”命令确认, 若无输出则表示还没有安装), 可以将 Ubuntu/install/arm-linux-gcc-3.4.5-glibc-2.3.6.tar.bz2 文件解压到某个目录, 然后设置环境变量 PATH。比如, 以下命令将它解压到/work/tools 目录下:

```

$ cp /media/cdrom0/Ubuntu/install/arm-linux-gcc-3.4.5-glibc-2.3.6.tar.bz2 /work/tools
$ cd /work/tools
$ tar xjf arm-linux-gcc-3.4.5-glibc-2.3.6.tar.bz2

```

假设以后将在/work/eclipse_projects 目录下存放代码, 则将 OpenOCD 的配置文件放到这个目录下:

```

$ cp /media/cdrom0/Ubuntu/install/openocd.cfg /work/eclipse_projects

```

(2) 安装交叉编译工具链:

然后编辑/etc/environment 文件, 修改 PATH 变量(红色部分是新加的):

```
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/work/tools/gcc-3.4.5-glibc-2.3.6/bin"
```

下次启动系统后, 即可直接使用交叉编译工具链; 如果想立刻使用, 可以使用以下命令设置 PATH 变量:

```
$ export PATH=$PATH:/work/tools/gcc-3.4.5-glibc-2.3.6/bin
```

(3) 安装 Eclipse:

① 安装 SUN Java:

Ubuntu 中已经安装了 GNU 版本的 Java, 只是它的性能比不上 SUN Java, Eclipse 的一些插件必须在 SUN Java 上才能使用。首先使用以下命令安装 SUN Java:

```
sudo apt-get install sun-java6-jre
sudo apt-get install sun-java6-jdk
```

然后执行以下命令, 在如图 3.2 的界面中将刚安装的 sun-java6 置为默认 Java:

```
sudo update-alternatives --config java
```

```
book@book-desktop:~$ sudo update-alternatives --config java
There are 2 alternatives which provide `java`.

   Selection    Alternative
-----
          1    /usr/bin/gij-4.2
   **          2    /usr/lib/jvm/java-6-sun/jre/bin/java

Press enter to keep the default[*], or type selection number: 2
Using `usr/lib/jvm/java-6-sun/jre/bin/java` to provide `java`.
```

图 3.2 选择 sun-java6 作为默认 Java

② 安装、设置 Eclipse:

首先将 Ubuntu/install/eclipse-SDK-3.4.1-linux-gtk_cdt-5.0.1.tar.bz2 解压到/opt 目录, 并且改变属性:

```
$ cd /media/cdrom0/Ubuntu/install
$ sudo tar xjf eclipse-SDK-3.4.1-linux-gtk_cdt-5.0.1.tar.bz2 -C /opt
$ cd /opt
$ sudo chmod 775 eclipse -R
```

然后将脚本文件 Ubuntu/install/eclipse 复制到/usr/bin 目录下, 并增加“可执行”的属性:

```
$ sudo cp /media/cdrom0/Ubuntu/install/eclipse /usr/bin
$ sudo chmod +x /usr/bin/eclipse
```

最后, 在菜单中添加 Eclipse 的快捷方式, 将光盘中的 Ubuntu/install/eclipse.desktop 文件复制到相应目录即可:

```
$ sudo cp /media/cdrom0/Ubuntu/install/eclipse.desktop /usr/share/applications
```


现在来确定一下安装是否成功，在终端执行以下命令确定 `openocd`、`gdb`、交叉编译工具链、`java` 是否已经安装：

```
$ openocd -v
$ arm-linux-gdb -v
$ arm-linux-gcc -v
$ java -version
```

可以从图 3.4 所示的启动菜单中启动 Eclipse，效果如图 3.5 所示，点击“Cancel”——先不要设置任何内容，这在后面会有描述。

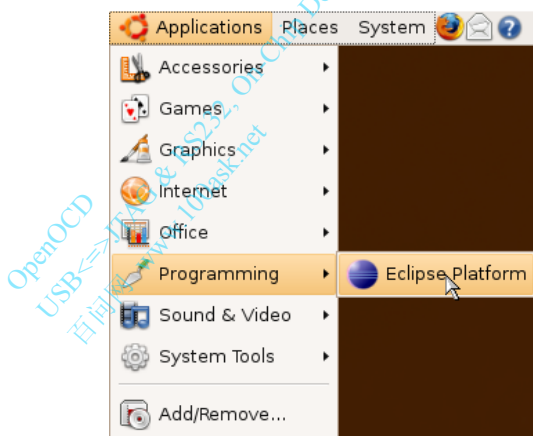


图 3.4 Eclipse 在 Ubuntu 中的启动菜单

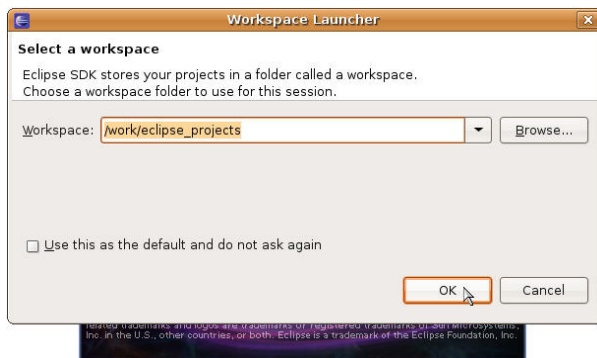


图 3.5 Eclipse 在 Ubuntu 中的启动界面

3.2 使用 OpenOCD、OpenJTAG 烧写程序、调试程序

在 Linux 下，对于 S3C2410、S3C2440 开发板，大多使用 Jflash-s3c2410、Jflash-s3c2410 工具通过并口 JTAG 接口板烧写程序，并口 JTAG 的速率非常低，烧写 128K 的文件至少耗时 10 多分钟。

后来又出现了 H-JTAG 工具，它也是使用并口 JTAG 接口板、速度很快，但是目前只支持烧写 Nor Flash，不支持 Nand Flash；并且只能在 Windows 下运行。

使用 OpenOCD，既可以烧写 Nor Flash，也可以烧写 Nand Flash，还可以进行简单的调试。

注意：OpenOCD 目前只支持 CFI 接口的 Nor Flash，但是可以通过 OpenOCD 将 u-boot 下载到单板内存后，启动 u-boot 来烧写 Flash，它支持 CFI 接口、JEDEC 接口的 Nor Flash，当然也支持 Nand Flash。

本节都假设工作目录在 `/work/eclipse_projects` 目录下。假设已经把光盘上的 `Ubuntu/examples` 目录下的所有文件夹、文件，及 `Ubuntu\install\openocd.cfg` 文件都复制到了 `/work/eclipse_projects` 目录下。

3.2.1 启动 OpenOCD，OpenOCD 常用命令

把开发板的 JTAG 口、串口都接到 OpenJTAG 板上去；然后给开发板供电(**注意：使用 JTAG 线连接 OpenJTAG 与开发板时，开发板一定要上电**)；最后使用 USB 线连接 OpenJTAG 和计算机。

在运行 `openocd` 之前，Ubuntu 会自动生成 `/dev/ttyUSB0`、`/dev/ttyUSB1` 两个设备节点。

注意：如果是使用 VMware 运行 Ubuntu，那么 VMware 的版本至少为 6.0，否则对 USB 设备的支持不好；要想在 VMware 中的 Linux 系统识别到 OpenJTAG，一是要设置 VMware 支持 USB 设备；二是接上 OpenJTAG 时，VMware 处于前台。

`openocd` 的运行需要一些配置信息，这些配置信息写在 `/work/eclipse_projects/openocd.cfg` 文件中，在终端里进入 `/work/eclipse_projects` 目录，执行“`openocd`”命令即可启动 OpenOCD，这时候可以发现 `/dev/ttyUSB0` 设备消失了，如图 3.6 所示。

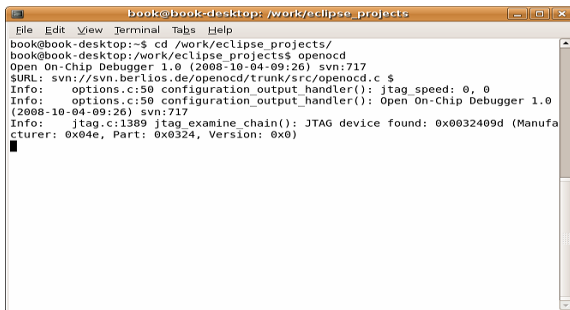


图 3.6 在 Ubuntu 中运行 OpenOCD

注意：`openocd.cfg` 中有一行“`#arm7_9 fast_memory_access enable`”，如果不是在 VMware 中使用 OpenOCD，可以把它前面的“#”号去掉，这将把 OpenOCD 的下载速度增加 6、7 倍。

如果 `openocd-ftd2xx` 运行出错，请确认开发板是否已经供电。如果供电正常，则先拔下 OpenJTAG 板的 USB 线、与开发板相连的 JTAG 线，然后重新接上 JTAG 线、USB 线，最后重新启动 `openocd-ftd2xx`。

OpenOCD 启动后，在另一个命令行中执行“`telnet localhost 4444`”即可登录 OpenOCD，以后就可以在 telnet 界面操作 OpenOCD 了：包括读/写开发板内存，烧写 Flash，使用命令行的方式调试程序(使用图形界面调试程序需要使用 Eclipse)。

telnet 界面如图 3.7 所示。

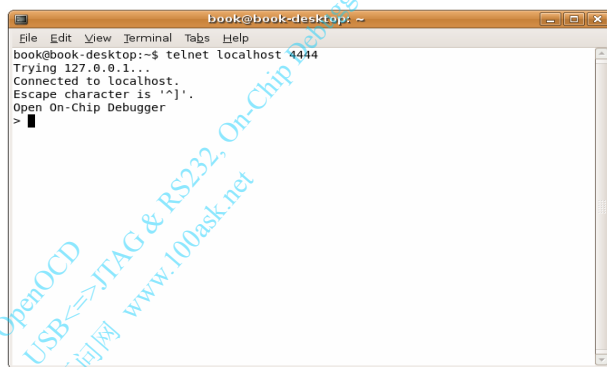


图 3.7 Ubuntu 下 telnet 登录 OpenOCD 的界面

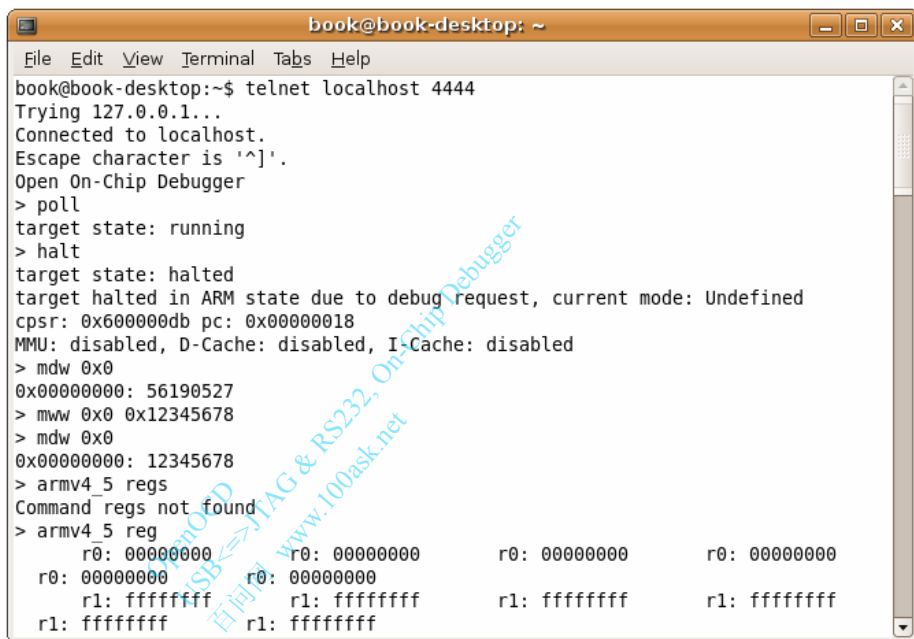
OpenOCD 的使用文档是 [Ubuntu/install/openocd.pdf](#)，下面列出常用命令。

表 3.1

OpenOCD 常用命令

目标板状态处理命令(Target state handling)	
poll	查询目标板当前状态
halt	中断目标板的运行
resume [address]	恢复目标板的运行，如果指定了 address，则从 address 处开始运行
step [address]	单步执行，如果指定了 address，则从 address 处开始执行一条指令
reset	复位目标板
断点命令	
bp <addr> <length> [hw]	在地址 addr 处设置断点，指令长度为 length，hw 表示硬件断点
rbp <addr>	删除地址 addr 处的断点
内存访问指令(Memory access commands)	
mdw <addr> [count]	显示从地址 addr 开始的 count(缺省是 1)个字(4 字节)
mdh <addr> [count]	显示从地址 addr 开始的 count(缺省是 1)个半字(2 字节)
mdb <addr> [count]	显示从地址 addr 开始的 count(缺省是 1)个字节
mww <addr> <value>	向地址 addr 写入一个字，值为 value
mwh <addr> <value>	向地址 addr 写入一个半字，值为 value
mwb <addr> <value>	向地址 addr 写入一个字节，值为 value
load_image <file> <address> ['bin' 'ihex' 'elf'] 将文件<file>载入地址为 address 的内存，格式有'bin'、'ihex'、'elf'	
dump_image <file> <address> <size> 将内存从地址 address 开始的 size 字节数据读出，保存到文件<file>中	
verify_image <file> <address> ['bin' 'ihex' 'elf'] 将文件<file>与内存 address 开始的数据进行比较，格式有'bin'、'ihex'、'elf'	
CPU 架构相关命令(Architecture Specific Commands)	
armv4 reg	打印寄存器的值
arm7_9 sw_bkpts <enable disable> 使能或禁止软件断点，在 ARMv4 系统中有两个硬件断点，软件断点将占用一个硬件断点	
arm920t cp15 <num> [value]	显示/修改 cp15 协处理器的寄存器
arm920t md<bhw>_phys <addr> [count]	显示物理地址 addr 处的内存
arm920t mw<bhw>_phys <addr> <value>	修改物理地址 addr 处的内存
arm920t virt2phys <va>	获得虚拟地址 va 对应的物理地址
其他命令	
script <file>	执行 file 文件中的命令

下面试验一下，S3C2410 或 S3C2440 从 Nand 启动时，内部内存地址是 0；从 Nor Flash 启动时，内部内存地址是 0x40000000。以下假设从 Nand 启动，您可以按照图 3.8 操作一下。



```

book@book-desktop: ~
File Edit View Terminal Tabs Help
book@book-desktop:~$ telnet localhost 4444
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^'.
Open On-Chip Debugger
> poll
target state: running
> halt
target state: halted
target halted in ARM state due to debug request, current mode: Undefined
cpsr: 0x600000db pc: 0x00000018
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> mdw 0x0
0x00000000: 56190527
> mww 0x0 0x12345678
> mdw 0x0
0x00000000: 12345678
> armv4 5 regs
Command regs not found
> armv4 5 reg
r0: 00000000 r0: 00000000 r0: 00000000 r0: 00000000
r0: 00000000 r0: 00000000
r1: ffffffff r1: ffffffff r1: ffffffff r1: ffffffff
r1: ffffffff r1: ffffffff

```

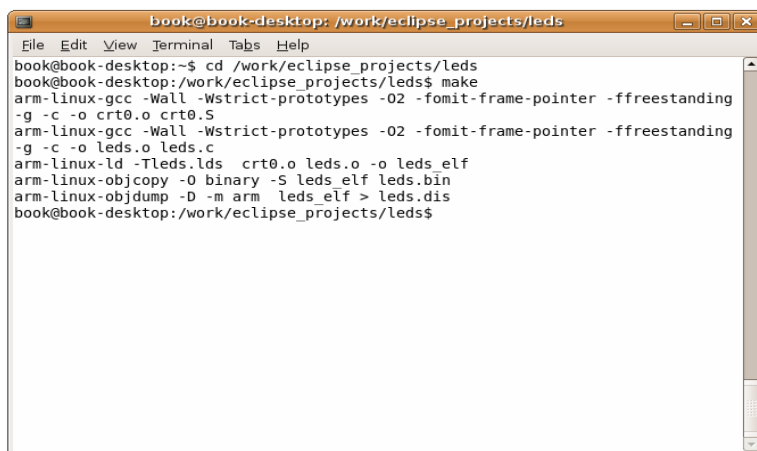
图 3.8 OpenOCD 命令操作示例

3.2.2 使用 OpenOCD 烧写小程序到内部 RAM，并运行、调试

S3C2410、S3C2440 有 4K 的内部 RAM，当程序小于 4K 时，可以通过 OpenOCD 直接把程序烧到内部 RAM 中、运行、调试。

(1) 编译程序：

以光盘上的/work/eclipse_projects/leds 程序为例，先编译程序：在命令行进入文件源码所在目录，执行“make”命令，如图 3.9 所示。



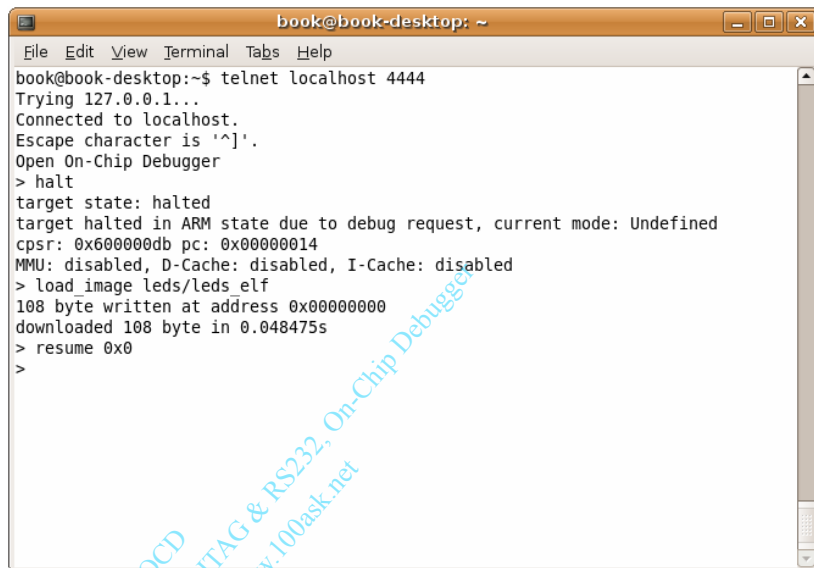
```

book@book-desktop: /work/eclipse_projects/leds
File Edit View Terminal Tabs Help
book@book-desktop:~$ cd /work/eclipse_projects/leds
book@book-desktop:/work/eclipse_projects/leds$ make
arm-linux-gcc -Wall -Wstrict-prototypes -O2 -fomit-frame-pointer -ffreestanding
-g -c -o crt0.o crt0.S
arm-linux-gcc -Wall -Wstrict-prototypes -O2 -fomit-frame-pointer -ffreestanding
-g -c -o leds.o leds.c
arm-linux-ld -Tleds.lds crt0.o leds.o -o leds.elf
arm-linux-objcopy -O binary -S leds.elf leds.bin
arm-linux-objdump -D -m arm leds.elf > leds.dis
book@book-desktop:/work/eclipse_projects/leds$

```

图 3.9 在命令行编译程序

(2) 下载、运行程序：



```
book@book-desktop: ~  
File Edit View Terminal Tabs Help  
book@book-desktop:~$ telnet localhost 4444  
Trying 127.0.0.1...  
Connected to localhost.  
Escape character is '^]'.  
Open On-Chip Debugger  
> halt  
target state: halted  
target halted in ARM state due to debug request, current mode: Undefined  
cpsr: 0x600000db pc: 0x00000014  
MMU: disabled, D-Cache: disabled, I-Cache: disabled  
> load_image leds/leds.elf  
108 byte written at address 0x00000000  
downloaded 108 byte in 0.048475s  
> resume 0x0  
>
```

图 3.10 使用 OpenOCD 加载、运行程序

注意：下载程序之前先使用“halt”命令暂停单板，才能下载代码；如果使用“poll”命令发现单板的 MMU 或 D-cache 已经使能，则需要使用“arm920t cp15 2 0”命令禁止 MMU 和 D-cache。

在 telnet 界面，先执行“load_image leds/leds.elf”加程序，它会根据 leds_elf 里面的信息把可执行代码加载到地址为 0 的内存处，“load_image leds/leds.bin 0x0”可以达到同样的效果；然后执行“resume 0x0”命令执行程序。如图 3.10 所示。

(3) 简单的调试命令：

运行 leds 程序后，可以使用 “halt” 命令暂停程序，使用 “resume” 命令恢复运行程序，使用 “arm7_9 sw_bkpts enable” 命令使能软件断点，使用 bp 命令设置断点，使用 rbp 命令删除断点。如图 3.11 所示。

```

book@book-desktop:~$ telnet localhost 4444
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
Open On-Chip Debugger
> halt 暂停程序
target state: halted
target halted in ARM state due to debug request, current mode: Undefined
cpsr: 0x200000db pc: 0x00000034
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> load_image leds/leds.elf 加载代码到内部RAM中
168 byte written at address 0x00000000
downloaded 168 byte in 0.088846s
> resume 0x0 启动程序
> halt 暂停程序
target state: halted
target halted in ARM state due to debug request, current mode: Undefined
cpsr: 0x200000db pc: 0x00000024
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> arm7_9 sw_bkpts enable 使能软中断
software breakpoints enabled
> bp 0x84 4 在地址0x84处设置断点
breakpoint added at address 0x00000084
> bp 查看断点
0x00000084, 0x4, 0, 0x00
> resume 恢复运行程序
target state: halted
target halted in ARM state due to breakpoint, current mode: Undefined
cpsr: 0x600000db pc: 0x00000084
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> resume 在断点处停下了，再次恢复运行程序
target state: halted
target halted in ARM state due to breakpoint, current mode: Undefined
cpsr: 0x600000db pc: 0x00000084
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> resume 在断点处停下了，再次恢复运行程序
target state: halted
target halted in ARM state due to breakpoint, current mode: Undefined
cpsr: 0x600000db pc: 0x00000084
MMU: disabled, D-Cache: disabled, I-Cache: disabled
> rbp 0x84 删除断点
> bp 查看断点，无输出表示无断点
> resume 恢复运行程序，不会再停下了
>
  
```

图 3.11 在 telnet 界面操作 OpenOCD 调试程序

程序在断点停下时，可以看到 LED 也停止闪烁，恢复运行时 LED 就再次变化、再次停止。

至于为什么在 0x84 处设置断点，主要是想在 leds.c 中操作 GPBDA 寄存器以控制 LED 的代码处设置断点，如图 3.12 所示。

```

int main(void)
{
    unsigned long i = 0;

    GPBCON = GPB5_out|GPB6_out|GPB7_out|GPB8_out;    // 将LED1-4对应的GPB5/6/7/8四个引脚设为输出

    while(1){
        wait(30000);
        GPBDAT = ~(i<<5); // 根据i的值, 点亮LED1-4 想在此设置断点
        if(++i == 16)
            i = 0;
    }

    return 0;
}

```

图 3.12 leds.c 部分代码

可以从反汇编 leds.dis 中知道这段代码的地址在 0x84 处, 如图 3.13 所示。

```

00000044 <main>:
44: e52de004 push {lr} ; (str lr, [sp, #-4]!)
48: e24dd004 sub sp, sp, #4 ; 0x4
4c: e3a03000 mov r3, #0 ; 0x0
50: e58d3000 str r3, [sp]
54: e3a03456 mov r3, #1442840576 ; 0x56000000
58: e2833010 add r3, r3, #16 ; 0x10
5c: e3a02b55 mov r2, #87040 ; 0x15400
60: e5832000 str r2, [r3]
64: e3a00c75 mov r0, #29952 ; 0x7500
68: e2800030 add r0, r0, #48 ; 0x30
6c: ebffffe9 bl 18 <wait>
70: e3a02456 mov r2, #1442840576 ; 0x56000000
74: e2822014 add r2, r2, #20 ; 0x14
78: e59d3000 ldr r3, [sp]
7c: e1a03283 lsl r3, r3, #5
80: e1e03003 mvn r3, r3
84: e5823000 str r3, [r2] 给GPBDA寄存器赋值
88: e59d3000 ldr r3, [sp]
8c: e2833001 add r3, r3, #1 ; 0x1
90: e58d3000 str r3, [sp]
94: e59d3000 ldr r3, [sp]
98: e3530010 cmp r3, #16 ; 0x10
9c: 1afffff0 bne 64 <main+0x20>
a0: e3a03000 mov r3, #0 ; 0x0
a4: e58d3000 str r3, [sp]
a8: eaffffed b 64 <main+0x20>

```

图 3.13 反汇编文件 leds.dis 部分内容

从反汇编代码确定程序功能, 是件难度很高的事情, 所以一般不直接操作 OpenOCD 来调试程序, 而是通过其他工具来调试:

- ① 使用 arm-linux-gdb 调试器: 可以指定在 leds.c 的某行设置断点;
- ② 使用 Eclipse 来调试: 可以直接在源文件中通过双击设置断点。

3.2.3 使用 OpenOCD 下载 u-boot，通过 u-boot 烧写 Nor/Nand Flash

首页所说的“1 个命令 2 秒钟即可在裸板上启动 u-boot”，是通过在 telnet 界面上执行“script u-boot.cmd”命令实现的。u-boot.cmd 文件就是把下面(1)、(2)小节所描述的命令写在一起。

通过 OpenOCD 可以快速地读写开发板的 SDRAM，那么可以把 u-boot 先放到 SDRAM 中，然后启动它，就可以使用 u-boot 来烧写 Nor/Nand Flash 了——u-boot 的功能及速度都比一般的烧写工具要强。

在 SDRAM 能被访问之前，必须初始化存储控制器，这可以通过下载一个特定的小程序到内部 RAM 中，并运行它，由它来进行初始化，这个程序在/work/eclipse_projects/init 目录下。

(1) 下载、运行初始化程序 init.bin:

注意：下载程序之前先使用“halt”命令暂停单板，才能下载代码；如果使用“poll”命令发现单板的 MMU 或 D-cache 已经使能，则需要使用“arm920t cp15 2 0”命令禁止 MMU 和 D-cache。

init.bin 是位置无关的代码，所以无论是从 Nor Flash 启动，还是从 Nand Flash 启动，都可以将它下载到内部 RAM 去运行——不同启动方式下内部 RAM 的地址不同，Nor Flash 启动时，地址是 0x40000000；Nand Flash 启动时，地址是 0。

启动 OpenOCD(参考图 3.6)、telnet 登录(参考图 3.7)后，执行以下命令下载、运行 init.bin:

① 从 Nand Flash 启动:

```
load_image init/init.bin 0x0
resume 0x0
```

② 从 Nor Flash 启动:

```
load_image init/init.bin 0x40000000
resume 0x40000000
```

这时，存储控制器已经初始化好，下面就可以把 u-boot 下载到 SDRAM 去了。

(2) 下载、运行 u-boot:

u-boot 的可执行文件为/work/eclipse_projects/u-boot/u-boot.bin，接着在 telnet 界面执行以下命令，即可启动 u-boot:

```
halt
load_image u-boot/u-boot.bin 0x33f80000
resume 0x33f80000
```

然后，就可以在串口工具上看到 u-boot 的启动信息，如下:

```
U-Boot 1.1.6 (Sep 23 2008 - 02:50:03)

DRAM: 64 MB
Flash: 1 MB
NAND: 64 MiB
```

```

*** Warning - bad CRC, using default environment
In:    serial
Out:   serial
Err:   serial

Use these steps to program the image to flash:
1. In OpenOCD
   Run the 'halt' command to halt u-boot
   Run the 'load_image <file> <address>' command to load file to SDRAM
   Run the 'resume' command to resume u-boot
2. In u-boot, use the flash commands to program the image to flash
Or, use the tftp or nfs command to download file, and then program the flash.
OpenJTAG>

```

现在，可以通过 u-boot 来烧写文件了，有两种方法将文件下载到单板内存中：

① 通过 OpenOCD 的命令：

这需要在 telnet 界面，执行以下 3 个命令：

```

halt: 暂停u-boot
load_image <file> <address>: 将文件烧写到内存中(这个地址一般取0x30000000)
resume: 恢复运行u-boot

```

② 通过 u-boot 的网络命令：

可以使用 tftp 命令，或者 nfs 命令，这需要在 PC 上启动 TFTP 服务或者 NFS 服务。

当文件被下载到单板内存中后，就可以使用 u-boot 的各种 Flash 命令进行烧写了。图 3.14、图 3.15 是两个例子，分别使用上述两种下载方式，将 u-boot 本身烧写到 Nor Flash、Nand Flash 上去。

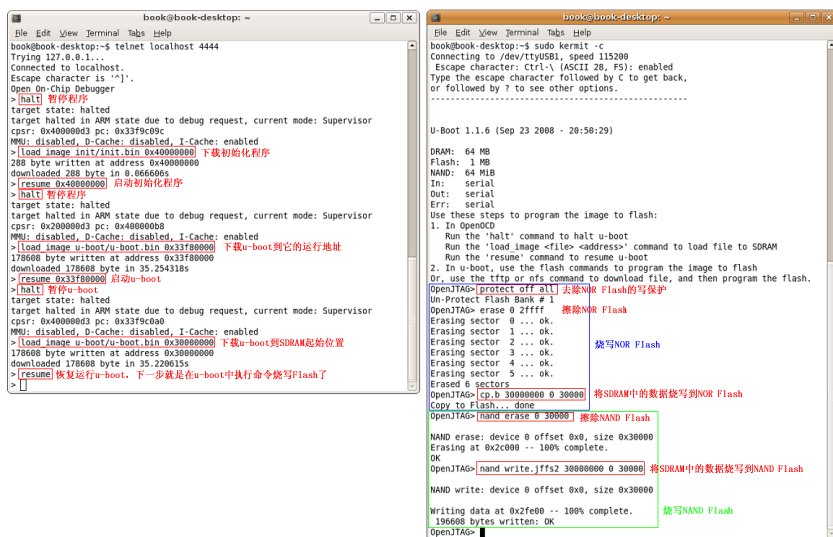


图 3.14 通过 OpenOCD 启动 u-boot、传输数据，通过 u-boot 烧写 Flash

```

book@book-desktop: ~
File Edit View Terminal Tabs Help
book@book-desktop:~$ telnet localhost 4444
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
Open On-Chip Debugger
> [halt] 暂停程序
target state: halted
target halted in ARM state due to debug request, current mode: Supervisor
cpsr: 0x40000053 pc: 0x33f9c09c
MMU: disabled, D-Cache: disabled, I-Cache: enabled
> [load_image init/init.bin 0x40000000] 下载初始化程序
288 byte written at address 0x40000000
downloaded 288 byte in 0.201311s
> [resume 0x40000000] 启动初始化程序
target state: halted
target halted in ARM state due to debug request, current mode: Supervisor
cpsr: 0x20000003 pc: 0x400000bc
MMU: disabled, D-Cache: disabled, I-Cache: enabled
> [load_image u-boot/u-boot.bin 0x33f80000] 下载u-boot到它的运行地址
178608 byte written at address 0x33f80000
downloaded 178608 byte in 44.554943s
> [resume 0x33f80000] 启动u-boot
>

book@book-desktop: ~
File Edit View Terminal Tabs Help
U-Boot 1.1.6 (Sep 23 2008 - 20:58:29)

DRAM: 64 MB
Flash: 1 MB
NAND: 64 MiB
*** Warning - bad CRC, using default environment

In: serial
Out: serial
Err: serial

Use these steps to program the image to flash:
1. In OpenOCD
   Run the 'halt' command to halt u-boot
   Run the 'load_image <file> <address>' command to load file to SDRAM
   Run the 'resume' command to resume u-boot
2. In u-boot, use the flash commands to program the image to flash
Or, use the 'tftp or nfs command' to download file, and then program the flash.
OpenJTAG> get_ipaddr 192.168.0.17 0x200000
OpenJTAG> set_serverip 192.168.0.57 设置服务器IP
OpenJTAG> nfs 30000000 192.168.0.57:/work/eclipse_projects/u-boot/u-boot.bin
File transfer via NFS from server 192.168.0.57; our IP address is 192.168.0.17
Filename '/work/eclipse_projects/u-boot/u-boot.bin'.
Load address: 0x30000000
Loading: #####
done.
Bytes transferred = 178608 (2b9b0 hex)
OpenJTAG> protect off all 去除NOR Flash的写保护
Un-Protect Flash Bank # 1 擦除NOR Flash
OpenJTAG> erase 0 2ffff
Erasing sector 0 ... ok.
Erasing sector 1 ... ok.
Erasing sector 2 ... ok.
Erasing sector 3 ... ok.
Erasing sector 4 ... ok.
Erasing sector 5 ... ok.
Erased 6 sectors
OpenJTAG> cp.b 30000000 0 $(filesize) 将SDRAM中的数据烧写到NOR Flash
Copy to Flash... done
OpenJTAG> nand erase 0 30000 擦除NAND Flash
NAND erase: device 0 offset 0x0, size 0x30000
Erasing at 0x2c000 -- 100% complete.
OK
OpenJTAG> nand write.jffs2 30000000 0 $(filesize) 将SDRAM中的数据烧写到NAND Flash
NAND write: device 0 offset 0x0, size 0x2b9b0
Writing data at 0x2b800 -- 100% complete.
178608 bytes written: OK
OpenJTAG>
  
```

图 3.15 通过 OpenOCD 启动 u-boot，通过 u-boot 下载、烧写 Flash

注意：

- u-boot 的命令把所有的数字当作 16 进制，所以不管是否在数字前加前缀“0x”，这个数字都是 16 进制的。
- 图 1.29、图 1.30 中，擦除 Flash 的长度、烧写的数据长度，这些数值都是根据要烧写的文件的长度确定的。u-boot.bin 的长度是 178704 字节，即 0x2BA10 字节，上面使用的长度都是 0x30000，一是为了与 Flash 的可擦除长度相配(16K 的整数倍)，二是方便。
- 在 u-boot 中使用 nfs 命令之前，要先启动 NFS 服务：首先在 Ubuntu 的/etc/exports 文件增加一行“/work/eclipse_projects *(rw, sync, no_root_squash)”；然后重启 NFS 服务(运行“sudo /etc/init.d/nfs-kernel-server restart”命令)

完成烧写之后，重启单板即可看到运行情况。

首页所说的“再加 3 个命令即可烧写单板”，就是指下载、擦除、烧写这 3 个命令。

/work/eclipse_projects/u-boot/u-boot.bin 是经过改造的，它有如下特点：

- 支持 S3C2410、S3C2440；
- 可以从 Nor Flash 或 Nand Flash 启动；
- 可以烧写 Nor Flash 和 Nand Flash；
- 支持 CS8900 或 DM9000 网卡、USB 传输(通过 OpenOCD 下载、启动时，USB 功能不可用)；
- 支持烧写 jffs2、yaffs 文件系统映像文件

注意：单板从 Nand Flash 启动时，无法操作 Nor Flash。

3.3 使用 Eclipse 进行开发

3.3.1 启动 Eclipse，进行简单设置：

从图 2.4 所示的启动菜单中启动 Eclipse，等待一会，可以看到如图 3.16 所示界面。在里面设置工作目录为/work/eclipse_projects，并选择“Use this as the default and do not ask again”，表示使用/work/eclipse_projects 作为缺省目录，并不再提示。

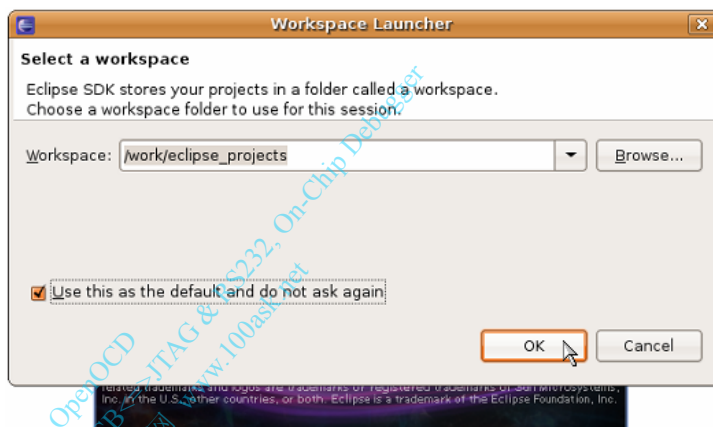


图 3.16 设置 Eclipse 工作目录

在图 3.16 中，点击 OK 后，得到如图 3.17 所示界面。

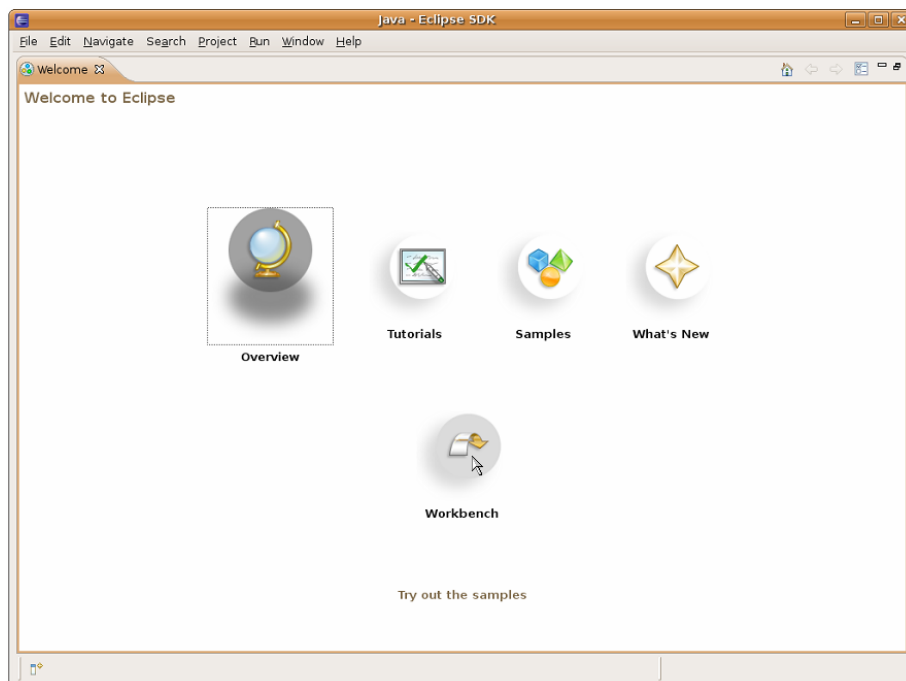


图 3.17 Eclipse 启动后的界面

图 3.17 中下边的图标名为“Workbench”，点击它，得到图 3.18 所示界面。

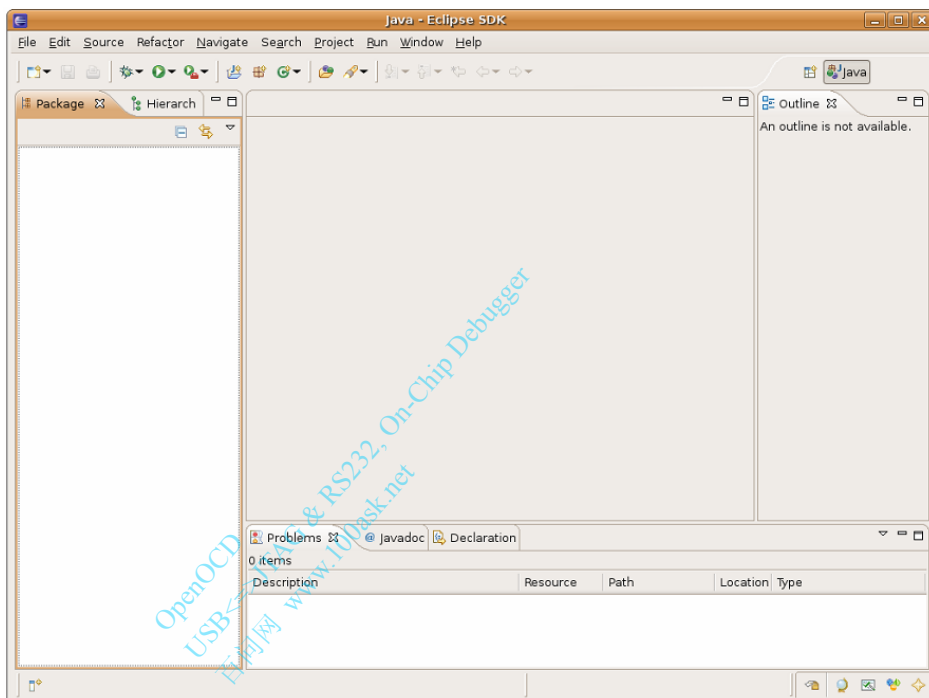


图 3.18 Eclipse 工作界面

3.3.2 新建一个 Eclipse 工程:

(1) 创建工程:

在图 3.18 的界面中，选择菜单项“File -> New -> Project”，如图 3.19 所示。

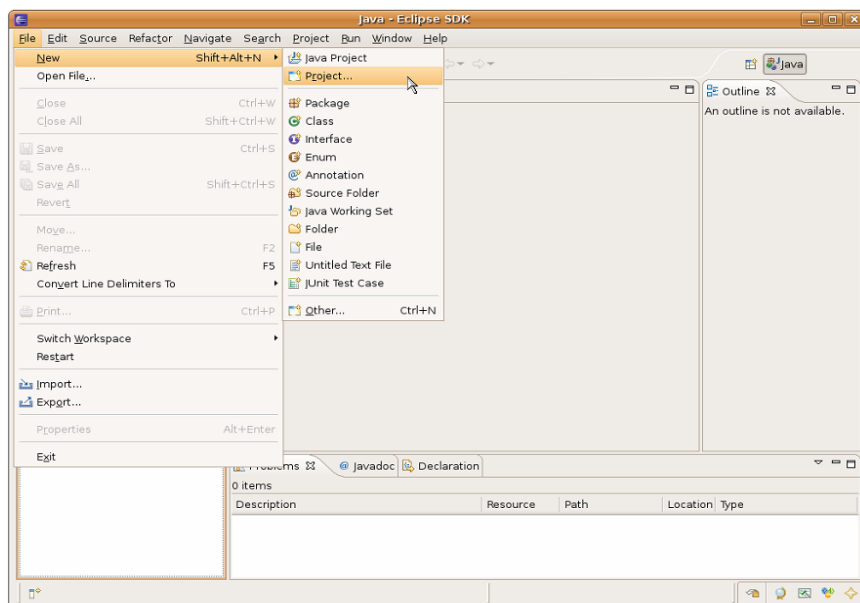


图 3.19 新建一个工程(步骤 1)

在随后出现的窗口中(如图 3.20 所示), 选择 “C Project”:

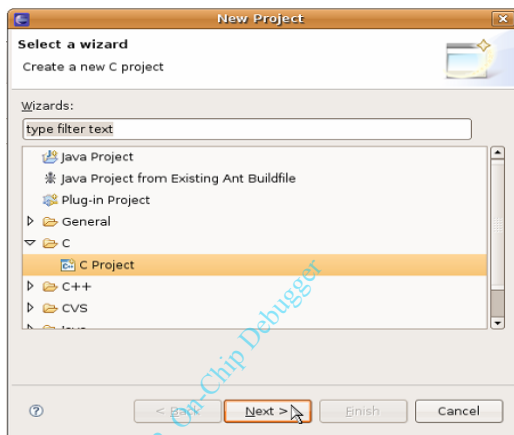


图 3.20 新建一个工程(步骤 2)

点击“Next”，在图 3.21 中指定工程名字、选择“Makefile project -> Empty Project”、“Other Toolchain”，然后点击 “Finish” 按钮。

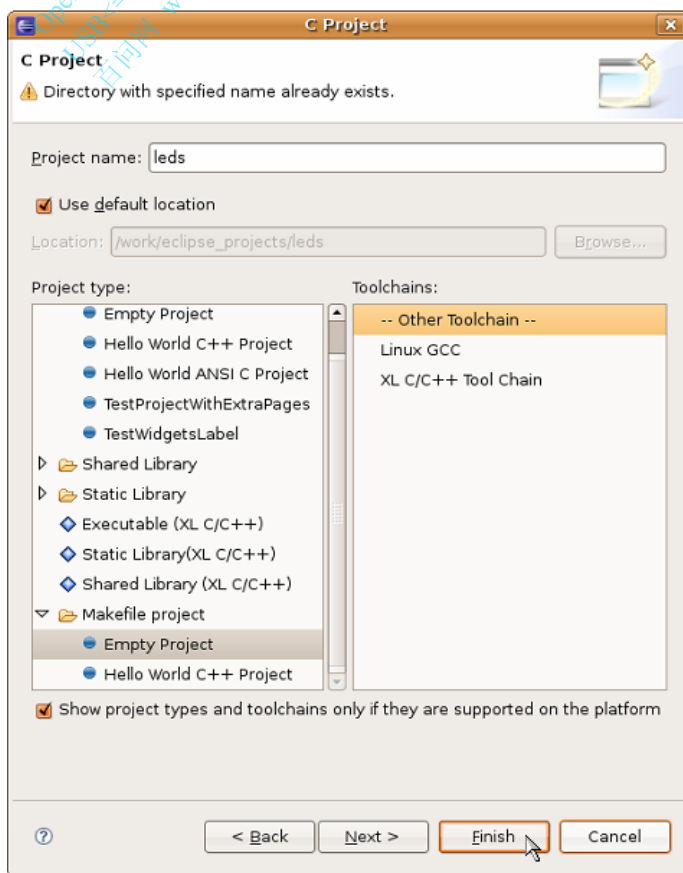


图 3.21 新建一个工程(步骤 3)

紧接着会出现如图 3.22 所示的窗口，选中“Remember my decision”，然后点击“Yes”，表示打开“C/C++ perspective”。

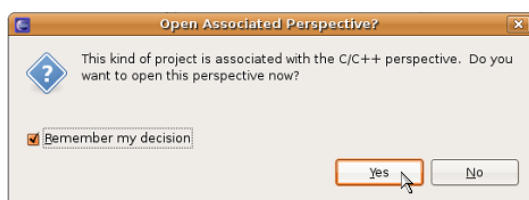


图 3.22 选择打开“C/C++ perspective”（步骤 4）

(2) 添加文件：

在创建 leds 工程时，如果/work/eclipse_projects/leds 目录下已经有源文件，则这些文件会被自动加入工程中。如果/work/eclipse_projects/leds 目录是空的话，或者想添加新文件时，可以使用以下方法导入文件。

下面先在/work/eclipse_projects/下建立一个测试文件 test_file_to_add.txt，然后以它为例介绍怎么添加文件到工程里。

选择菜单项“File -> Import...”，然后选择“File System”作为文件来源，如图 3.23 所示。

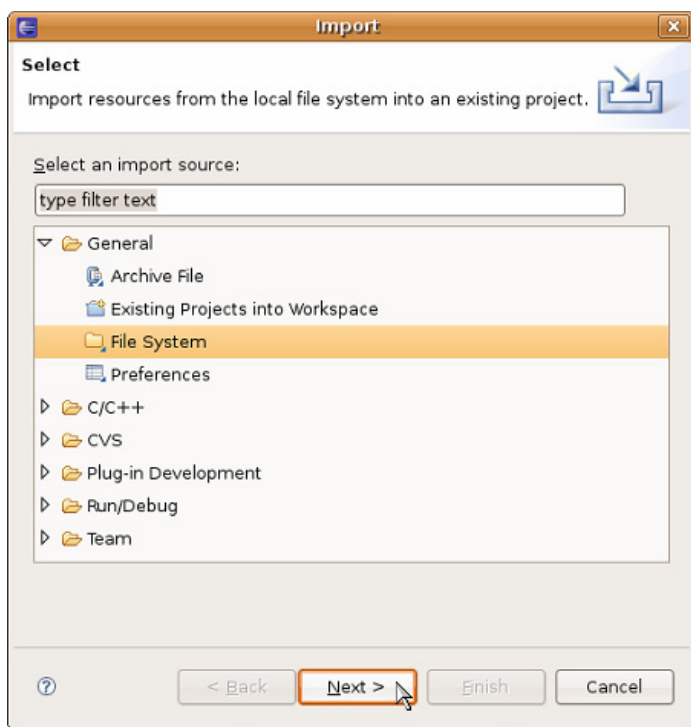


图 3.23 导入文件（步骤 1）

点击“Next”，打开“Import”对话框，在里面指定源目录“From directory”、选择要导入的文件、指定目的目录“Into folder”。如图 3.24，点击“Finish”按钮即可添加文件。

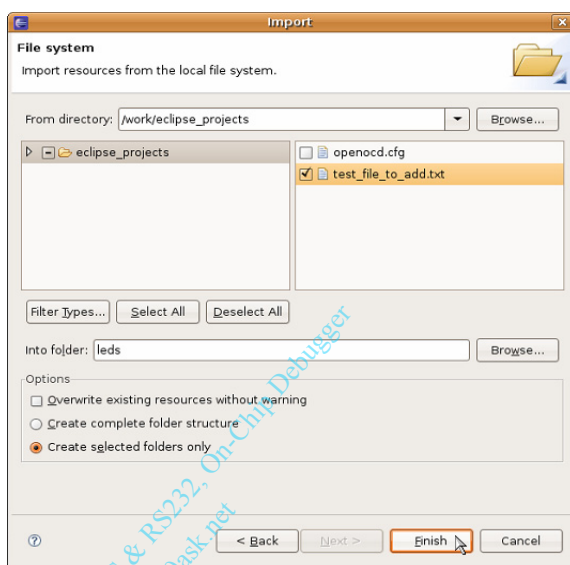


图 3.24 导入文件(步骤 2)

至此，新的 Eclipse 工程建立完毕，如图 3.25 所示。双击左边的文件名，即可打开进行编辑。

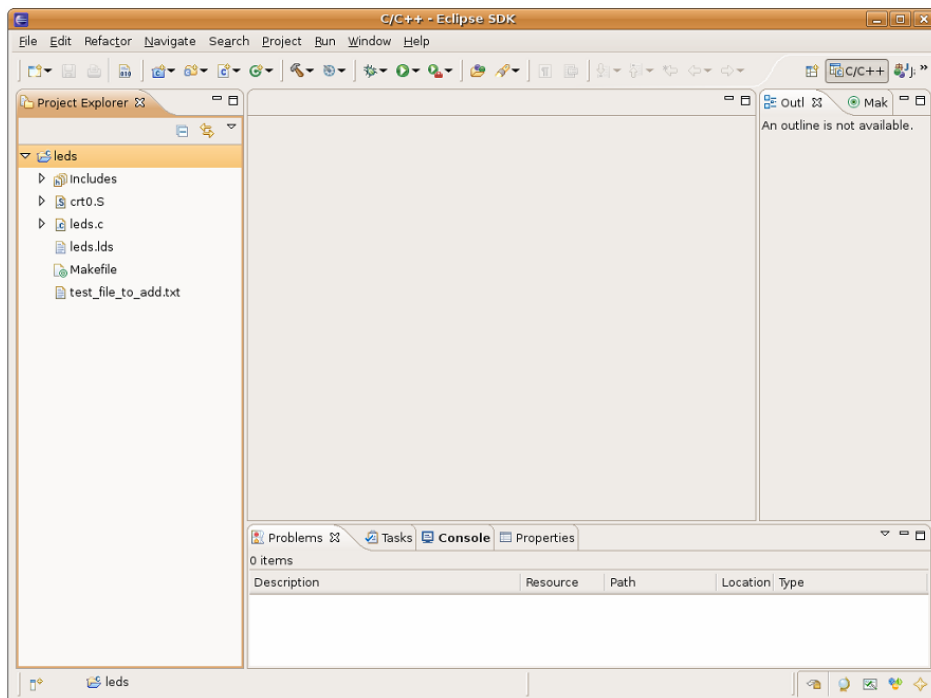


图 3.25 Eclipse 工程建立完成

3.3.3 配置 Eclipse 工程:

建立一个 Eclipse 工程后,需要进行一些配置。在“Project”菜单里,点击去掉“Build Automatically”前面的标记,表示不自动编译工程,如图 3.26 所示。

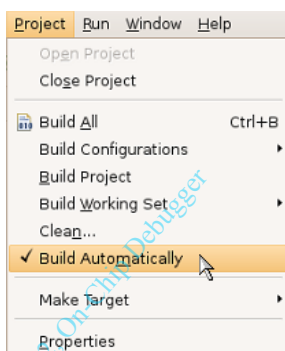


图 3.26 配置工程(步骤 1)

然后再次选择“Project”菜单,点击“Properties”项,如图 3.27 所示。

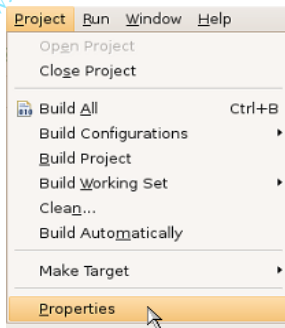


图 3.27 配置工程(步骤 2)

新出现的窗口如图 3.28 所示。

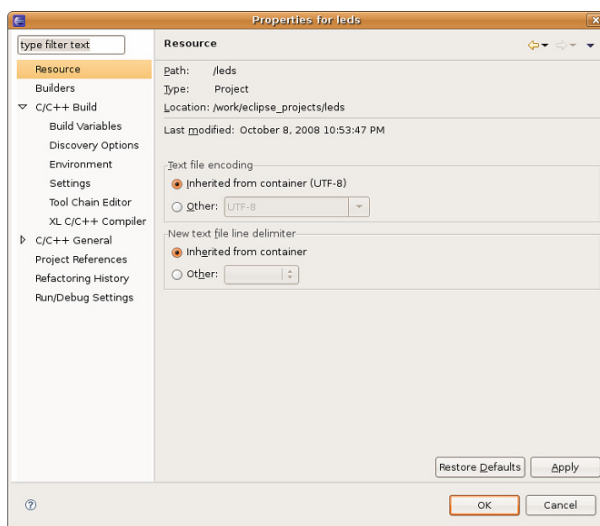


图 3.28 配置工程(步骤 3)

选择其中的“C/C++ Build -> Discovery options”，将“Compiler invocation command”改为 arm-linux-gcc，如图 3.29 所示。

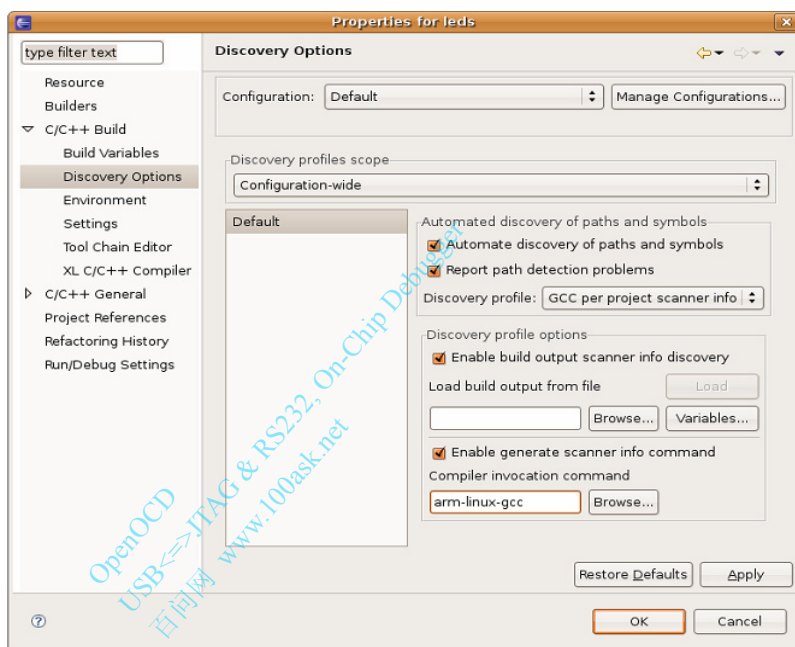


图 3.29 配置工程 (步骤 4)

继续选择“C/C++ Build -> Settings”，在“Binary Parsers”中选择“GNU Elf Parser”（点击“GNU Elf Parser”文字，然后勾选），然后将“addr2line Command”、“c++filt Command”换成 arm-linux-addr2line 和 arm-linux-c++filt，如图 3.30 所示。

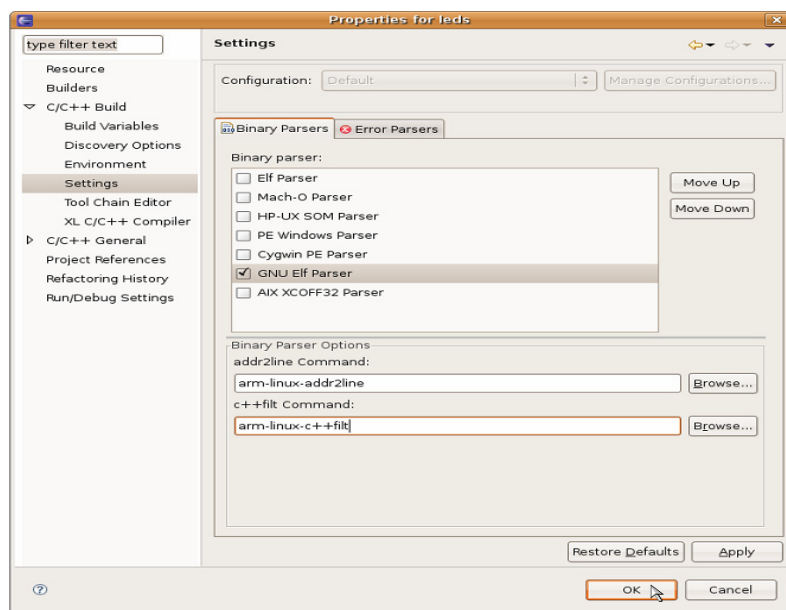


图 3.30 配置工程 (步骤 5)

3.3.4 编译、清除程序：

当配置完成后，就可以使用按钮来编译、清除程序了。点击菜单“Project -> Build Project”开始编译程序，如图 3.31 所示；编译过程见图 3.32。

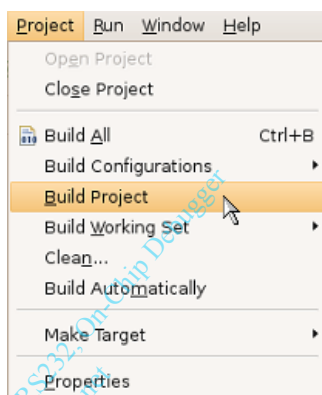


图 3.31 点击此菜单项开始编译程序

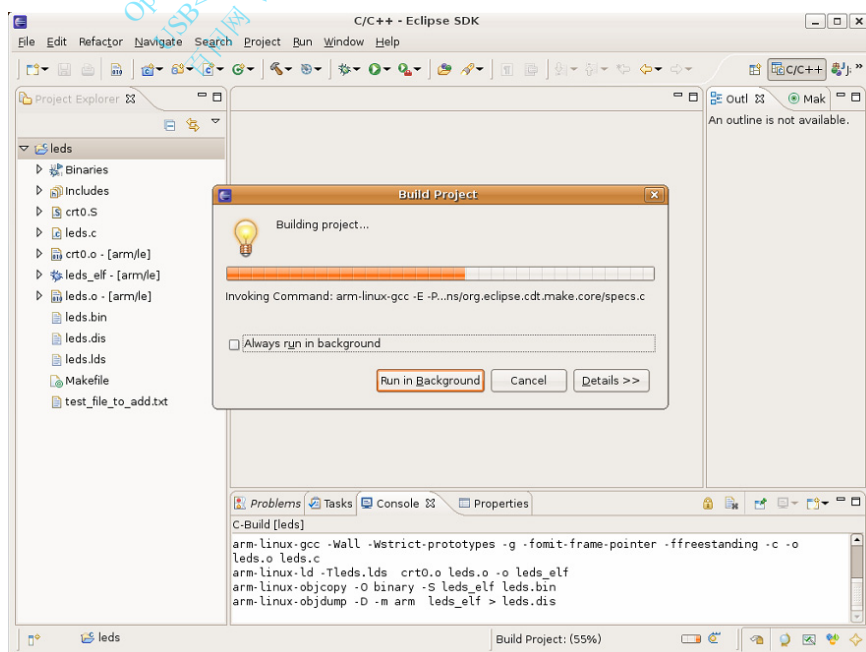


图 3.32 编译程序过程

编译的效果与在命令行执行“make all”命令完全一样。

清除程序的方法是，选择菜单“Project -> Clean...” (如图 3.33)，即可弹出一个对话框(如图 3.34)。

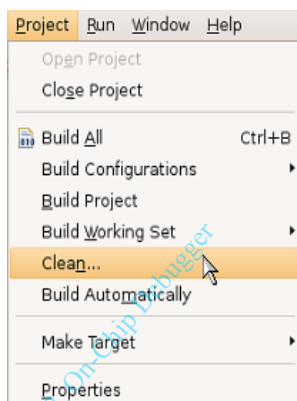


图 3.33 点击此菜单项弹出清除对话框

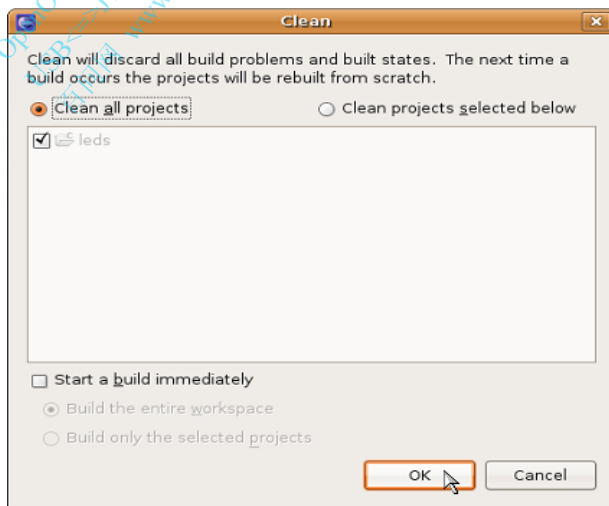


图 3.34 点击此菜单项弹出清除对话框

在图 3.34 中，不要选择“Start a build immediately”，然后点击“OK”即可清除程序。

3.3.5 使用 Eclipse 调试程序：以 leds、u-boot、Linux 内核为例

在调试之前，需要配置调试器、启动 OpenOCD——这也可以在 Eclipse 里启动；如果程序是在 SDRAM 中运行，还要使用 init.bin 程序来初始化存储控制器。

下面以三个例子来进行说明，leds 程序在内部 RAM 运行，无需运行 init.bin 来进行初始化；u-boot 程序在 SDRAM 中运行，需要运行 init.bin 来进行初始化；先通过 u-boot 下载内核，然后直接调试。

假设已经建立好 leds 工程、u-boot 工程、linux-2.6.22.6 工程。

(1) 配置调试器:

点击调试图标  右边的小箭头，如图 3.35 所示，选择 “Debug Configurations...”。

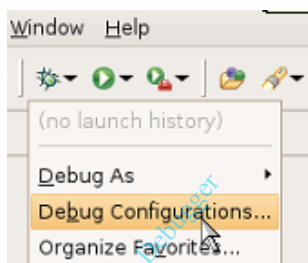


图 3.35 配置调试器(步骤 1)

弹出的窗口如图 3.36 所示，在里面可以看到 Zylín 字样的调试器。

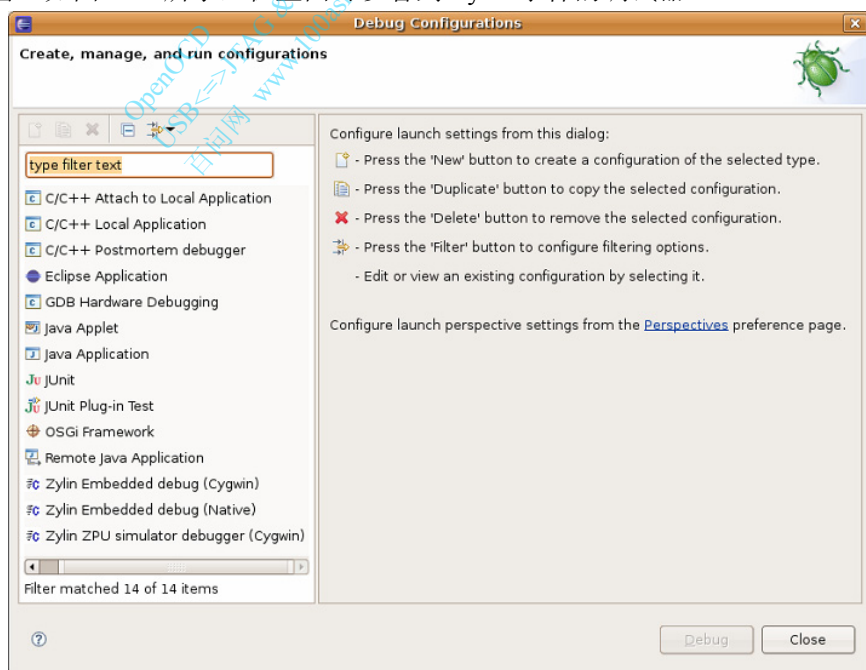


图 3.36 配置调试器(步骤 2)

选择里面的“Zylin Embedded debug (Native)”，如图 3.37 所示。

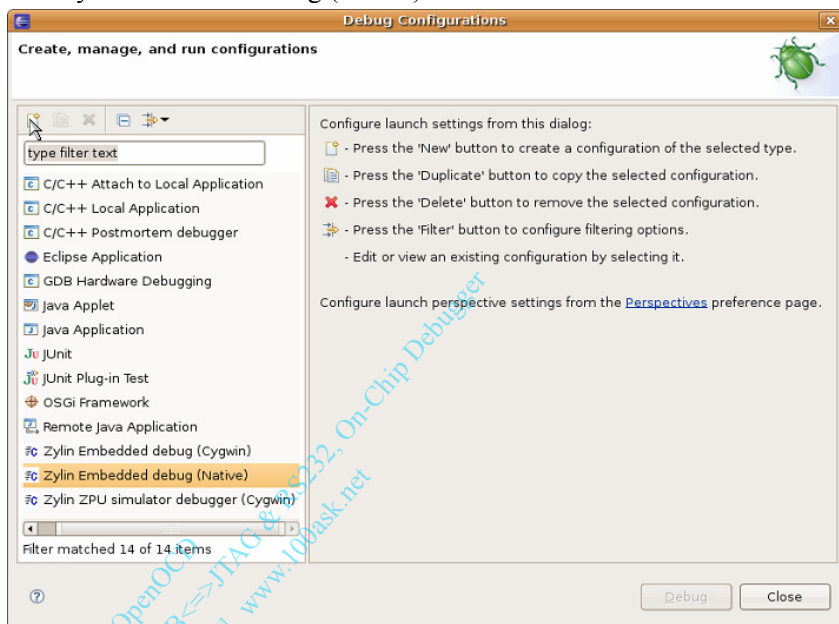


图 3.37 配置调试器(步骤 3)

在图 3.37 中，然后点击左上角的  按钮后，配置界面将发生变化，如图 3.38 所示，对于 leds 工程，图 3.38 中的名字是“leds Default”；对于 u-boot 工程则是“u-boot Default”，对于 linux-2.6.22.6 工程则是“linux-2.6.22.6 Default”。

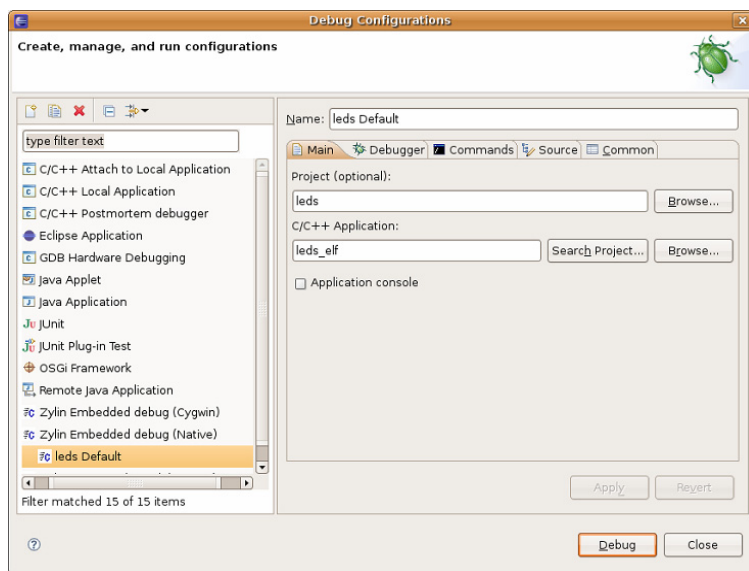


图 3.38 配置调试器(步骤 4)

现在来配置调试器。

首先选择 “C/C++ Application”，点击底下的 “Browse” 按钮，选中 ELF 的可执行文件，对于 leds 工程，这个文件是 leds_elf；对于 u-boot 工程，这个文件是 u-boot；对于 linux-2.6.22.6 工程，这个文件是 vmlinux。如图 3.39 所示。

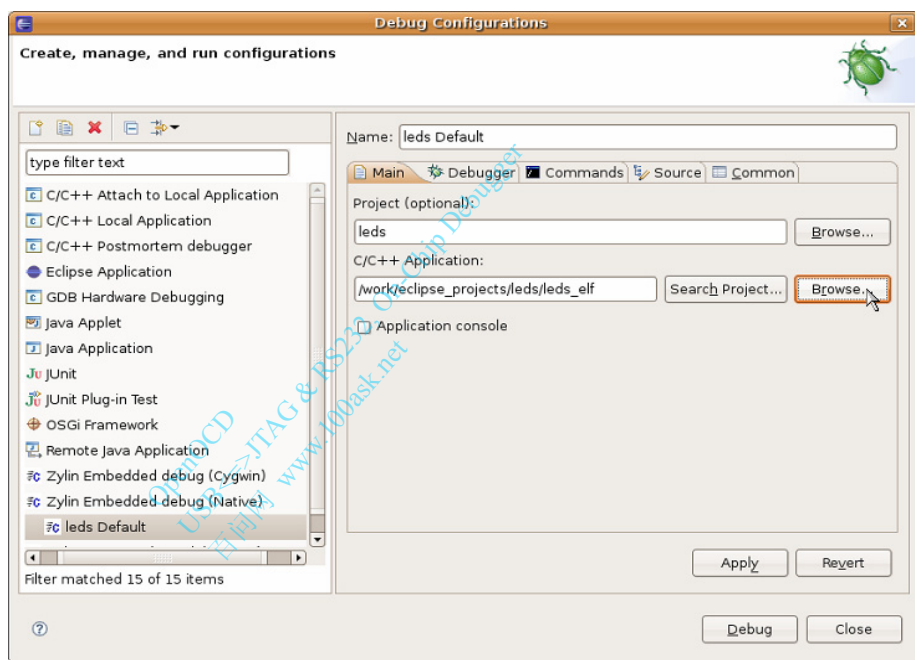


图 3.39 配置调试器(步骤 5)

然后选择调试器为 arm-linux-gdb，如图 3.40 所示：设置“GDB debugger”为“arm-linux-gdb”，设置 “GDB command file” 为空。

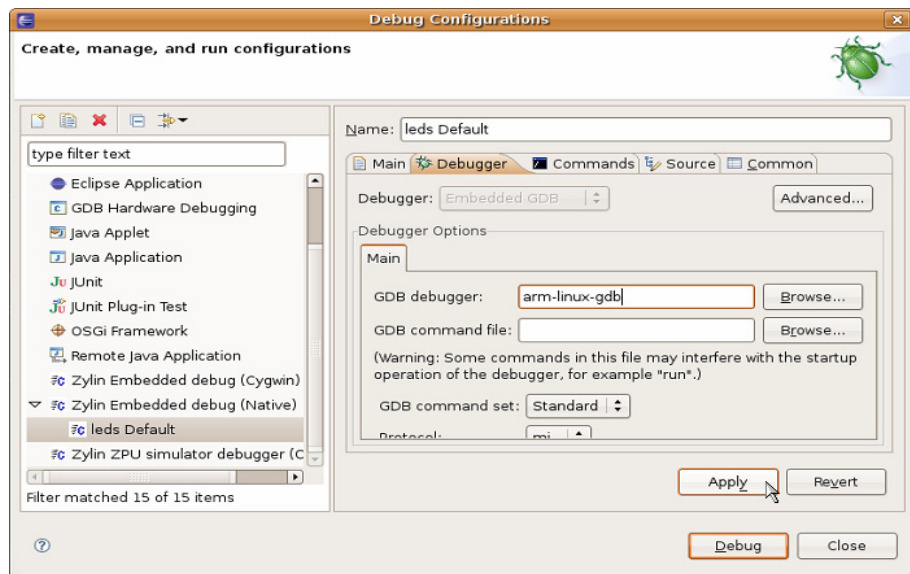


图 3.40 配置调试器(步骤 6)

最后设置启动调试器 arm-linux-gdb 后，自动运行的命令，如图 3.41 所示。

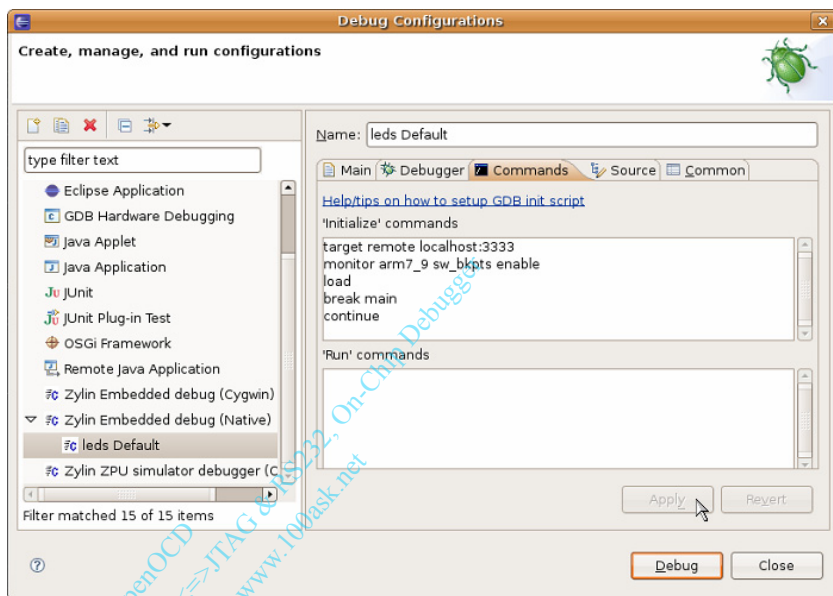


图 3.41 配置调试器(步骤 7)

当上述设置都完成后，然后点击“Apply”、“Close”退出。

这些命令，就是各个工程下 gdb.cmd 文件的内容，以 leds、u-boot 工程为例(linux-2.6.22.6 工程的调试稍为复杂，后面单独说明)：

① 对于 leds 工程，这些命令如下(“//”开始的是注释，不要把它们写进 Eclipse 中)：

```
target remote localhost:3333           // 登录本地3333接口，即连接OpenOCD
monitor arm7_9 sw_bkpts enable         // 使能软中断
load                                  // 加载leds_elf，elf格式的文件包含地址，所以不用指定
break main                            // 在main函数设置断点
continue                              // 运行程序，它会在执行到main函数时停下来
```

② 对于 u-boot 工程，命令与上面类似，只不过由于 u-boot 是在 SDRAM 中运行，所以要先使用 init.bin 初始化存储控制器(“//”开始的是注释，不要把它们写进 Eclipse 中)：

```
target remote localhost:3333           // 登录本地3333接口，即连接OpenOCD
monitor arm7_9 sw_bkpts enable         // 使能软中断
monitor load_image /work/eclipse_projects/init/init.bin 0x0 // 下载init.bin到内部RAM中
monitor resume 0x0                    // 运行init.bin
monitor halt                           // 停止运行init.bin
load                                   // 加载u-boot，elf格式的文件包含地址，所以不用指定
break start_armboot                   // 在start_armboot函数设置断点
continue                              // 运行程序，它会在执行到start_armboot函数时停下来
```


至此，其实已经可以调试程序了：您可以在命令行启动 OpenOCD(启动方法请参考 3.2.1 小节)，然后参考“(3) 调试程序”的内容来调试。

(2) 配置 Eclipse 的外部工具——目的是通过点击按钮启动 OpenOCD:

点击调试图标 右边的小箭头，如图 3.42 所示，选择“External Tools Configurations...”。

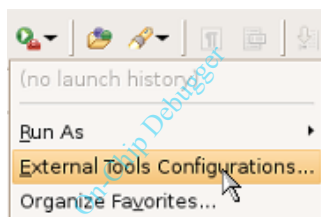


图 3.42 配置外部工具(步骤 1)

弹出的窗口如图 3.43 所示，在里面选中“Program”，然后点击左上角的按钮。

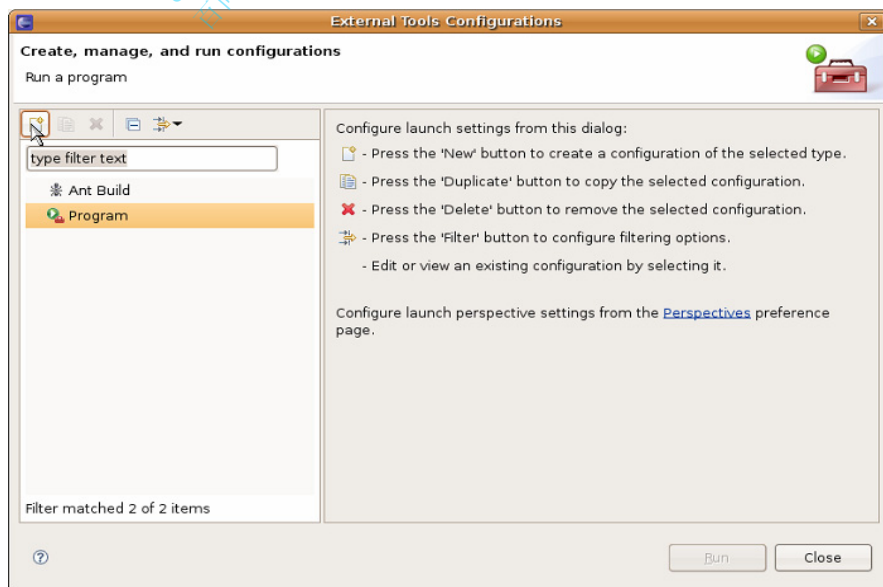


图 3.43 配置外部工具(步骤 2)

这时候配置界面发生了变化，如图 3.44、图 3.45 所示，按照图中的红框进行设置，然后点击“Apply”、“Close”退出。

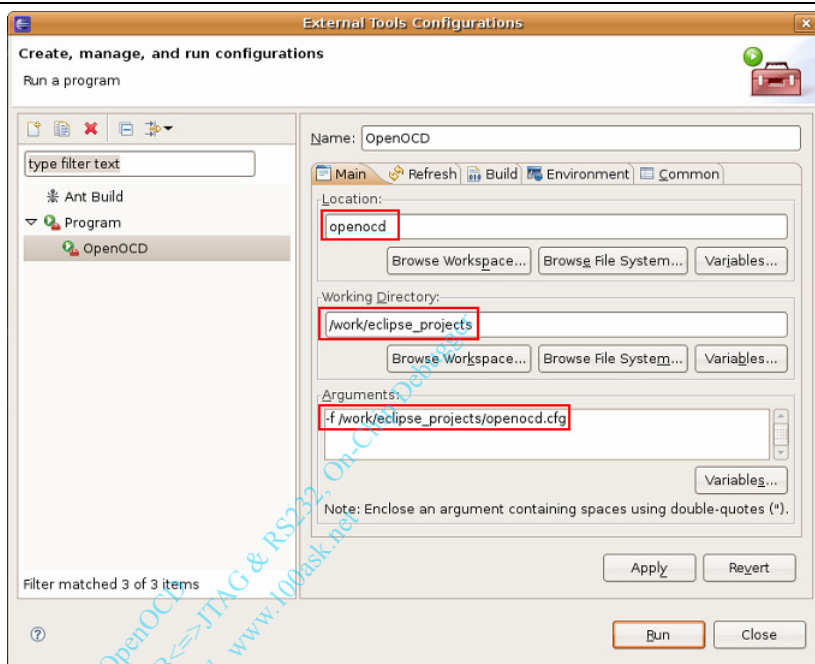


图 3.44 配置外部工具(步骤 3)

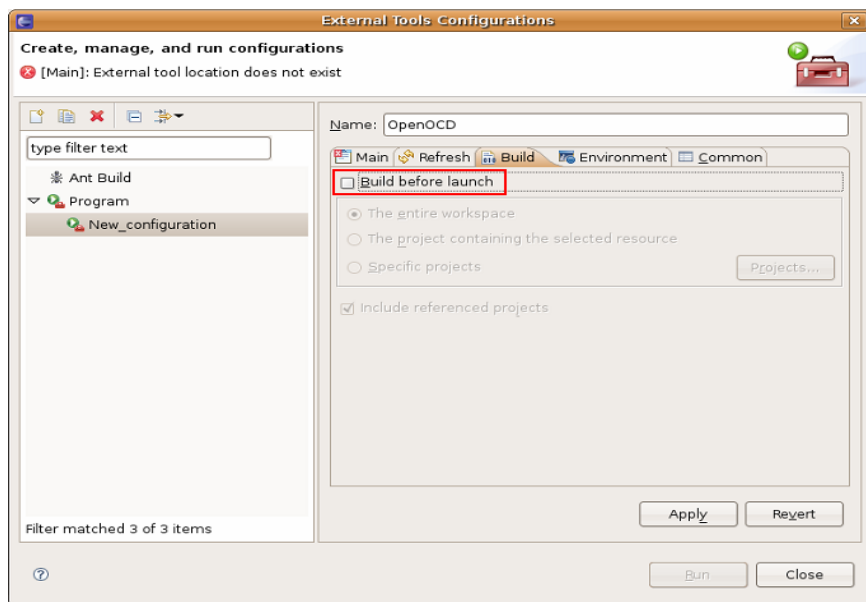


图 3.45 配置外部工具(步骤 4)

(3) 调试程序：

① 在调试程序之前，需要先启动 OpenOCD，有两种方法：

a. 可以通过命令行启动 OpenOCD(启动方法请参考 3.2.1 小节)；

b. 如果参照“(2) 配置 Eclipse 的外部工具”配置了外部工具，则可以点击图标右边的小箭头，选择“External Tools Configurations...”，进入如图 3.46 所示的界面，然后点击“Run”按钮启动 OpenOCD。

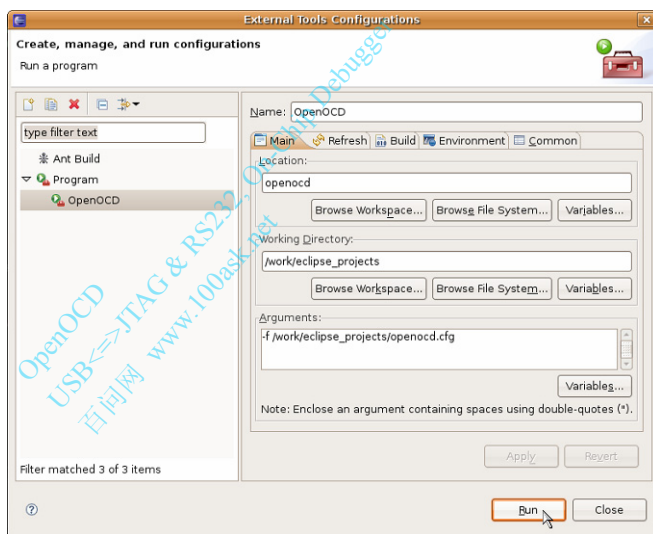


图 3.46 在对话框里通过按钮启动 OpenOCD

注意：第一次通过按钮启动 OpenOCD 之后，以后要启动它时，就可以直接点击图标

右边的小箭头，从中选择“OpenOCD”，如图 3.47 所示。OpenOCD 在 Eclipse 的整个运行过程中，只需要启动 1 次。

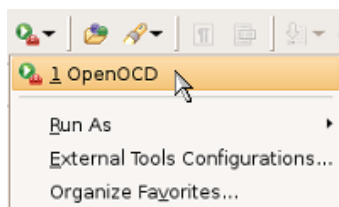


图 3.47 在下拉框中通过按钮启动 OpenOCD

② 启动调试器

点击调试图标右边的小箭头，选择“Debug Configurations...”，进入如图 3.48 所示的界面，然后点击“Debug”按钮启动调试器。

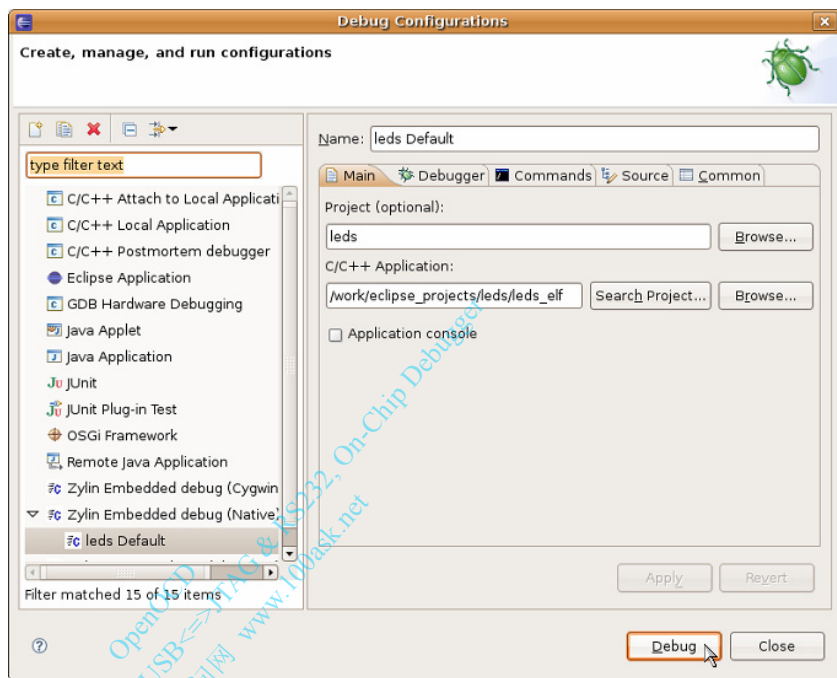


图 3.48 在对话框里通过按钮启动调试器

注意：第一次通过按钮启动调试器之后，以后要启动它时，就可以直接点击图标  右边的小箭头，从中选择“leds Default” (对于 leds 工程)，如图 3.49 所示。

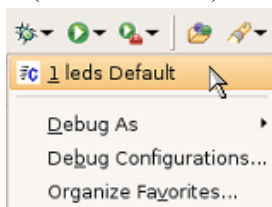


图 3.49 在下拉框中通过按钮启动调试器

③ 调试程序简介：以 leds 工程为例

调试器启动后，它会根据图 3.41 所设置的命令，将代码下载到单板上，设置断点，运行。

首先出现如图 3.50 所示的对话框询问是否进入调试界面，选择“Remember my decision”，然后选择“Yes”，表示以后调试器启动自动进入调试界面。

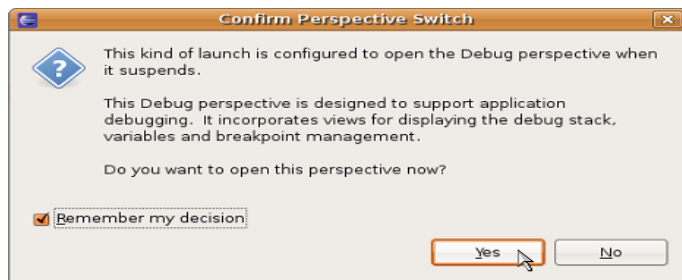


图 3.50 设置启动调试器时自动进入调试界面

紧接着出现的就是调试界面，如图 3.51 所示。

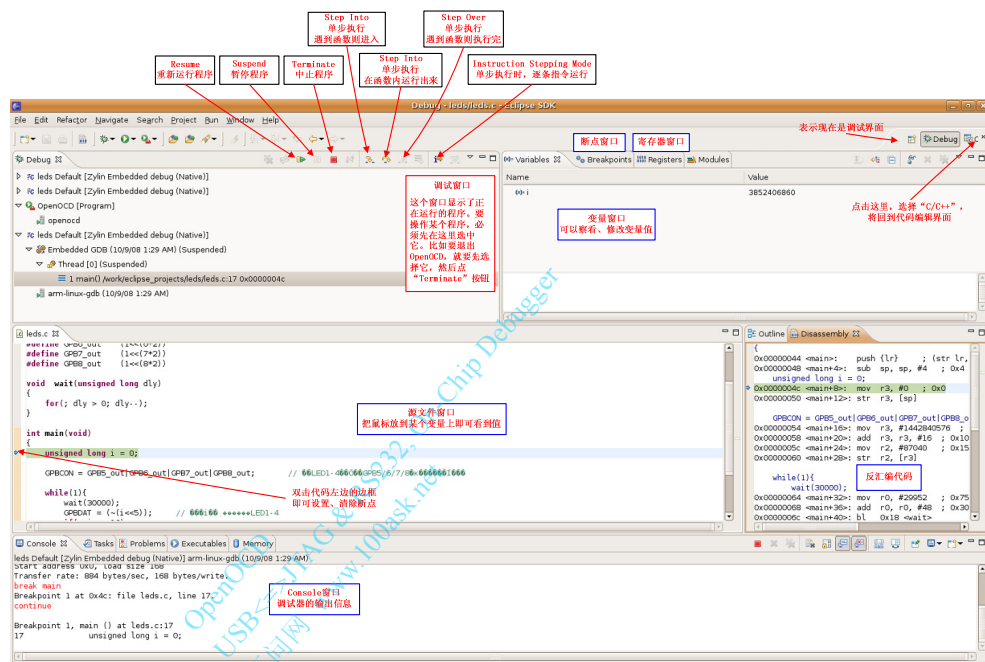


图 3.51 设置启动调试器时自动进入调试界面

在调试界面中，可以很方便地进行各类操作。比如双击代码“GPBDAT = (~i<<5));”

// 根据 i 的值，点亮 LED1-4”左边的边框就可以设置断点，然后不断点击“Resume”

按钮 ，就可以看到 4 个 LED 逐渐地变化。

④ 调试程序简介：以 u-boot 工程为例

注意：调试网上下载的 u-boot 时，需要定义 **CONFIG_SKIP_LOWLEVEL_INIT**，它表示“跳过底层的初始初始化”，就是不要初始化存储控制器，不要再次复制 u-boot 本身到 SDRAM 中。对于光盘中的 u-boot，已经增加的自动识别代码，无需定义这个宏。

/work/eclipse_projects/u-boot 目录下的 u-boot.bin、u-boot 是 u-boot-1.1.6_OpenJTAG.tar.bz2 在 Linux 下编译好的可执行文件，u-boot.bin 是二进制格式，u-boot 是 elf 格式。

首先，在 /work/eclipse_projects/u-boot 目录下将 u-boot-1.1.6_OpenJTAG.tar.bz2 解压缩。

然后，仿照前面创建 leds 工程的方法创建 u-boot 工程。

最后，为 u-boot 重新配置调试器，配置内容如图 3.52~3.56 所示。

注意：

- a. 图 3.54 中的“Commands”显示不全，它们的内容如下：

```
target remote localhost:3333
monitor arm7_9 sw_bkpts enable
monitor load_image /work/eclipse_projects/init/init.bin 0x0
monitor resume 0x0
monitor halt
load
break start_armboot
continue
```

- b. 图 3.55~3.56 指定源码位置：由于 u-boot 代码不在/work/eclipse_projects/u-boot 目录下，而是在次级目录 u-boot-1.1.6_OpenJTAG 下，需要自己指定。

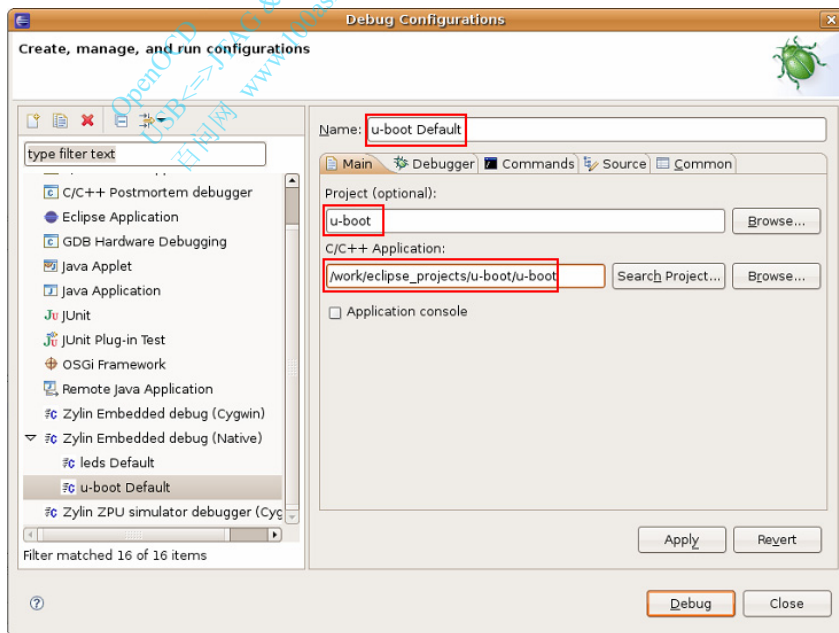


图 3.52 u-boot 工程调试器配置信息(1)

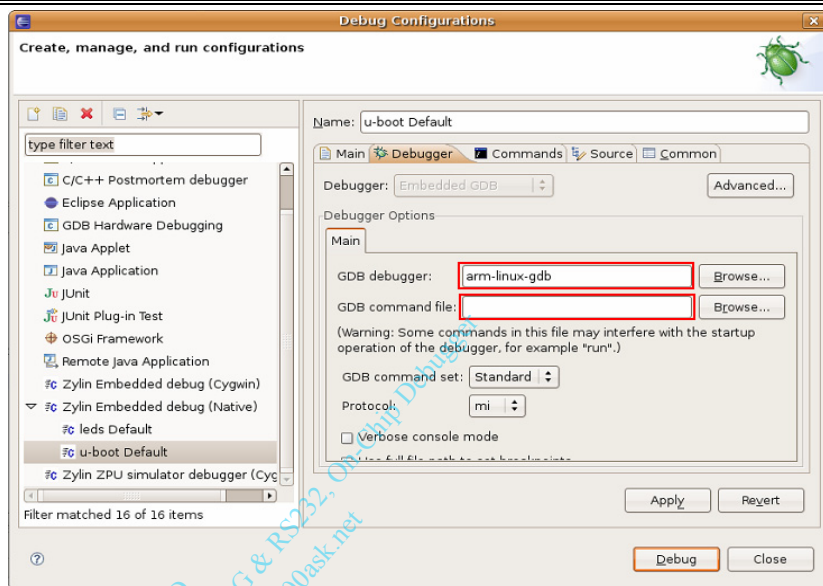


图 3.53 u-boot 工程调试器配置信息 (2)

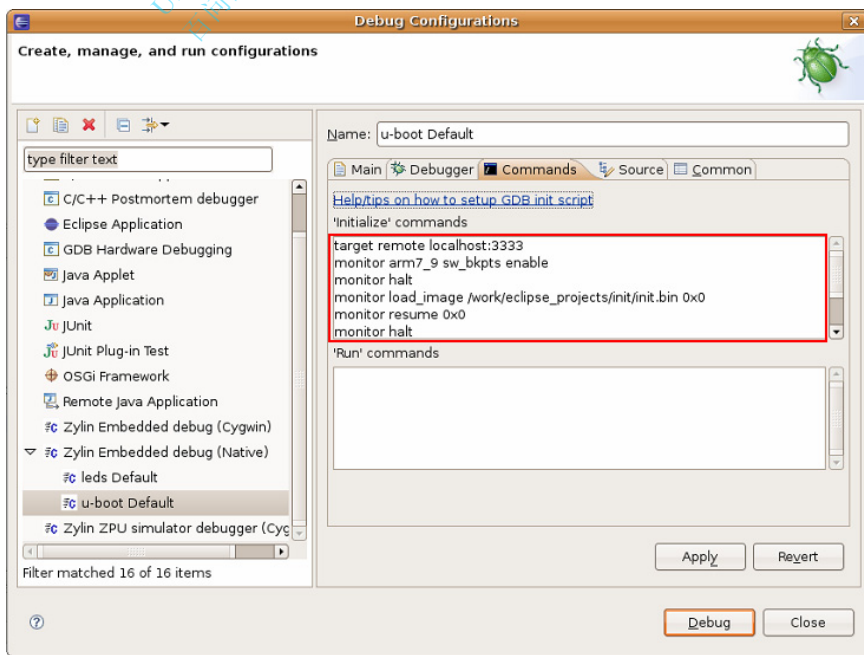


图 3.54 u-boot 工程调试器配置信息 (3)

图 3.55~3.56 是修改源码路径。在图 3.55 中，先选中“Default”，然后点击“Remove”按钮去除；在图 3.56 中，先点击“Add”按钮，在弹出的对话框中选中“File System Directory”，然后点击“OK”，最后在再次弹出的对话框中输入源码位置“/work/eclipse_projects/u-boot/u-boot-1.1.6_OpenJTAG”，并选中“Search subfolders”。

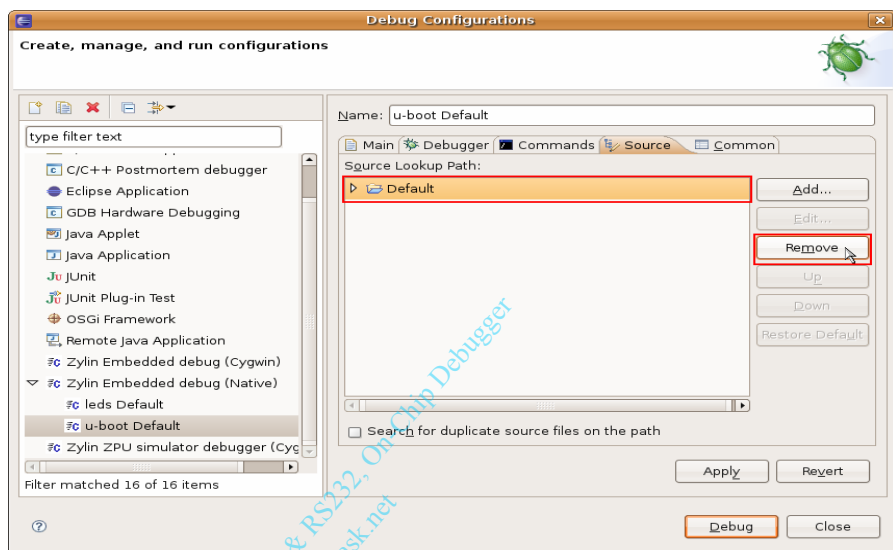


图 3.55 u-boot 工程调试器配置信息(4)——删除默认路径

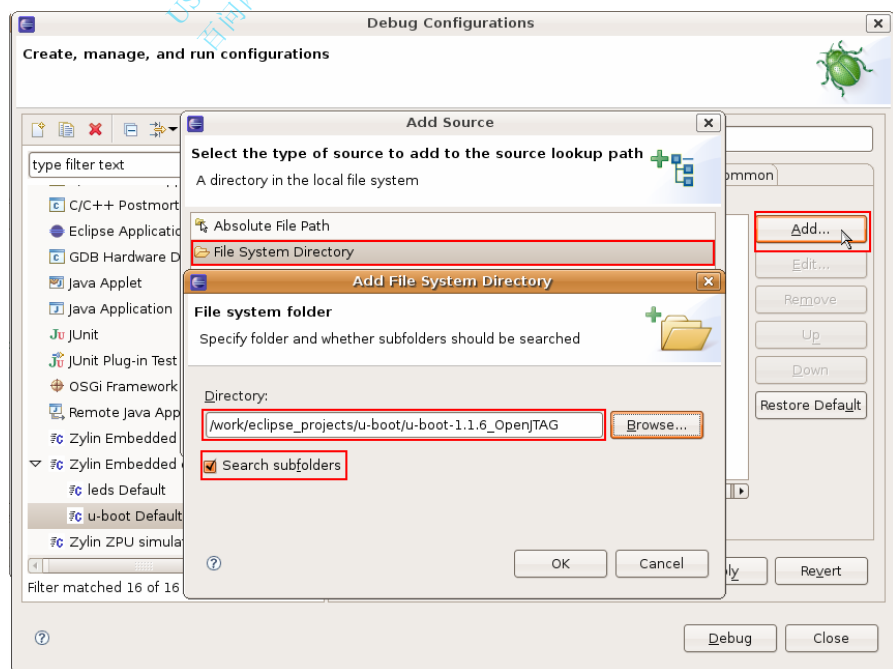


图 3.56 u-boot 工程调试器配置信息(5)——修改源码路径

配置完成后，先启动 OpenOCD，然后启动“u-boot Default”调试器，稍等一会就可以看到如图 3.57 所示的界面，u-boot 停在 start_armboot 函数的第一条指令上。

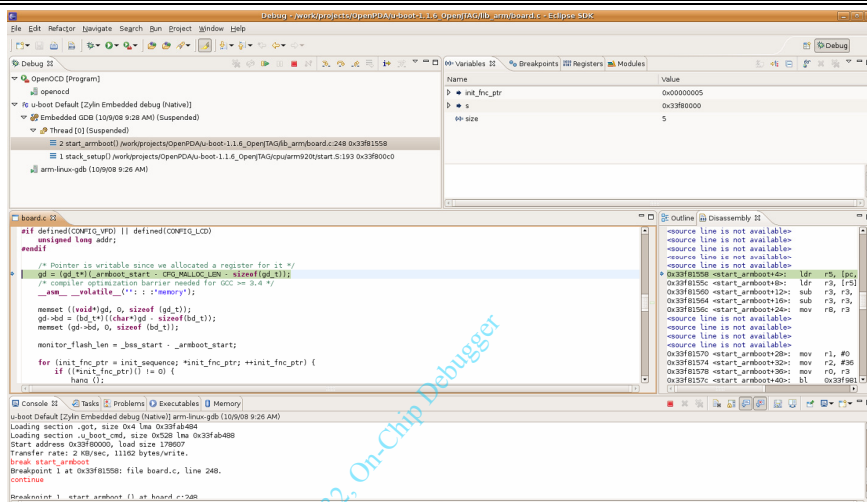


图 3.57 u-boot 工程调试界面

除调试器的设置不同外，u-boot 工程的调试方法与 leds 工程的基本一样，不再赘述。

附录：u-boot 使用简要说明

在串口中进入 U-Boot 控制界面后，可以运行各种命令，比如下载文件到内存，擦除、读写 Flash，运行内存、NOR Flash、NAND Flash 中的程序，查看、修改、比较内存中的数据等。

使用各种命令时，可以使用其开头的若干个字母代替它。比如 tftpboot 命令，可以使用 t、tf、tft、tftp 等字母代替，只要其他命令不以这些字母开头即可。

当运行一个命令之后，如果它是可重复执行的(代码中使用 U_BOOT_CMD 定义这个命令时，第 3 个参数是 1)，若想再次运行可以直接输入回车。

U-Boot 接受的数据都是 16 进制，输入时可以省略前缀 0x、0X。

下面介绍常用的命令：

(1) 帮助命令 help。

运行 help 命令可以看到 U-Boot 中所有命令的作用，如果要查看某个命令的使用方法，运行“help 命令名”，比如“help bootm”。

可以使用“?”来代替“help”，比如直接输入“?”、“? bootm”。

(2) 下载命令。

U-Boot 支持串口下载、网络下载，USB 下载，相关命令有：loadb、loads、loadx、loady 和 tftpboot、nfs，usbslave。

前几个串口下载命令使用方法相似，以 loadx 命令为例，它的用法为“loadx [off] [baud]”。中括号“[]”表示里面的参数可以省略，off 表示文件下载后存放的内存地址，baud 表示使用的波特率。如果 baud 参数省略，则使用当前的波特率；如果 off 参数省略，存放的地址为配置文件中定义的宏 CFG_LOAD_ADDR。

tftpboot 命令使用 TFTP 协议从服务器下载文件，服务器的 IP 地址为环境变量 serverip。用法为“tftpboot [loadAddress] [bootfilename]”，loadAddress 表示文件下载后存放的内存地址，bootfilename 表示要下载的文件名称。如果 loadAddress 省略，存放的地址为配置文件中定义的宏 CFG_LOAD_ADDR；如果 bootfilename 省略，则使用单板的 IP 地址构造一个文件名，比如单板 IP 为 192.168.1.17，则缺省的文件名为 C0A80711.img。

nfs 命令使用 NFS 协议下载文件，用法为“nfs [loadAddress] [host ip addr:bootfilename]”。loadAddress、bootfilename 的意义与 tftpboot 命令一样，host ip addr 表示服务器的 IP 地址，默认为环境变量 serverip。

usbslave 命令：在 PC 段使用 dnw 工具发送文件，u-boot 通过 USB Device 接口接收文件。用法为“usbslave [wait] [loadAddress]”，wait 可以取值 1 或 0，表示是否等待数据传输完成。当 wait 取 0 时，在后台进行下载，这时在 u-boot 仍可执行其他操作。

下载文件成功后，U-Boot 会自动创建或更新环境变量 filesize，它表示下载的文件长度，可以在后续命令中使用“\$(filesize)”来引用它。

(3) 内存操作命令。

常用的命令有：查看内存命令 md、修改内存命令 md、填充内存命令 mw、拷贝命令 cp。这些命令都可以带上后缀“.b”、“.w”或“.l”，表示以字节、字(2 个字节)、双字(4 个字节)

为单位进行操作。比如“cp.l 30000000 31000000 2”将从开始地址 0x30000000 处，拷贝 2 个双字到开始地址为 0x31000000 的地方。

md 命令用法为“md[b, .w, .l] address [count]”，表示以字节、字或双字(默认为双字)为单位，显示从地址 address 开始的内存数据，显示的数据个数为 count。

mm 命令用法为“mm[b, .w, .l] address”，表示以字节、字或双字(默认为双字)为单位，从地址 address 开始修改内存数据。执行 mm 命令后，输入新数据后回车，地址会自动增加，Ctrl+C 退出。

mw 命令用法为“mw[b, .w, .l] address value [count]”，表示以字节、字或双字(默认为双字)为单位，往开始地址为 address 的内存中填充 count 个数据，数据值为 value。

cp 命令用法为“cp[b, .w, .l] source target count”，表示以字节、字或双字(默认为双字)为单位，从源地址 source 的内存拷贝 count 个数据到目的地址的内存。

(4) NOR Flash 操作命令。

常用的命令有查看 Flash 信息的 flinfo 命令、加/解写保护命令 protect、擦除命令 erase。由于 NOR Flash 的接口与一般内存相似，所以一些内存命令可以在 NOR Flash 上使用，比如读 NOR Flash 时可以使用 md、cp 命令，写 NOR Flash 时可以使用 cp 命令(cp 根据地址分辨出是 NOR Flash，从而调用 NOR Flash 驱动完成写操作)。

直接运行“flinfo”即可看到 NOR Flash 的信息，有 NOR Flash 的型号、容量、各扇区的开始地址、是否只读等信息。比如对于本书基于的开发板，flinfo 命令的结果如下：

```
Bank # 1: AMD: 1x Amd29LV800BB (8Mbit)
Size: 1 MB in 19 Sectors
Sector Start Addresses:
00000000 (RO) 00004000 (RO) 00006000 (RO) 00008000 (RO) 00010000 (RO)
00020000 (RO) 00030000      00040000      00050000      00060000
00070000      00080000      00090000      000A0000      000B0000
000C0000      000D0000      000E0000      000F0000 (RO)
```

其中的 RO 表示该扇区处于写保护状态，只读。

对于只读的扇区，在擦除、烧写它之前，要先解除写保护。最简单的命令为“protect off all”，解除所有 NOR Flash 的写保护。

erase 命令常用的格式为“erase start end”——擦除的地址范围为 start 至 end、“erase start +len”——擦除的地址范围为 start 至(start + len - 1)，“erase all”——表示擦除所有 NOR Flash。

注意：其中的地址范围，刚好是一个扇区的开始地址到另一个(或同一个)扇区的结束地址。比如要擦除 Amd29LV800BB 的前 5 个扇区，执行的命令为“erase 0 0x2ffff”，而非“erase 0 0x30000”。

(5) NAND Flash 操作命令。

NAND Flash 操作命令只有一个：nand，它根据不同的参数进行不同操作，比如擦除、读取、烧写等。

“nand info”查看 NAND Flash 信息。

“nand erase [clean] [off size]”擦除 NAND Flash。加上“clean”时，表示在每个块的第一个扇区的 OOB 区加写入清除标记；off、size 表示要擦除的开始偏移地址和长度，如果省

略 off 和 size，表示要擦除整个 NAND Flash。

“nand read[jffs2] addr off size”从 NAND Flash 偏移地址 off 处读出 size 个字节的数据，存放到开始地址为 addr 的内存中。是否加后缀“jffs”的差别只是读操作时的 ECC 校验方法不同。

“nand write[jffs2] addr off size”把开始地址为 addr 的内存中的 size 个字节数据，写到 NAND Flash 的偏移地址 off 处。是否加后缀“jffs”的差别只是写操作时的 ECC 校验方法不同。

“nand read.yaffs addr off size”从 NAND Flash 偏移地址 off 处读出 size 个字节的数据(包括 OOB 区域)，存放到开始地址为 addr 的内存中。

“nand write.yaffs addr off size”把开始地址为 addr 的内存中的 size 个字节数据(其中有要写入 OOB 区域的数据)，写到 NAND Flash 的偏移地址 off 处。

“nand dump off”，将 NAND Flash 偏移地址 off 的一个扇区的数据打印出来，包括 OOB 数据。

(6) 环境变量命令。

“printenv”命令打印全部环境变量，“printenv name1 name2 ...”打印名字为 name1、name2、……”的环境变量。

“setenv name value”设置名字为 name 的环境变量的值为 value。

“setenv name”删除名字为 name 的环境变量。

上面的设置、删除操作只是在内存中进行，“saveenv”将更改后的所有环境变量写入 NOR Flash 中。

(7) 启动命令。

不带参数的“boot”、“bootm”命令都是执行环境变量 bootcmd 所指定的命令。

“bootm [addr [arg ...]]”命令启动存放在地址 addr 处的 U-Boot 格式的映像文件(使用 U-Boot 目录 tools 下的 mkimage 工具制作得到)，[arg ...]表示参数。如果 addr 参数省略，映像文件所在地址为配置文件中定义的宏 CFG_LOAD_ADDR。

“go addr [arg ...]”与 bootm 命令类似，启动存放在地址 addr 处的二进制文件，[arg ...]表示参数。

“nboot [[[loadAddr] dev] offset]”命令将 NAND Flash 设备 dev 上偏移地址 off 处的映像文件复制到内存 loadAddr 处，然后，如果环境变量 autostart 的值为“yes”，就启动这个映像。如果 loadAddr 参数省略，存放地址为配置文件中定义的宏 CFG_LOAD_ADDR；如果 dev 参数省略，则它的取值为环境变量 bootdevice 的值；如果 offset 参数省略，则默认为 0。